

从 StarVLA 到 VLAct： VLA 持续预训练的表示中心路线 调研全文报告

三篇论文（StarVLA 技术报告 · StarVLA- α · VLAct）+ 一个代码库的系统调研：9 份深度报告合订
核心论点——VLM 骨干是 VLA 的一阶设计变量，预训练不是双刃剑，配方决定符号

构建日期：2026-09-05

材料范围：arXiv 2604.05014 · 2604.11757 · 2608.27550；StarVLA 代码库快照 starVLA_dev @ d81fc66；120 篇文献编目

配套材料：7 篇英文 PDF · 7 篇中译 PDF · 29 页 Beamer 幻灯片 · 图 1 时间线 / 图 2 设计空间（assets/）· VLAct 缺失组件的 StarVLA 扩展代码
（code/vlact_ext/）

仓库：github.com/asimfish/awesome_starvla · 许可：CC BY 4.0

目录

1. 01 · VLAct 精读：以表示为中心的 VLA 持续预训练
2. StarVLA 代码库深度分析报告
3. 03 · StarVLA- α 解读：把 VLA 系统的复杂度降下来
4. 04 · StarVLA 技术报告解读：一个乐高式 VLA 代码库的设计契约
5. 05 · 动作头与动作表示：原理、证据与选型
6. 06 · StarVLA 支持的评测基准全景与 VLAct 评测协议
7. 07 · 研究路线图：在 StarVLA / VLAct 之上做什么
8. 08 · 讲稿：四种动作头（FAST / OFT / PI / GR00T）到底是什么
9. 09 · EventVLA 解读：给 StarVLA-OFT 装上稀疏视觉证据记忆



图 2 · VLA 持续预训练的设计空间：七个维度与三篇论文给出的证据



α = StarVLA- α (2604.11757); VLAct = 2608.27550; ST4VLA = 2602.10109。证据出处见 reports/。由 scripts/make_figures.py 生成。

图 2 · VLA 持续预训练的设计空间：七个维度，每格是三篇论文给出的一条证据。

01 · VLAct 精读：以表示为中心的 VLA 持续预训练

项目	内容
论文	Beyond Data Scaling: Representation-Centric Continued Pre-training for Vision-Language-Action Models
arXiv	2608.27550, 2026-08
作者	Senqiao Yang†, Chengyao Wang†, Yuxin Chen, Zixuan Wang, Longxiang Tang, Haokun Gui, Jinhui Ye, Changsheng Lu, Xiaoyang Wu, Mingkan Zhu; 导师 Pengguang Chen, Shu Liu, Zhuotao Tian, Hengshuang Zhao, Bei Yu, Jiaya Jia
项目页	https://starvla.github.io/VLAct
代码库	基于 StarVLA; 骨干 Qwen3-VL-4B; 16 GPU
本仓库文件	英文 PDF · 中文 PDF

0. 一句话

机器人轨迹比图文数据难扩展一个量级，所以在**固定机器人数据预算**下，VLA 持续预训练的目标应该从“拟合更多动作”换成“把有限轨迹蒸馏成可迁移的视觉-动作表征”。VLAct 用三个都很轻的组件（冻结浅层 + caption 混训、三头共监督、部分统一动作空间）做到了这一点：**只换骨干权重、其余全部固定**，下游成功率提升 7.6—21.4 个点，在 RoboCasa-GR1 这个预训练从未见过的人形本体上，20% 数据就超过全量 GR00T-N1.6。

1. 问题设定

作者的出发点是一组事实，而非一个假设：

- 网页图文可以爬取，机器人轨迹只能靠遥操作或专门采集协议产生（DROID、RoboCoin、MolmoAct 都是这样来的）。
- 机器人策略要泛化的空间是组合且连续的：场景 × 物体 × 任务目标 × 本体 × 接触动力学。即便是几十万条轨迹的数据集，覆盖仍然稀疏且不均匀。
- 因此持续预训练不能指望“穷尽覆盖”。给定固定数据预算，决定下游表现的是这些轨迹**如何塑造骨干表征**，而不只是有多少条。

由此引出论文的核心视角：VLM 骨干不是“从通用视觉-语言预训练继承来的固定部件”，而是 VLA 的一阶设计变量。

术语澄清：这里的“持续预训练”

从一个已经预训练好的 VLM 出发，在广泛、多本体的机器人轨迹上训练，然后再做下游任务特定微调。 π_0 、 $\pi_{0.5}$ 、GR00T N1/N1.5 都是这个设定，只是它们叫“VLA pre-training”。VLAct 用更精确的词，但正文里在语境清楚时仍简称 pre-training。这一区分很重要：本文不训练任何从零开始的基础模型。

2. 先导实验：动作监督如何重塑骨干

第 2 节的 pilot study 是全文最有启发性的部分。固定 Qwen3-VL-4B，只改变预训练与微调阶段使用的动作头，在 LIBERO-Plus 和 RoboTwin-Clean 上观察。四种头的定义见附录 I.1：

头	接口	训练目标	推理
FAST	离散自回归 token	next-token 交叉熵	逐 token 生成后逆 tokenizer
OFT	并行连续回归（MLP 读 K 个 action query 的 hidden state）	L1	一次前向
PI	flow-matching action expert	$ v_\phi(A^T, \tau; H) - (A - \epsilon) ^2$	N 步积分
GR00T	双系统：VLM 慢推理 + DiT 快运动模块	同 flow matching，额外条件于状态与本体标识	N 步去噪

两个失效模式：

(a) **离散监督能迁移，但丢信息**。FAST-token 预训练的骨干接 GR00T 头微调，比从零 GR00T 微调略好——说明离散动作 token 确实往骨干里注入了可迁移的结构。但如果保留 FAST 头本身，成功率远低于任何连续头，且预训练也补不回来。离散化丢掉了操作所需的细粒度幅度与时序信息。

(b) **单一连续头会造成 head-specific 表征坍塌**。OFT 预训练 → OFT 微调大涨；同一个 OFT 预训练骨干 → PI 或 GR00T 微调却**低于从零微调**。附录表 8 给了数字：PI 从零微调 60.5，OFT-only 预训练后 55.1 (−5.4)，OFT+GR00T 双头预训练后 63.1 (+2.6)，三头含 PI 预训练后 77.0。作者把这个现象命名为 **decoder lock-in**：动作信息仍在骨干里，但被组织成只有预训练那个头才好解码的几何形态。同头成功率高估了骨干的可复用性。

结论：可迁移的 VLA 骨干需要同时满足“保留细粒度动作信息”和“能被多种头解码”。这直接引出 3.3 节的多头共监督。

3. 方法：三个组件 + 一个微调协议

所有组件**只在持续预训练阶段使用**，目的是塑造骨干；下游用户可以自由选择动作头。

3.1 保护 VLM 先验

- 浅层保护**：冻结整个视觉编码器 + LLM 下半层，只更新上半层 LLM 和动作头。理由是低层负责底层视觉处理与早期视觉-语言对齐（附录图 6 用逐层 attention 可视化支撑：低层关注广泛视觉/空间信息，深层聚焦语义与任务相关区域）。下游微调时全模型解冻。

- **caption 混训**: 每个 minibatch 同时含机器人样本和辅助 VLM 样本, $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{action}} + 0.5 \mathcal{L}_{\text{VLM-CE}}$ 。消融了五类辅助数据 (caption、BBot-QA、Point-QA、Spatial-QA、纯文本指令), caption 是最强的单一锚点。

3.2 多头连续共监督

三个连续头 OFT、PI、GR00T 并联在同一骨干上, 接收同一潜表征 $\mathbf{z}$ 、预测同一 ground-truth 动作块 $\mathbf{a}$:

$$\mathcal{L}_{\text{action}} = \mathcal{L}_{\text{OFT}} + \mathcal{L}_{\text{PI}} + \mathcal{L}_{\text{GR00T}}$$

三个头共享一次骨干前向, 额外开销只是头本身。作者刻意不引入新头或对齐模块, 而是把头的**多样性本身当作监督信号**: 三种不同的解码偏置迫使骨干把动作信息编码成多种参数化都能读取的形式。附录 E 显示这不只改善跨头迁移, 同头微调也涨 1.6–4.3 个点。

3.3 跨本体的部分统一动作空间

三种设计对比 (图 4): 每个本体一个独立头 (监督被隔离); 朴素全统一 (低维本体零填充到公共维度, 把物理含义不同的坐标对齐到一起); **部分统一** (VLAct 采用)。

具体布局是一个固定 20 维输出向量 (附录 I.2):

维度	含义	表示
1–6	双臂本体左臂 (AgileX)	绝对关节角
7–12	双臂本体右臂	绝对关节角
13–18	单臂本体 (Franka)	delta 末端位姿
19	共享夹爪 : Franka 单夹爪与 AgileX 左夹爪	归一化到 [0,1]
20	双臂本体右夹爪	归一化到 [0,1]

每个样本只在本体的激活维度上计算 loss, 非激活维 mask 掉。没有本体适配器、路由模块或本体条件解码器。

wrap-aware loss: 绝对关节角是周期量, 标准回归把 179° 和 −179° 当成差 358°。数据侧先把角度 wrap 到 $[-\pi, \pi]$: $\mathbf{a}_{\text{wrap}} = (\mathbf{a} + \pi) \bmod 2\pi - \pi$; loss 侧再对残差 wrap: $\delta_{\text{wrap}} = ((\hat{\mathbf{a}} - \mathbf{a}) + \pi) \bmod 2\pi - \pi$, $\mathcal{L}_{\text{wrap}} = |\delta_{\text{wrap}}|$, 加到每个头各自的目标上。对 PI/GR00T, 它作用在最终生成的动作样本上, 而不是中间的噪声或速度目标。只用于绝对关节角维度。

3.4 数据、训练与微调协议

- 预训练数据全开源: DROID (v1.0.0 + v1.0.1)、InternData-A1、RoboCoin、MolmoAct, 加 caption 数据。两个本体: Franka 单臂 (7 维: 6 维 delta EE + 1 夹爪) 与 AgileX 双臂 (14 维: 12 关节角 + 2 夹爪)。
- 数据清洗 (附录 H): 删无效任务名; delta EE 动作按 FPS 换成每秒量, 平移速度 > 0.5 或旋转速度 > 1.0 的步标记无效, **按步 mask 而非丢整条轨迹**, 仅当一个 chunk 内无效比例 > 0.5 才丢弃; 关节角先删 $[-2\pi, 2\pi]$ 之外的值再 wrap; 夹爪去极值后逐数据集 min-max 归一化。
- 代码库 StarVLA, 骨干 Qwen3-VL-4B, 16 GPU。
- **微调协议**: 丢弃预训练头和 caption 流, 重新随机初始化任务头, 全参数解冻, 与每个基线使用完全相同的下游数据、优化器、训练预算。所有 VLAct 对比中, 唯一变量是骨干权重。

4. 主结果

4.1 汇总表

基准	设定	VLAct	同骨干基线 Qwen3VL-OFT	最强外部对手
LIBERO-Plus	7 类扰动均值	82.6	75.0	ABot-M0 80.5
VLA-Arena	11 套件加权	54.8	33.4	π 0.5 44.3
RoboTwin 2.0 Base	Clean / Random	80.5 / 41.5	61.7 / 10.5	X-VLA 70.0 / 39.0
RoboTwin 2.0 Data Scaling	Clean / Random	92.5 / 90.8	88.2 / 88.3	HoloBrain-0 91.9 / 92.3; Fast-WAM 91.9 / 91.8
DOMINO	SR / MS	18.50 / 34.20	10.86 / 30.49	π 0.5 9.63 / 26.17
RoboCasa-GR1 (未见本体)	全量	54.0	48.8	GR00T-N1.6 47.6
RoboCasa-GR1	20% 数据	49.5	—	已超全量 GR00T-N1.6
RoboDojo (ARX X5, 未见本体)	分数 / 成功率	10.66 / 7.60	StarVLA- α 6.40 / 3.24	DM0.5 24.90 / 19.34
真机 Franka 单臂短程	4 任务均值	92.5	77.5	—
真机 Franka 双臂协调	5 任务均值	72.0	44.0	—

4.2 逐基准要点

- **LIBERO-Plus**: 最大增益在 Camera (+26.9)、Robot、Noise、Layout 四个维度, Language 维度反而比基线低 5.5 (81.5 vs 87.0)。说明表示中心预训练主要强化了视觉-空间鲁棒性, 对语言扰动没有帮助甚至略有代价。

- **RoboTwin 2.0**: Base 设定只用 50 条 clean 轨迹/任务却在 Random 测试集上从 10.5 涨到 41.5，clean→random 的泛化提升最能体现骨干的作用。三种下游头在 Data Scaling 下 Clean 分别为 OFT 92.5、GR00T 89.6、PI 93.0，相差 3.4 以内，支撑"迁移的是骨干表征而非特定头"。但 Base/Random 下 OFT 41.5 vs GR00T 22.9 vs PI 23.7，头之间差距仍然很大。
- **真机**: 预训练只用单臂数据，却把双臂协调从 44.0 拉到 72.0，叠裤子 +30、叠毛巾 +30。长程 scoop beans 从 33.3 到 80.0——基线经常跳过"舀豆子"直接空勺倒。OOD 物体替换场景基线跌到 46—47，VLAct 保持在 82—83。
- **RoboDojo**: 35 个策略中成功率第 6、分数第 8。超过所有四个显式标注的 WAM (X-WAM 7.69/3.83 最强)，也超过 Xiaomi-Robotics-0、GalaxeaVLA G0、LingBot-VLA、ABot-M0。相对同骨干的 StarVLA-α 涨 4.26 分 / 4.36 个点，增益主要在 Precision 和 Long-Horizon；**Memory 维度只有 0.66 / 0.56，几乎为零。**

5. 消融汇总

组件	设定	结果	来源
浅层保护	全骨干更新 → 冻视觉编码器 → 冻视觉编码器 + 下半 LLM	LIBERO-Plus 78.9 → 81.3 → 82.6；RoboTwin 77.1 → 79.3 → 80.5	表 6
辅助数据	robot-only → +caption	75.0 → 82.6 (caption 单源最强；Spatial-QA、Point-QA、BBox-QA、纯文本指令均有提升；全混合 82.5 略低于 caption-only，因固定步数下稀释了 caption 采样)	图 8
多头 vs 单头 (跨头)	PI 微调：从零 60.5 / OFT-only 55.1 / OFT+GR00T 63.1 / 三头 77.0	decoder lock-in 被双头缓解	表 8
多头 vs 单头 (同头)	OFT 78.8 → 80.5；PI 75.4 → 77.0；GR00T 71.7 → 76.0	+1.7 / +1.6 / +4.3	表 9
动作空间	独立头 → 统一头 → 统一表征	RoboTwin 78.5 → 79.5 → 80.5；LIBERO-Plus 81.1 → 81.4 → 82.6	表 12
关节角处理	原始回归 → 数据侧 wrap → +wrap loss	RoboTwin Clean 75.5 → 78.6 → 80.5	表 10
异构数据	+20K RealOmin UMI 轨迹 (仅轻量过滤)	LIBERO-Plus 82.6 → 83.7	表 11

一个容易被忽略的结论：**纯文本指令数据** (Nemotron SFT) 也能提升下游操作成功率。它与机器人感知、控制、视觉 grounding 都无关，所以增益只能解释为"把可训练层拉回基础模型的工作区间 + 让梯度更多样"，而不是任务层面的直接迁移。这为"辅助数据 = 表征保护而非额外监督"的解释提供了对照实验。

6. 与 StarVLA-α 的对话：预训练是双刃剑，还是配方问题

同一团队四个月前的 **StarVLA-α** (2604.11757) 报告了一个相反方向的结论：在 Qwen3-VL-4B + MLP 头上做朴素动作预训练是"双刃剑"——OXE 预训练把 RoboTwin Clean 50×50 从 50.3 拉到 30.2，把 RoboCasa-GR1 24×10 从 9.8 拉到 1.2；即便是同域的 InternData-A1，也只在 RoboTwin 低数据段有帮助，在 GR1 上仍然掉点。

VLAct 恰好回应了这个矛盾：用的数据同样包含 InternData-A1，同样的骨干，但把"朴素拟合动作"换成"保护先验 + 多头 + 统一动作空间"后，GR1 (预训练未见) 从 48.8 涨到 54.0，20% 数据就到 49.5。两篇放在一起读，结论是：**动作预训练是否有用，取决于它如何塑造表征，而不是数据量本身。**这也是 VLAct 标题"Beyond Data Scaling"的实证含义。

另一个值得对照的点：StarVLA-α 表 6 显示"简单零填充到 32 维"优于 RDT 动作空间和多头设计 (GR1 57.3 vs 52.3 / 53.5)，而 VLAct 表 12 显示"部分统一"优于"朴素统一">"独立头"。两者并不冲突——StarVLA-α 的对比对象是"每本体独立头"和 RDT 的物理统一空间，VLAct 进一步指出：在零填充之上，把物理同义维度 (夹爪) 显式对齐、把物理不同义维度 (不同运动学的手臂) 分开，还能再拿 1 个点。

7. 批判性评价

做得好的地方

1. **归因干净**。所有对比中动作头、初始化、下游数据、优化器、预算全部相同，只换骨干权重。7.6—21.4 个点的增益可以直接归到持续预训练配方上。
2. **组件足够简单，能被验证也能被推翻**。没有新模块、新 head、新 loss family。浅层冻结是一个正则表达式，多头是三个已有 head 并联，动作布局是一张 20 维的表。任何人在 StarVLA 上都能复现。
3. **pilot study 把"动作头选择"这个长期争论换了问法**。从"哪个头最好"变成"如何让骨干对头不敏感"。decoder lock-in 是一个可测量的现象，表 8 给出了最小可复现的对照。
4. **成本诚实**。开源数据 + 16 GPU，与榜上工业系统的差距被明确标注，并没有宣称绝对 SOTA。

局限与未回答的问题

1. **规模单一**。只有 4B。作者自己指出更大 VLM 的最优配方可能不同。StarVLA-α 表 10 显示 4B→8B 增益 <1%，但那是无预训练设定，持续预训练下的 scaling 曲线仍是空白。
2. **Memory 维度接近零**。RoboDojo 上 0.66 / 0.56，与 DM0.5 的 47.74 / 47.44 差了两个量级。单帧输入、无历史的设计天然不适合需要记忆的任务，这个短板配方本身不解决。

3. "头无关"有边界。RoboTwin Base/Random 下 OFT 41.5 而 GR00T 22.9、PI 23.7；LIBERO-Plus 的 Language 维度掉了 5.5 个点。低数据 + 强扰动时，头之间的差距没有被抹平。
4. Data Scaling 对比不可控。训练算力、调度、checkpoint 选择在各方法间不统一（作者已标注）。RoboDojo 榜单也不按算力归一化。
5. 真机每任务 10 次 rollout。初始状态固定共享保证了可比性，但 10 次的方差仍然大，差距在 10—20 个点以内的结论应谨慎对待。
6. 三个头共训的代价没有量化。论文说"只增加头的计算"，但 PI 和 GR00T 各自带一个 DiT，预训练阶段的显存和吞吐开销没有给出数字。
7. 同名基线数字不一致。VLAct 表 2 的内部基线 Qwen3VL-OFT 在 RoboTwin Base Clean 为 61.7，而 StarVLA 仓库 `examples/simBenchmarks/Robotwin/README.md` 报告的同配置 OFT 基线为 50.38 (48/50 任务)，StarVLA- α 表 1 为 50.3。三处是不同训练轮次，比较 VLAct 的增益幅度时应以论文内部同轮次的对照为准（详见 06 · 基准生态 §2.3、§3）。
8. 20 维布局是手工设计的。它只覆盖两类本体（6-DoF 双臂关节角 + 6-DoF 单臂 delta EE）。加入人形、灵巧手、移动底盘时怎么扩展，论文没有讨论。GR1 和 ARX X5 的迁移是在下游微调时重新初始化头做到的，不是零样本。

8. 可复现性

- 代码与模型：论文声明全部开源（StarVLA 训练脚本 + checkpoint）。写作本报告时（2026-09）StarVLA 主仓库尚无名为 VLAct 的 example 目录，建议关注 <https://starvla.github.io/VLAct> 与 HuggingFace StarVLA 组织。
- 数据全部公开：DROID、InternData-A1、RoboCoin、MolmoAct、LLaVA-ReCap-CC3M、LLaVA-OneVision、RefCOCO、COCO-ReM、PixMo-Points、RoboPoint、SenseNova-SI-800K、Nemotron-SFT-Instruction-Following-Chat-v2。
- 配方各项在 StarVLA 代码中的现状（已有 / 部分 / 缺失）见 02 · StarVLA 代码库解析 第 9 章。

9. 对后续研究的直接启示

- 把"骨干如何被动作监督重塑"当作可测量对象：decoder lock-in、表征漂移（RefCOCO IoU 随训练步数下降）、跨头线性可读性，都可以做成诊断指标。
- 多头共监督是一个廉价的正则化器，可以推广到"多任务头"（动作 + 未来帧预测 + 空间 QA）。
- 部分统一动作空间的"物理同义维度对齐"原则，可以推到人形与灵巧手：哪些维度该共享、哪些该 mask，本身是一个可学习的问题。
- Memory 与长程是现有配方未触及的空白，也是 RoboDojo 榜单上拉开差距的地方。

具体的研究路线图见 07 · 研究路线图。

StarVLA 代码库深度分析报告

分析对象： /Users/liyufeng/Desktop/research/starVLA_code (GitHub starVLA/starVLA, MIT, 分支 starVLA_dev , HEAD d81fc66 , 2026-09-04)。
参考：技术报告 arXiv 2604.05014 (StarVLA)、arXiv 2608.27550 (VLAct)。所有结论来自逐文件阅读，行号以本地快照为准。

一句话结论与关键数字

StarVLA 是一个“VLM 骨干 + 可插拔动作头”的 VLA 组装框架：模型层抽象清晰 (forward() / predict_action() 双接口 + 注册表)，数据层完整移植了 GR00T 的 LeRobot 管线，训练层是三份复制粘贴的 Accelerate/DeepSpeed 显式循环；VLAct 配方六项中，冻结 (部分)、caption 共训 (已有)、丢头重训 (已有) 可直接配置，多头共监督与 wrap-aware loss 完全缺失，统一 20 维动作布局只有零散的 mask 基础设施。

指标	数值 (实测)
Python 文件 / 总行数	261 个 / 57,625 行 (starVLA/ 139 个 25.7k+8.7k+3.4k 行; examples/ 98 个 16.8k 行; deployment/ 15 个; tests/ 7 个)
已注册框架类	28 个 (VLM4A 20、WM4A 6、VM4A 2), 31 个注册键 (含别名 QwenFM 、 Pi0 、 Pi05)
动作头实现文件	11 个 (starVLA/model/modules/action_model/*.py , 不含 __init__)
VLM 接口文件	10 个, get_vlm_model 按模型名分派 9 个分支
示例目录	25 个 (simBenchmarks 13、realRobots 5、modelExtensions 6、human2robots 1); 12 个 model2*_interface.py ; 16 个外部 data_registry
训练入口	4 个脚本 (train_starvla.py 528 行、train_starvla_cotrain.py 514 行、train_starvlm.py 375 行、train_starvln.py 252 行)
测试	7 个文件、41 个测试函数、1,067 行; 无 CI workflow
Git	131 次提交 (浅历史, 起于 2026-02-21)、40 位作者

1. 目录总览与设计哲学

1.1 顶层结构

目录	关键文件	职责
starVLA/model/framework/	base_framework.py 、 share_tools.py 、 VLM4A/*.py 、 WM4A/*.py 、 VM4A/*.py	框架层: 把骨干 + 动作头组装成一个 baseframework 子类; 每个文件即一个“论文架构图”
starVLA/model/modules/	vlm/ 、 world_model/ 、 action_model/ 、 dino_model/ 、 projector/	可复用模块: VLM 接口包装、世界模型包装、动作头、DINO、QFormer
starVLA/model/tools.py	Registry 、 FRAMEWORK_REGISTRY 、 FrameworkTools	注册表、反归一化工具、可训练模块发现
starVLA/dataloader/	lerobot_datasets.py 、 gr00t_lerobot/ 、 vlm_datasets.py 、 umi_datasets.py 、 qwenvl_llavajson/	数据层: LeRobot 机器人数据、LLaVA-json VLM 数据、UMI 适配器
starVLA/training/	train_starvla*.py 、 trainer_utils/	训练循环、冻结/分组学习率/权重加载工具、配置访问追踪
starVLA/config/	deepseeds/*.yaml 、 training/*.yaml	Accelerate+DeepSpeed 启动配置; 5 份历史训练 yaml
deployment/	model_server/ 、 docker/ 、 upload/	WebSocket / ZMQ 策略服务器、Docker 镜像、HF 上传脚本
examples/	simBenchmarks/ 、 realRobots/ 、 modelExtensions/ 、 human2robots/	每个基准/机器人一个目录: train_files/ (yaml、sh、data_registry/、modality.json) + eval_files/ (model2*_interface.py 、 run_policy_server.sh)
docs/	starVLA_guideline.md 、 faq.md 、 WM4A.md 、 VM4A.md 、 agent_skills/	上手指南、FAQ、两条非 VLM 骨干路线、agent skill 模板
tests/	7 个 unittest 文件	配置覆盖、ZMQ 服务、MiniCPM、图像预处理、单进程安全

1.2 设计哲学 (代码中的落点)

- top-down 分解、高内聚低耦合**: build_framework(cfg) (starVLA/model/framework/base_framework.py:L85-119) 是模型的唯一构建入口, 按 cfg.framework.name 查 FRAMEWORK_REGISTRY ; _auto_import_framework_modules (L60-82) 用 pkgutil 扫描 framework/ 下所有子包并 import, 触发 @FRAMEWORK_REGISTRY.register("...") 装饰器 (starVLA/model/tools.py:L136-164)。新增框架只需新增一个文件, 无需改任何 import 列表。
- forward() / predict_action() 双接口**: baseframework (base_framework.py:L126-321) 继承 transformers.PreTrainedModel , 只约束两个方法: forward(examples: List[dict]) -> {"action_loss": Tensor} (L151-166) 和 predict_action(examples) -> {"normalized_actions": np.ndarray[B,T,D]} (L168-180)。两者都直接吃“原始样本 dict” (PIL 图像、字符串指令、numpy 动作), 模型内部自己做 tokenize / processor, 训练与部署输入形态一致。

3. **dataloader 返回模型无关的原始 dict**: `LeRobotSingleDataset._pack_sample` (`starVLA/dataloader/gr00t_lerobot/datasets.py:L1379-1436`) 产出 `{"action": np.ndarray[T,D], "image": List[PIL], "lang": str, "robot_tag": str, "state"?: np.ndarray[1,S]}`; `collate_fn` 就是 `return batch` (`lerobot_datasets.py:L21-22`)。例外见 §3.6: VLM 数据在 dataloader 内已 tokenize。
4. ****bar/ 忽略目录**: `.gitignore:L176,L186` 含 `bar` 与 `**bar/`; `pyproject.toml` 的 `exclude` 也含 `**bar**`。约定用户把私有脚本放在任何 `bar/` 子目录下。`lerobot_datasets.py:L124` 的 `smoke test` 默认路径就指向 `.../train_files/bar/starvla_cotrain_libero.yaml`。
5. **单文件 smoke test**: 每个框架文件末尾带 `if __name__ == "__main__":` (如 `Qwen0FT.py:L353-408`): 加载 yaml、构造模型、造假样本跑一次 `forward` 与 `predict_action`。VLM 接口 (`Qwen3.py:L174-199`)、`dataloader` (`lerobot_datasets.py:L119-162`) 同样可单独运行。
6. **Config-as-API**: 每个框架定义一个 `*DefaultConfig` dataclass (如 `Qwen0FTDefaultConfig`, `Qwen0FT.py:L65-101`)，用 `merge_framework_config` (`share_tools.py:L186-258`) 与 yaml 的 `framework`: 子树合并, yaml 优先、冗余键保留。

1.3 类关系

```
classDiagram
    class PreTrainedModel
    class baseframework {
        +forward(examples) dict
        +predict_action(examples) dict
        +forward_vlm(batch) dict
        +from_pretrained(ckpt, config_overrides)
    }
    class QwenVl_OFT {
        qwen_vl_interface
        action_model: L1RegressionActionHead
        action_token_id : int
    }
    class QwenVl_Fast {
        action_model: Fast_Action_Tokenizer
    }
    class Qwen_PI_v3 {
        project_layers: ModuleList
        action_model: LayerwiseFlowmatchingActionHead
    }
    class Qwen_GR00T {
        action_model: FlowmatchingActionHead
    }
    class _Qwen3_VL_Interface {
        model: Qwen3VLForConditionalGeneration
        processor
        +build_qwenVl_inputs(images, instructions, solutions)
    }
    class DiT
    PreTrainedModel <|-- baseframework
    baseframework <|-- QwenVl_OFT
    baseframework <|-- QwenVl_Fast
    baseframework <|-- Qwen_PI_v3
    baseframework <|-- Qwen_GR00T
    QwenVl_OFT --> _Qwen3_VL_Interface
    Qwen_PI_v3 --> _Qwen3_VL_Interface
    Qwen_GR00T --> _Qwen3_VL_Interface
    Qwen_PI_v3 --> DiT : LayerwiseFM 36 layers
    Qwen_GR00T --> DiT : DiT-B 16 layers
```

2. 配置系统

2.1 全局 config 对象的构建链

```
OmegaConf.load(--config_yaml)          train_starvla.py:L508
→ normalize_dotlist_args(cliargs)      trainer_tools.py:L31-54
→ OmegaConf.merge(cfg, OmegaConf.from_dotlist(...)) train_starvla.py:L509-511
→ apply_config_compat(cfg)             share_tools.py:L418-524
→ wrap_config(cfg) -> AccessTrackedConfig config_tracker.py:L476-478
→ build_framework(cfg) 内 merge_framework_config(DefaultConfig, cfg) share_tools.py:L186-258
```

- **点路径覆盖**: `argparse.parse_known_args()` 只认 `--config_yaml`, 其余参数交给 `normalize_dotlist_args`: `['--framework.qwenVl.base_vlm', 'X']` → `'framework.qwenVl.base_vlm=X'`; 孤立的 `--flag` → `'flag=true'` (`trainer_tools.py:L43-51`)。随后 `OmegaConf.from_dotlist` 解析并 merge, 因此任何未在 yaml 中出现的新键都会被“加进”全局 config (README 所说 “只添加到 config, 行为由框架决定”)。
- **版本兼容层**: `apply_config_compat` (`share_tools.py:L418-524`) 把 `action_horizon` 与旧键 `future_action_window_size` 互补 (`action_horizon = future + 1`, `L472-487`), 自动填 `diffusion_model_cfg.output_dim`、`cross_attention_dim`、`action_hidden_dim`、`past_action_window_size=0`, 并盖章 `version_id: "0.21"`。
- **访问追踪**: `AccessTrackedConfig` (`config_tracker.py:L7-473`) 用 `__getattr__` 记录被读取过的键; `save_accessed_config` (`L434-447`) 只把访问过的叶子写到 `<run_dir>/config.yaml`, 完整合并结果另存 `config.full.yaml` (`train_starvla.py:L215-227`)。从 `from_pretrained` 读取的是

精简版 `config.yaml` (`share_tools.read_mode_config`, L357–399)。

- **框架默认值合并**: `merge_framework_config` 把 `dataclass` 默认值转 `OmegaConf`, 与 `yaml` 的 `framework` 子树 `merge` (`yaml` 赢), 并处理 `AccessTrackedConfig` 的子节点缓存失效 (L232–248)。
- **推理期覆盖**: `baseframework.from_pretrained(ckpt, config_overrides=...)` (`base_framework.py`:L254–321) 与 `server_policy.py --config_override key=value` 走 `merge_config_overrides` (L27–57), 可在不改 `checkpoint` 的前提下改配置 (`tests/test_config_overrides.py` 覆盖此路径)。

2.2 starVLA/config/ 内容

文件	内容
<code>deepseeds/deepspeed_zero2.yaml</code>	Accelerate 配置: <code>distributed_type: DEEPSPEED</code> , <code>num_processes: 8</code> , 指向 <code>ds_config.yaml</code>
<code>deepseeds/ds_config.yaml</code>	实为 JSON: <code>ZeRO stage 2</code> , <code>bf16</code> , <code>gradient_accumulation_steps: 1</code> , <code>gradient_clipping: 1.0</code> , <code>train_micro_batch_size_per_gpu: auto</code>
<code>deepseeds/deepspeed_zero3.yaml</code> + <code>zero3.yaml</code>	<code>ZeRO stage 3</code> , <code>stage3_gather_16bit_weights_on_model_save: true</code> , 无 <code>num_processes</code>
<code>deepseeds/zero2.yaml</code>	另一份不带外部 <code>json</code> 的 <code>ZeRO-2 Accelerate</code> 配置 (<code>mixed_precision: bf16</code>)
<code>training/starvla_cotrain_libero.yaml</code>	<code>QwenGR00T</code> + <code>Qwen2.5-VL-3B</code> , 历史版本
<code>training/starvla_cotrain_oxe.yaml</code>	<code>QwenPI</code> + <code>OXE</code> (<code>bridge_rt_1</code>)
<code>training/starvla_train_adapter.yaml</code>	<code>QwenAdapter</code> (<code>VLA-Adapter</code> 头)
<code>training/starvla_train_discrete_diffusion{,_real}.yaml</code>	<code>QwenDiscreteDiffusion</code> , <code>RoboTwin 14D</code> / <code>FastUMI 10D</code>

实际使用的训练 `yaml` 都在 `examples/*/train_files/` (31 个), `starVLA/config/training/` 更像历史遗留; `pyproject.toml` 还把 `starVLA.config` 排除在 `package` 之外。

3. 数据管线

3.1 数据流

```
flowchart LR
    subgraph Registry
        A[examples/**/train_files/data_registry/data_config.py] -->|discover_and_merge| B[ROBOT_TYPE_CONFIG_MAP / DATASET_NAMED_MIXTURES]
    end
    B --> C[get_vla_dataset data_mix]
    C --> D[LeRobotSingleDataset x N]
    D --> E[LeRobotMixtureDataset]
    E -->|_getitem_| F["{image: List[PIL], lang, action[T,D], state[1,S], robot_tag}"]
    F --> G[DataLoader collate=identity]
    G --> H[framework.forward examples]
    H --> I[VLM interface build_qwenvl_inputs]
    I --> J[hidden_states]
    J --> K[action head loss]
    K -->|save_dataset_statistics| L[run_dir/dataset_statistics.json]
    L --> M[PolicyNormProcessor.unapply_actions at serving]
```

3.2 lerobot_datasets.py 与 gr00t_lerobot/

- `get_vla_dataset(data_cfg)` (`lerobot_datasets.py`:L77–115): 从 `DATASET_NAMED_MIXTURES[data_mix]` 读 [(`dataset_dir`, `weight`, `robot_type`)], 去重后为每个条目调 `make_LeRobotSingleDataset` (L24–75), 再包成 `LeRobotMixtureDataset`。`robot_type` 查 `ROBOT_TYPE_CONFIG_MAP` 得到 `DataConfig` (含 `modality_config()`、`transform()`、`embodiment_tag`)。`DataConfig` 可选实现 `make_dataset(...)` 工厂钩子替换数据集类 (L55–65)。
- **注册表自动发现**: `gr00t_lerobot/registry.py`:L89–102 用 `examples/**/train_files/data_registry` `glob` 找到 16 个目录, `discover_and_merge` (L116–144) 逐个 `import` 并 `update` 三个全局 `dict`。基础注册在 `data_config.py`:L1082–1096 (10 个 `robot_type`) 与 `mixtures.py`。
- **单样本 schema** (`datasets.py`:L1379–1436):
 - `image: List[PIL.Image]`, 按 `video_keys` 顺序, 每路取 `delta_indices=[0]` 那一帧并 `resize(image_size)` (默认 224×224, 可由 `data_cfg.image_size` 改);
 - `lang: str`, 来自 `language_keys[0]`;
 - `action: np.concatenate([data[k] for k in action_keys], axis=1).astype(float16)`, 形状 `[len(action_indices), D]`;
 - `state`: 仅当 `data_cfg.include_state` 为真时加入, 形状 `[1, S]`;
 - `robot_tag`: `embodiment tag` 字符串。

- **action chunk 与 horizon:** chunk 长度由 DataConfig 的 `action_indices = list(range(N))` 决定 (LIBERO 为 `range(8)`, `examples/simBenchmarks/LIBERO/train_files/data_registry/data_config.py:L39`); 越界步用 `retrieve_data_and_pad` (`datasets.py:L1544-1590`) 按 `first_last / zero` 策略填充。模型侧再取最后 `action_horizon` 步: `actions[:, -self.action_horizon:, :]` (`Qwen0FT.py:L219`), 因此 yaml 的 `action_horizon` 必须 $\leq \text{len}(\text{action_indices})$ 。`data_cfg.drop_incomplete_action_chunks` (`datasets.py:L2228-2260`) 可只采样完整 chunk 的起点。
- **动作模式:** `data_cfg.action_mode` $\in \{\text{abs}, \text{delta}, \text{rel}\}$ (`_apply_action_mode`, `datasets.py:L1238-1277`): `delta` 为相邻步差分且首步减 `state`, `rel` 为整段减 `state[0]`; 统计量随模式分开缓存。
- **归一化统计:** `calculate_dataset_statistics` (`datasets.py:L76-121`) 读全部 parquet, 按列算 `mean/std/min/max/q01/q99`, 缓存到数据集目录 `meta/stats_gr00t.json` (`LE_ROBOT_STATS_FILENAME`, L65; 带 `__format_version=2` 与 `__cache_config`, L162-225)。`StateActionTransform` (`transform/state_action.py:L277-`) 在 `set_transforms_metadata` 后按 `normalization_modes` 逐 key 归一化, `Normalizer` (L98-213) 支持 `q99` (映射到 `[-1,1]` 再 clamp 到 `[-2.2,2.2]`)、`mean_std`、`min_max`、`scale`、`binary` 五种。
- **多数据集合并统计:** `LeRobotMixtureDataset.update_metadata` (`datasets.py:L2700-2735`) 按 `embodiment tag` 分组, 同 tag 的多个数据集用 `merge_metadata` (L2653-2698, 要求 `modality` 配置完全一致) 与 `compute_overall_statistics` (L2543-2651) 合并: `mean/std` 按采样权重加, `q01/q99` 默认 `percentile_mixing_method="min_max"` (取各集 `q01` 的 `min`、`q99` 的 `max`, L2194-2196)。训练开始时 `save_dataset_statistics` (L2737-2846) 把每个 tag 的 `action/state` 统计 + `mask` (`generate_action_mask_for_used_keys`, L2118-2160: 仅 `binary` 维为 `False`) + `num_transitions/num_trajectories` 写到 `<run_dir>/dataset_statistics.json`, 供推理反归一化。
- **混合采样权重:** `LeRobotMixtureDataset.__init__` (L2274-2325): 数据集权重 = 配置权重 (`balance_dataset_weights=True` 时再乘数据集长度) 后归一化; 轨迹权重默认均匀 (`balance_trajectory_weights` 时按长度)。`sample_step` (L2386-2411) 用 `hash(epoch, index, seed)` 播种, 先按数据集权重抽数据集、再抽轨迹、再抽起点。`__len__` (L2478-2541) = 权重为 1.0 的"主数据集"中 `len/weight` 的最大值。`build_dataloader` 默认两个 `balance` 开关均为 `False` (`dataloader/_init__.py:L44-45`)。
- **跨本体动作维度:** 核心管线不做维度对齐。每个 DataConfig 自己决定 `action_keys` 的拼接顺序 (`AgileX` 为 `left_joints, right_joints, left_gripper, right_gripper` $\rightarrow$ 14 维, `data_config.py:L852-857`; `LIBERO` 7 维)。同一 mixture 若混入不同维度的本体, 样本级别可以共存 (`batch` 是 `list`), 但框架侧 `torch.tensor(np.array(actions))` (`Qwen0FT.py:L216`) 要求同 `batch` 形状一致, 会直接报错。现有的对齐手段都在框架/适配器层: `PI0._pad_array_2d` 零填到 32 维无 `mask` (`PI0.py:L231-239, L287`); `umi_datasets.UMISampleAdapter._fit_matrix` 右下角填充并生成 `action_mask[T,D]` (`umi_datasets.py:L86-114`); `MiniCPMRobotManip._to_80d` 把 10 维 `EE6D` 写进 80 维统一布局的 `[7:17]` 槽位并生成 `mask` (`MiniCPMRobotManip.py:L141-150`)。技术报告 §7 提到的"统一 32 维 padding 多基准共训"在代码中未找到对应实现。

3.3 umi_datasets.py

对 `LeRobotMixtureDataset` 的安全包装 (`make_umi_dataloader`, L222-267): 从 `datasets.vla_data` 与 `framework.action_model` 解析并校验 `action_horizon/action_dim/state_dim` (不一致即抛错, L48-58); 拒绝混合 `action_semantics` 不同的 mixture (L235-247); `UMISampleAdapter` (L117-212) 对每个样本做形状规整、NaN 检查、`max_abs_action`、静止动作剔除, 失败时以奇数步长重采样, 输出附带 `action_mask`、`image_mask`、`state_mask`。这是仓库里唯一在 `dataloader` 层输出 `action_mask` 的路径, 目前只被 `Qwen0FT.masked_ll_loss` 消费。

3.4 vlm_datasets.py 与 qwenvl_llavajson/

- 数据注册在 `qwenvl_llavajson/qwen_data_config.py`: `data_dict = {sharegpt4v_coco, r2r, rxr}`, `dataset_use: "a,b%50"` 支持 `%N` 采样率后缀 (L37-56)。
- `LazySupervisedDataset` (`vlm_datasets.py:L255-`) 读 LLaVA 格式 json (`image / conversations`), `_build_messages` (L142-211) 把 `<image>` 占位符替换为 `PIL`, `preprocess_qwen_visual` (L213-252) 用 `processor.apply_chat_template(tokenize=True)` 得到 `input_ids/pixel_values`, `labels` 只保留 `assistant` 回答区间 (依据硬编码 `token id 77091` 与 `151645`, L238-246)。`get_rope_index_{2,25,3}` (`rope2d.py`) 按 `model_type` 计算 `mRoPE position_ids`。
- `make_vlm_dataloader` (L704-732) 用 `cfg.framework.qwenvl.base_vlm` 的 `processor`, `DataCollatorForSupervisedDataset` 左 pad 成 `tensor batch`, `num_workers=4` 硬编码。
- 结论: `VLM` 分支在 `dataloader` 内完成 `tokenize`, 是"原始 dict"原则的例外, 也把它绑定在 `Qwen` 词表上 (`Gemma4/MiniCPM` 骨干无法直接复用)。

3.5 VLM 数据与机器人数据的混批方式

不是同一 `batch` 内混合, 而是两个独立 `DataLoader` (`train_starvla_cotrain.py:prepare_data`, L68-77), 每步各取一个 `batch` (`_get_next_batch`, L254-274), 分别 `forward/backward` (L358-426)。混合比例由 `per_device_batch_size` (`LIBERO` 默认 `vla 16 : vlm 4`) 间接控制, `VLM` 侧 `loss` 乘 `trainer.loss_scale_vlm` (默认 0.1)。

3.6 数据层的设计边界小结

约定	是否被遵守
dataloader 不做模型相关预处理	机器人数据遵守; VLM 数据违反 (已 tokenize)
归一化归属 dataloader 的 transform	遵守, 部署端复用同一 <code>ComposedModalityTransform.unapply</code> (<code>deployment/model_server/policy_norm_processor.py</code>)
跨本体维度对齐	未在核心层实现, 散落在 <code>PI0 / UMI / MiniCPMRobotManip</code> 三处

4. 模型层

4.1 starVLA/model/framework/ 一览

子包	框架 (注册名)	骨干	动作头	文件
VLM4A	QwenOFT	Qwen2.5/3-VL	MLP_ActionHeader.L1RegressionActionHead	VLM4A/QwenOFT.py
VLM4A	QwenFast	Qwen-VL-Action (扩词表)	fast_ActionHeader.Fast_Action_Tokenizer	VLM4A/QwenFast.py
VLM4A	QwenPI / QwenFM、QwenPI_v3	Qwen-VL	LayerwiseFM_ActionHeader	VLM4A/QwenPI.py、 QwenPI_v3.py
VLM4A	QwenGR00T	Qwen-VL	GR00T_ActionHeader.FlowmatchingActionHead	VLM4A/QwenGR00T.py
VLM4A	QwenDual、LangForce、CosmosGR00T、Gemma4GR00T、MiniCPMGR00T	Qwen-VL+DINO / Qwen-VL / Cosmos-Reason2 / Gemma4 / MiniCPM-V	GR00T 头	各同名文件
VLM4A	InternVLA-M1	Qwen-VL + DINO + QFormer	DiTActionHeader (DDPM)	VLM4A/M1.py
VLM4A	QwenAdapter	Qwen-VL-Action	VLA_AdapterHeader	VLM4A/QwenAdapter.py
VLM4A	QwenDiscreteDiffusion	Qwen-VL	LayerwiseDiscreteDiffusion_ActionHeader (MaskGIT)	VLM4A/QwenDiscreteDiffusion.py
VLM4A	PI0 / PI05	PaliGemma (vlm/OpenPIPaliGemma.py)	OpenPI_ActionHeader	VLM4A/PI0.py、PI05.py
VLM4A	ABot_M0	Qwen-VL + VGGT	AML_ActionHeader	VLM4A/ABot_M0.py
VLM4A	EgoVLA、MiniCPMPI、Gemma4PI、MiniCPMRobotManip	VLA / MiniCPM / Gemma4 / MiniCPM-RobotManip	各自	各同名文件
WM4A	CosmoPredict2{OFT,GR00T,PI}、Wan{OFT,GR00T,PI}	Cosmos-Predict2-2B / Wan2.2 DiT	OFT / GR00T / PI 头	WM4A/*.py
VM4A	ACT、DiffusionPolicy	ResNet-18	ACT / 1D U-Net DDPM	VM4A/ACT.py、 DiffusionPolicy.py

所有框架共用同一套 `__init__` 模板：`merge_framework_config` → `get_vlm_model(config)` → 运行期对齐维度 (把 VLM `hidden_size` 写回 `action_model.action_hidden_dim` 或 `diffusion_model_cfg.cross_attention_dim`) → `get_action_model(config)` → 读 `action_horizon`。

4.2 四种动作头逐个说明

(1) FAST: 自回归离散 token (QwenFast.py + fast_ActionHeader.py)

- 注入方式: 动作不进模型输入, 而是作为 assistant 回答。 `encoder_action2fasttoken` (`fast_ActionHeader.py:L78-83`) 调 `physical-intelligence/fast` 的 `UniversalActionProcessor` 把 [B,T,D] 动作压成 token id 序列; `map_fast_token_to_vlm_action` (`QwenFast.py:L265-271`) 拼成 `<robot_action_{k}>` 字符串; `build_qwenvl_inputs(..., solutions=...)` (`Qwen3.py:L114-171`) 把它放进 assistant turn, labels 把第一个动作 token 之前全部置 -100 (L147-169)。动作 token 在 Qwen3-VL-Action 词表中占 id `151669..153716` (`Qwen3.py:L21-24`, 2048 个), Qwen2.5-VL-Action 为 `151665..153712`。
- 头结构: 无可学习参数, `Fast_Action_Tokenizer` 只是 tokenizer 包装; 预测能力全部由 VLM 的 `lm_head` 承担。
- Loss: VLM 自带的 next-token CE (`qwenvl_outputs.loss`, `QwenFast.py:L172`), NaN 时置 0。
- 推理: `model.generate(max_length=2048)` (L210-213) → `_extract_action_token_ids` 按 id 区间筛出动作 token (L224-246) → 减 `_ACTION_TOKEN_MIN` 还原 FAST id → `fast_tokenizer.decode` (L220)。解码失败返回 None 条目。
- 与 hidden state 的连接: 完全经过 LM logits, 不取 hidden state。

(2) OFT: 并行连续回归 (QwenOFT.py + MLP_ActionHeader.py)

- 动作 query 注入: 在指令末尾拼接 " Please predict the next {N} robot actions: <action>🔍...🔍<action>." (`QwenOFT.py:L188-192`), 🔍 是词表中现成的单 token (L145-146), 重复 `chunk_len` 次充当 query。
- 头结构: `L1RegressionActionHead` (`MLP_ActionHeader.py:L60-88`) = `MLPResNet(num_blocks=2, input=H, hidden=2H, output=D)`; `get_action_model` (L91-115) 把 `hidden_dim` 设为 `2*action_hidden_dim`, 而 `action_hidden_dim` 在框架 `__init__` 中被覆盖为 VLM `hidden_size` (`QwenOFT.py:L134`)。
- 与 hidden state 的连接: 取 `hidden_states[-1]` (L204), `_gather_action_token_embeddings` (L295-347) 用 `input_ids == action_token_id` 找位置, `topk(chunk_len)` 取最后 N 个并按时间排序, gather 出 [B, N, H], 逐 token 过 MLP 得 [B, N, D]。
- Loss: `masked_l1_loss(pred, target, mask)` (L40-51) ——无 mask 时为普通 L1 均值; 样本携带 `action_mask[T,D]` 时按有效格子求均值 (L175-177 要求 batch 内全有或全无)。
- 推理: 单次前向, 无迭代 (L230-293)。

(3) PI: 层级 cross-attention flow matching (QwenPI_v3.py + LayerwiseFM_ActionHeader.py)

- 与 hidden state 的连接: `output_hidden_states=True` 得到 `num_layers+1` 个 hidden, 取最后 `num_action_dit_layers` (= LLM 层数, Qwen3-VL-4B 为 36) 个 (QwenPI_v3.py:L271); 每层过独立的 `LayerNorm + Linear(H → action_dit_hidden_dim=1024)` 投影 (`project_layers`, L227-239, H==1024 时退化为 Identity)。DiT 第 i 个 block 的 cross-attention K/V 来自 VLM 第 i 层 (`cross_attention_dit.py:L308-318`, `is_layerwise_encoder`)。populate_layerwise_dit_cfg (`share_tools.py:L261-295`) 把 `num_layers/input_embedding_dim/cross_attention_dim/num_attention_heads` 写进 `diffusion_model_cfg`。
- 头结构 (`LayerwiseFlowmatchingActionHead`, `LayerwiseFM_ActionHeader.py:L197-279`): `ActionEncoder` (`Linear(D-W)`), 与正弦时间嵌入拼接后 `Linear(2W-W) + swish + Linear(W-W)`, L61-100)、可选 `state_encoder` (MLP)、`future_tokens` (`num_target_vision_tokens=32` 个可学习 token)、可学习位置嵌入、DiT (默认 `interleave_self_attention=False` 即全 cross-attn, QwenPI_v3.py:L130)、`action_decoder = MLP(W-1024-D)`。DiT 输出用 `return_pre_output=True` 跳过 AdaLN 输出层 (L333-339)。
- 状态: 不用 `state_encoder`, 而是把 state 量化到 256 桶写进指令文本 "{instr} [STATE] 95 133 ... [ACTION]" (`add_discretized_state_to_instruction`, QwenPI_v3.py:L412-423, $\pi 0.5$ 风格), 随后 `state=None`。
- Loss (L288-347): $t \sim \text{Beta}(1.5, 1.0)$, $t = (0.999 - \text{sample})/0.999$; $\text{noisy} = (1-t) \cdot \epsilon + t \cdot a$, $\text{velocity} = a - \epsilon$; $\text{loss} = \text{mean}((\text{pred_velocity} - \text{velocity})^2)$, 只对最后 T 个位置 (动作段) 计算。框架侧把 batch 复制 `repeated_diffusion_steps` 份以采多个 t (QwenPI_v3.py:L315-321, 从 `cfg.trainer` 读, 默认 16)。
- 推理 (L349-408): 从 $N(0, I)$ 起步, `num_inference_timesteps=4` 步 Euler: $a \leftarrow a + (1/N) \cdot v$, t 离散化为 `int(t*1000)` 桶。另有 RTC 推理 `predict_action_realtime` (L410-631, Π GDM 引导或 simulated-delay 两种模式)。
- 参数量 (QwenPI_v3.py 文档字符串 L33-42): Qwen3-VL-4B 4.44B + action_model 538.7M + project_layers 94.6M。

(4) GR00T: 双系统 (QwenGR00T.py + GR00T_ActionHeader.py)

- 与 hidden state 的连接: 只取 `hidden_states[-1]` (QwenGR00T.py:L190), `cross_attention_dim` 运行期设为 VLM `hidden_size` (L155-157); `backbone_attention_mask` 传入 DiT 作 `encoder_attention_mask` (L216-219)。
- 头结构 (`FlowmatchingActionHead`, `GR00T_ActionHeader.py:L196-429`): `DiTConfig["DiT-B"]` (`input_embedding_dim=768`, `heads=12`, `head_dim=64`, L190-193) + `diffusion_model_cfg` (`num_layers=16`, `interleave_self_attention=True` → 偶数层 cross、奇数层 self, `cross_attention_dit.py:L242-243`); `state_encoder = MLP(S-1024-768)`、`action_encoder`、`future_tokens(32)`、位置嵌入、`action_decoder = MLP(1024-1024-D)` (DiT 经 AdaLN 输出层投到 `output_dim=1024`)。
- Loss (L312-363): 与 PI 相同的 flow-matching MSE, `sample_time` 多一个 `clamp(max=noise_s)`; `repeated_diffusion_steps` 从 `framework.action_model` 读 (默认 8, QwenGR00T.py:L97,L199-203)。
- 推理 (L365-421): 4 步 Euler, state 直接经 `state_encoder` 拼接在序列最前 ([state, future_tokens, actions])。
- 与 PI 的差异: 单层条件 vs 逐层条件; state 走 encoder vs 走文本; DiT 隐维 768 vs 1024; 交错 self-attn vs 全 cross。

四头对比

	FAST	OFT	PI (v3)	GR00T
条件来源	LM logits	<code>hidden_states[-1]</code> 中 query token	最后 36 层 hidden (逐层投影)	<code>hidden_states[-1]</code>
动作表示	离散 token	连续点估计	flow matching	flow matching
Loss	CE	L1 (可 mask)	MSE(velocity)	MSE(velocity)
推理	自回归 generate	1 次前向	4 步 Euler	4 步 Euler
头参数	0 (复用 lm_head)	MLPResNet 2 块 ($\approx 10H^2$)	~540M (36 层)	DiT-B 16 层
state	无	文本量化	文本量化	MLP encoder

4.3 VLM 接口层的抽象

`get_vlm_model(config)` (`starVLA/model/modules/vlm/__init__.py:L1-45`) 按 `framework.qwenvl.base_vlm` 的子串分派: Qwen2.5-VL / nora → `Qwen2_5_Qwen_VL_Interface`; Qwen3-VL → `Qwen3_Qwen3_VL_Interface`; Qwen3.5 → `Qwen3_5`; gemma-4 → `Gemma4`; molmo2 → `Molmo2`; minicpm-v → `MiniCPM_V`; florence → `Florence2`; cosmos-reason2 → `CosmosReason2`; egovla/vila → `VILA`。InternVL 没有接口实现 (仅 M1.py 的 InternVLA-M1 框架名沿用, 骨干仍是 Qwen-VL)。PaliGemma (`OpenPIPaliGemma.py`) 由 `PI0/PI05` 直接实例化, 不经此工厂。

接口契约 (`vlm/README.md`, 对应 Qwen3.py:L30-171): `__init__(config)` 加载模型+processor (`padding_side="left"`) 并把 `config.hidden_size` 对齐到 `text_config.hidden_size`; `forward(**kwargs)` 在 `autocast(bf16)` 下透传; `generate(**kwargs)`; `build_qwenvl_inputs(images, instructions, solutions=None)` 组 chat message、可选用 `datasets.vla_data.CoT_prompt` 包裹指令 (L126-130), 返回 `BatchFeature.to(device)`。框架通过 `self.qwen_vl_interface` 统一持有 (连 WM4A 的 `CosmosGR00T` 也沿用该属性名), 这个命名也被 `baseframework.forward_vlm` 与 `cotrain trainer` 硬编码依赖。

4.4 WM4A 与 VM4A

- WM4A (`docs/WM4A.md`): 把视频生成 DiT 当视觉编码器。get_world_model (`modules/world_model/__init__.py`) 按 `framework.world_model.base_wm` 分派到 `_CosmoPredict2_Interface` (`CosmoPredict2.py`: T5 文本编码 + VAE 图像编码 + DiT, `register_forward_hook` 抓取指定 block 的 hidden, `timestep=0`、`condition_mask` 标记真实帧, L126-147、L274-298) 或 `_Wan2_Interface` (UMT5 + Wan2.2 DiT)。`build_inputs()` 替代 `build_qwenvl_inputs()`。以 `CosmoPredict20FT` 为例 (`WM4A/CosmoPredict20FT.py:L82-183`):

`hidden_states[-1]` 全局平均池化 → `Linear(H → chunk_len·H)` 生成 N 个 action query → 复用 `MLP_ActionHeader` → `nn.L1Loss`。GR00T/PI 变体同理复用两个 flow-matching 头。文档声称 7 种组合含 `VM4A_0FT.py`，实际目录只有 6 个文件。

- **VM4A** (`docs/VM4A.md`): 无 VLM 的轻量视觉运动策略基线。ACT 包装 LeRobot `ACTPolicy`，`DiffusionPolicy` vendored `real-stanford/diffusion_policy` 子集 (`VM4A/_dp_vendor/`)，两者安装恒等归一化器以避免与 StarVLA 的 `StateActionTransform` 双重归一化；`DiffusionPolicy` 重写 `state_dict/load_state_dict` 把 `EMAModel.averaged_model` 以 `ema_averaged.*` 前缀持久化 (`DiffusionPolicy.py:L147-260`)。这是仓库中唯一的 EMA 实现。

5. 训练

5.1 四个入口的差异

脚本	数据	循环	模型	备注
<code>train_starvla.py</code>	单 VLA DataLoader	自写 while 循环 + <code>accelerator.accumulate</code>	<code>build_framework</code>	支持 <code>is_resume</code> (从 ckpt 并快进 LR scheduler)； <code>STARVLA_DISABLE_</code> 模式 (L50-65)
<code>train_starvla_cotrain.py</code>	VLA + VLM 两个 DataLoader	同上，每步两次 backward	同上	DeepSpeed engine 分 <code>accumulate()</code> (L108-110)； <code>vlm_loss = out.l</code> <code>loss_scale.vlm</code>
<code>train_starvlm.py</code>	仅 VLM DataLoader	同上	<code>build_framework</code> (用其 <code>qwen_vl_interface</code>)	纯 VLM SFT，产物可推理
<code>train_starvln.py</code>	<code>make_supervised_data_module</code>	HF Trainer	直接 <code>Qwen*VLForConditionalGeneration</code>	VLN-CE 用；HfArgumentParser 支持 LoRA、 <code>replace_qwen2_vl_flash-attn varlen</code> 补丁 (<code>monkey_patch.py</code>)

前三者约 80% 代码重复 (`setup_directories`、`_init_wandb`、`_save_checkpoint`、`_log_metrics`、`_finalize_training` 几乎逐行相同)。

5.2 `trainer_utils` 关键机制

- **冻结** `freeze_backbones(model, freeze_modules)` (`trainer_tools.py:L192-234`): 把逗号分隔字符串按 `.` 切成属性路径逐级 `getattr`，命中即对该子模块全部参数 `requires_grad=False`；路径不存在只打印警告。**实现是精确点路径匹配，不支持正则** (README/FAQ 所写 "regex" 与打印文案 "re pattern" 与代码不符)。`nn.ModuleList` 可用数字索引路径 (如 `...language_model.layers.3`)。
- **分模块学习率** `build_param_lr_groups(model, cfg)` (L92-148): 遍历 `cfg.trainer.learning_rate` 中除 `base` 外的键，同样按点路径找模块，其参数 (排除 `freeze_modules` 命中的参数) 成为一个 param group；剩余参数归 `base`。冻结列表在此处被**再解析一次** (L108-125)，与 `freeze_backbones` 构成隐式耦合——两处必须同步。注意 `setup_optimizer_and_scheduler` 在 `main()` 中先于 `prepare_training()` 里的冻结执行 (`train_starvla.py:L476-489`)，正是靠这段重复解析保证冻结参数不进优化器。
- **checkpoint 保存**: 仅 `state_dict` (`accelerator.get_state_dict`)，`pt` 或 `safetensors`，路径 `<run_dir>/checkpoints/steps_{N}_pytorch_model.pt`，并追加 `summary.jsonl` 与刷新 `config.yaml` (`train_starvla.py:L276-303`)；结束时另存 `final_model/pytorch_model.pt`。**不保存 optimizer/scheduler 状态** (FAQ 明说)。
- **加载** `load_pretrained_backbones(model, ckpt, reload_modules)` (`trainer_tools.py:L253-305`): `reload_modules="a,b,c"` 时按前缀切子 `state_dict` 并对子模块 `strict=True` 加载；为空则整模型 `strict=False` 加载 (缺键静默)。推理侧 `baseframework.from_pretrained` 则是 `strict=True` (`base_framework.py:L306`)。
- **EMA**: 训练器中无 EMA；仅 `VM4A/DiffusionPolicy` 内部实现。
- **梯度累积**: 来自 DeepSpeed 配置 (`ds_config.yaml` 的 `gradient_accumulation_steps: 1`) 经 `accelerator.gradient_accumulation_steps` 读出；yaml 里 `trainer.gradient_accumulation_steps: 4` **未被任何训练脚本读取**。LR scheduler 只在 `sync_gradients` 时 step (`train_starvla.py:L436-437`)。
- **DeepSpeed ZeRO-2/3 与多机**: `accelerate launch --config_file starVLA/config/deepseeds/deepspeed_zero2.yaml --num_processes 8 ...`；多机加 `--main_process_ip/--main_process_port/--machine_rank/--num_machines` (`run_libero_train.sh:L59-75` 注释模板)。ZeRO-3 用 `deepspeed_zero3.yaml`，训练脚本无需改动 (`accelerator.get_state_dict` 负责聚合)。
- **梯度裁剪**: `trainer.gradient_clipping` (默认 1.0) 经 `accelerator.clip_grad_norm`；`trainer.max_grad_norm` 未被读取。
- **精度**: VLM 前向 `autocast(bf16)`，动作头 `autocast(float32)` (各框架 forward 内显式切换)。
- **在线评估**: 每 `eval_interval` 步取一个训练 batch 跑 `predict_action`，记录归一化空间的欧氏距离 `mse_score` (`train_starvla.py:L386-404`)。

6. 部署

组件	文件	作用与接口
WebSocket 策略服务器	deployment/model_server/server_policy.py (入口)、 tools/websocket_policy_server.py	python server_policy.py --ckpt_path X.pt --port 10093 -- use_bf16 [--config_override k=v] [--idle_timeout 1800]；握手时发 送 wrapper.metadata (action_chunk_size 、 available_unnorm_keys 、 action_keys 、 state_keys 、 training_obs_image_size)，请求体 {"examples": [...], "unnorm_key":..., **kwargs}，响应 response["data"]["actions"] (已反归一化， [B,T,D])
策略包装	policy_wrapper.py:PolicyServerWrapper (L53-197)	baseframework.from_pretrained 加载 → predict_action(examples, unnorm_key) → 用 PolicyNormProcessor.unapply_actions 反归一化； 多数数据集 checkpoint 需客户端每次传 unnorm_key
反归一化	policy_norm_processor.py:PolicyNormProcessor (L238-)	从 dataset_statistics.json + 注册表里的 DataConfig 重建训练期 ComposedModalityTransform 并调用 unapply，是"单一归一化真源"
GR00T 协 议服务器	server_policy_gr00t_zmq.py + gr00t_obs_adapter.py + tools/zmq_policy_server.py	兼容 Isaac-GR00T N1.6 的 msgpack ZMQ 协议 (ping/reset/get_action/get_modality_config)，把命名 state dict 按 DataConfig state_keys 顺序展平、把动作按 action_key_dims 切回命名 组；端口 5555
客户端	tools/websocket_policy_client.py:WebsocketClientPolicy	get_server_metadata()、predict_action(query)； _check_eval_observation_contract 在图像尺寸/数量与训练元数据不符时告 警一次
序列化	tools/msgpack_numpy.py 、 tools/image_tools.py:to_pil_preserve	numpy 走 msgpack；框架内用 to_pil_preserve 把 ndarray 还原为 PIL
Docker	deployment/docker/Dockerfile.{server,train,robocasa} 、 docker-compose.yml	server 镜像 python:3.10-slim 不装 CUDA toolkit (sdpa 推理)；train 镜像含 DeepSpeed/flash-attn；compose 提供 eval / train profile；checkpoint 目录 挂到 /models，基座 VLM 挂到 /workspace/starVLA/playground/Pretrained_models
上传	deployment/upload/push_model_to_hf.py	12 行脚本：create_repo + HfApi.upload_large_folder 整个 run 目录 (含 config.yaml、dataset_statistics.json、checkpoints/)

checkpoint 目录约定 (share_tools.read_mode_config , L357-399)：<run_dir>/config.yaml、<run_dir>/dataset_statistics.json、
<run_dir>/checkpoints/steps_N_pytorch_model.pt，parents[1] 反推 run_dir。

7. examples 生态

7.1 simBenchmarks/ (13 个)

基准	做什么	入口	值得注意的设计
LIBERO	4 套件 (Spatial/Object/Goal/10) 单策略训练+评测；README 给出 4 头 × 2 骨干共 8 组结果 (Qwen3-VL-OFT 平均 96.6)	train_files/run_libero_train.sh ； eval_files/run_policy_server.sh + eval_libero.sh	model2libero_interface.py 5 chunk 缓存 (step % action_chu AdaptiveEnsembler；openpi/ 子 转换与评测
LIBERO-plus	用 LIBERO 训练的模型零样本评测 7 类扰动 (Camera/Robot/Language/Light/Background/Noise/Layout)	eval_files/eval_libero.sh 、 parallel_eval/	复用 LIBERO 接口；aggregate_re
RoboTwin 2.0	50 任务 AgileX 双臂 14 维关节；Clean/Randomized 两种数据	train_files/run_robotwin_train.sh ； eval_files/eval.sh + deploy_policy.yml	starvla_train_arx.yaml (AR) _abs.yaml (绝对关节) 分开
RoboDojo	ARX X5 14D abs_qpos，horizon 50，每 16 步重规划	train_files/run_robodojo_train.sh ； 评测经 XPolicyLab	三份 yaml 对应 QwenOFT/QwenGRC 直接读官方 LeRobot v2.1
VLA-Arena	11 套件 × 3 难度，success + constraint cost	data_preparation.sh ； eval_files/run_parallel_eval.sh	数据为 openpi 版 LeRobot (图像在 f total_videos==0 分支，datasi 2448)
DOMINO	35 个动态物体任务，SR + MS 指标	train_files/run_domino_train.sh ； eval_files/eval.sh	接口文件名沿用 model2robotwin_ history_flow_utils.py
Robocasa_tabletop	GR1 人形桌面任务 (NVIDIA fork)	train_files/run_robocasa.sh ； eval_files/batch_eval_args.sh	无预训练即 SOTA；wrappers 目录含 wrapper；有单测 tests/test_robocasa_tableto

基准	做什么	入口	值得注意的设计
Robocasa_365	官方 robocasa 365 任务、PandaOmron	<code>train_files/run_robocasa365.sh</code> ; <code>eval_files/run_eval.sh</code>	README 是 agent (Copilot) 自动集
SimplerEnv	Bridge/RT-1 (OXE) 训练, WidowX/Google 评测	<code>train_files/run_oxe_train.sh</code> ; <code>eval_files/start_simpler_env.sh</code>	<code>auto_eval_scripts/</code> 批量调度; <code>adaptive_ensemble.py</code> 被 LIBE
Behavior	BEHAVIOR-1K 50 任务挑战赛	<code>start_policy_server.sh</code> + <code>start_behavior_env.sh</code>	标注"under construction"; 需 RT Co
calvin	Calvin D→D 长程链任务	<code>train_files/run_calvin_train.sh</code> ; <code>eval_files/eval_calvin.sh</code>	UNT 团队维护; 数据需先经 RoboTro LeRobot
MetaWorld	MT50	<code>eval_files/run_policy_server.sh</code> + <code>eval_metaworld.sh</code>	使用 QwenPI_v3 yaml
VLN-CE	R2R/RxR 导航 (VLM 生成动作文本)	<code>train_files/run_vlnce_train.sh</code> (走 <code>train_starvln.py</code>); <code>eval_files/run_qwenvl_vlm_server.sh</code>	唯一使用 HF Trainer 与纯 VLM 服务

7.2 `realRobots/` (5 个)

目录	做什么	入口	值得注意
Franka	单/双 Franka 真 机: 数据转 LeRobot 2.1 → 注册 → 训练 → 推理	<code>train_files/run_franka_train_{single,dual}.sh</code> ; <code>eval_files/inference_{single,dual}_example.py</code>	完整的"新机器人接入"范本
Realman	RM-75 8 维 (7 关节 + 夹爪), VM4A 路线	<code>train_files/train_realman_{act,dp}.sh</code>	唯一 ACT / DiffusionPolicy 示例
UnitreeG1_WholeBody	G1 全身: teleop → LeRobot → 训练 → 策略服 务器 → SONIC/WBC 控 制	<code>step2_training/.../run_starvla_qwenoft_g1_sonic_train.sh</code> ; <code>step3_deployment/run_policy_server.sh</code>	分 step0-3 目录; <code>gr00t/policy/server_client.py</code> 对 接 GR00T ZMQ 协议
EgoVLA	VILA 基座双手 48 维 MANO 动 作	<code>train_files/starvla_egovla.yaml</code> ; <code>eval_files/server_egovla_g1.py</code>	把外部模型原生移植进 <code>modules/vlm/vila_egovla/</code> + <code>EgoVLA_ActionHeader</code> , 无 llava 依赖
RoboChallenge_table30v2	UR5 真机挑战平 台 (HTTP 协议)	<code>train_files/run_robochallenge_table30v2.sh</code> ; <code>eval_files/run_self_test.sh</code> → <code>run_test_with_mock.sh</code>	三段式验证: 离线自测 → mock server → 真机

7.3 `modelExtensions/` (6 个)

目录	做什么	入口	值得注意
CoTrainVLM	VLA + VLM 数据 共训指南	<code>train_files/run_libero_cotrain.sh</code> (<code>train_starvla_cotrain.py</code>)、 <code>run_train_starvlm.sh</code>	VLM 数据需 QwenVL 对话 json 格式
DiscreteDiffusion	MaskGIT 离散扩 散头 + RTC 推理	配置在 <code>starVLA/config/training/starvla_train_discrete_diffusion*.yaml</code>	与 QwenPI 共用 DiT 栈, 仅替换头
Gemma4	Gemma-4-E2B 骨干 (LIBERO 平 均 96.0)	<code>run_libero_local_smoke.sh</code> 、 <code>submit_hpc3_libero.sh</code>	需 transformers ≥ 5.5; <code>Gemma4PI</code> 直 接继承 <code>Qwen_PI</code> (<code>Gemma4PI.py:L22-</code>)
MiniCPM	MiniCPM-V 4.6 (1.3B) 骨干	同上形式	需 transformers ≥ 5.7、torch ≥ 2.11, 与主环境 transformers 4.57 冲突
MiniCPM- RobotManip	微调已发布的 80 维统一动作空间 VLA	<code>train_files/run_libero_train.sh</code> ; <code>export_checkpoint.py</code>	槽位映射 + 分组掩码 loss (xyzx500、 rot6dx10、gripper L1); 有单测
NeuralVLA	NeuroVLA 历史状 态窗口支持	<code>run_libero_train_yibu.sh</code> 、 <code>eval_libero.py</code>	以"替换 <code>data_config.py</code> 与 <code>eval_libero.py</code> "方式集成, 侵入式

7.4 `human2robots/UMI4Pretraining`

UMI 人类演示数据端到端接入：`tools/umi_pipeline.py` + `download_umi.sh` 拉取/转换 400 例 HF 数据为 LeRobot v2.1, `train_files/data_registry/data_config.py` 注册 24 个 mixture、13 个 robot config (全部外部注册, 不改核心文件), `umi_loader_overrides.yaml` 走 `dataset_py: umi_datasets`。 `test_masked_l1.py`、`test_umi_data_loader.py` 验证 `action_mask` 路径。

7.5 `examples/eval_protocol.md` 评测协议

- 架构: Sim/Real 控制器 (客户端) ↔ WebSocket ↔ PolicyServer ↔ `Framework.predict_action`。
- 数据契约: `example = {"image": List[np.ndarray], "lang": str, ...}`, 图像必须是 ndarray (PIL 不可序列化), 元数据放独立键。
- `model2{bench}_interface.py` 负责基准相关适配: 动作 ensembling、delta→absolute 转换、chunk 调度, 服务器保持通用。
- 文档滞后: 示例代码仍写 `result["normalized_actions"][0]`, 而服务器现已返回 `data["actions"]` (已反归一化, 见 `deployment/model_server/README.md`); 链接 `./LIBERO/eval_files/model2libero_client.py` 指向不存在的路径。

8. `docs/agent_skills/integrate-starvla-dataset`

本地快照只有 `assets/templates/` 5 个模板文件, `git ls-files docs` 确认仓库中没有 `SKILL.md` / `README.md` (`starVLA_guideline.md:L9` 链接的 `agent_skills/integrate-starvla-dataset/README.md` 与 `docs/integrate_your_dataset.md` 均不存在于此快照)。可推断的意图与结构:

模板	目标位置	作用
<code>data_config.py</code>	<code>examples/<BENCH>/train_files/data_registry/data_config.py</code>	带 <code><<TODO_*>></code> 占位符的 DataConfig, 强制导出 <code>ROBOT_TYPE_CONFIG_MAP</code> / <code>ROBOT_TYPE_TO_EMBODIMENT_TAG</code> / <code>DATASET_NAMED_MIXTURES</code> 三个字典
<code>modality.json</code>	每个 LeRobot 数据集 <code>meta/modality.json</code>	video/state/action 切片与 <code>annotation.human.task_description.original_key</code> 必须为 <code>task_index</code>
<code>training_config.yaml</code>	<code>examples/<BENCH>/train_files/starvla_<FW>_<BENCH>.yaml</code>	smoke 档位 (<code>max_train_steps: 100</code> 、 <code>per_device_batch_size: 2</code>), 注释标出 <code>action_dim/state_dim/action_horizon</code> 必须与 DataConfig 一致
<code>run_train.sh</code>	<code>examples/<BENCH>/train_files/run_<BENCH>_train.sh</code>	<code>accelerate launch ... train_starvla.py</code> 模板, 默认 2 卡
<code>model2bench_interface.py</code>	<code>examples/<BENCH>/eval_files/</code>	<code>RobotSpec</code> + <code>run_policy(obs, prompt)</code> 契约, 注释提示三阶段验证 (自测 → mock → 真机)

意图: 把"接入一个新数据集/机器人"固化为 agent 可执行的填空流程 (注册表 → modality → yaml → 启动脚本 → 评测接口), `README` 声称 Copilot 已借此自动集成 Robocasa_365 与 RoboChallenge。模板中 `ROBOT_TYPE_TO_EMBODIMENT_TAG` 已被注册表标记为 legacy (`registry.py:L12-16`), 模板与代码约定略有脱节。

9. VLAct 配方对照 (重点)

VLAct (arXiv 2608.27550) 在 StarVLA 上以 Qwen3-VL-4B 做"以表示为中心的持续预训练", 六项配方对照如下。状态定义: 已有 = 改 yaml 即可; 部分 = 有基础设施但需少量代码; 缺失 = 需新模块。

#	VLAct 配方	状态	代码现状	需改动
a	冻结视觉编码器 + LLM 下半层	部分	<code>trainer.freeze_modules</code> 精确点路径列表	列 18 条路径可用; 建议加正则/区间语法
b	caption 共训, <code>L = L_action + 0.5 · L_VLM-CE</code>	已有	<code>train_starvla_cotrain.py</code> + <code>loss_scale.vlm</code>	<code>loss_scale.vlm: 0.5</code> , <code>dataset_use: sharegpt4v_coco</code>
c	OFT + PI + GR00T 三头共享骨干, <code>L_action = Σ L_head</code>	缺失	所有框架单头; LangForce 有"同头双支路加权 loss"先例	新框架 <code>QwenMultiHead</code>
d	20 维部分统一布局 + 非激活维 mask	部分	mask 消费端: <code>QwenOFT.masked_l1_loss</code> 、 <code>AML_ActionHeader</code> ; 生产端: <code>umi_datasets</code> 、 <code>MiniCPMRobotManip._to_80d</code> ; GR00T/PI 头无 mask	新 transform + 两个 FM 头加 mask
e	周期关节 wrap-aware L1	缺失	无任何模 2π 逻辑	新 loss 函数 + 数据侧 wrap
f	下游丢弃预训练头、重初始化任务头、全参解冻	已有	<code>pretrained_checkpoint</code> + <code>reload_modules</code> + <code>freeze_modules: ''</code>	注意 <code>project_layers</code>

9.1 (a) shallow-layer protection: 冻结视觉编码器 + LLM 下半层

现状。TrainerUtils.freeze_backbones (trainer_tools.py:L192-234) 只接受逗号分隔的精确属性路径; build_param_lr_groups (L108-125) 用同一字符串把冻结参数排除出优化器。Qwen3-VL 在 _Qwen3_VL_Interface 下的路径 (依据 faq.md:L49 的 qwen_vl_interface.model.model.visual 与 HF Qwen3VLModel 结构推断, 需 print(model) 确认): 视觉塔 qwen_vl_interface.model.model.visual, LLM 层 qwen_vl_interface.model.model.language_model.layers.{i} (4B 版共 36 层, QwenPI_v3.py:L96 默认配置 num_vl_layers: 36)。

零代码方案: yaml 中写 19 条路径:

```
trainer:
  freeze_modules: "qwen_vl_interface.model.model.visual,qwen_vl_interface.model.model.language_model.embed_tokens,qwen_vl_interface.model.model.language_model.layers.0,...,qwen_vl_interface.model.model.language_model.layers.17"
```

nn.ModuleList 支持 getattr(layers, "3"), 因此可行; 缺点是冗长且与层数绑定。注意 LIBERO 默认 yaml 已把整个 qwen_vl_interface 冻结 (starvla_cotrain_libero.yaml:L57), 需覆盖。

建议改动 (starVLA/training/trainer_utils/trainer_tools.py):

```
# 新增: 把冻结规则解析集中到一处, 供 freeze_backbones 与 build_param_lr_groups 共用
def resolve_frozen_param_ids(model, freeze_modules: str) -> set[int]:
    ids = set()
    for pat in [p.strip() for p in (freeze_modules or "").split(",") if p.strip()]:
        if pat.startswith("re:"):
            # 正则: re:~qwen_vl_interface\\.model\\.model\\.language_model\\.layers\\.\\d\\{10-7}\\}\\.
            rx = re.compile(pat[3:])
            ids |= {id(p) for n, p in model.named_parameters() if rx.search(n)}
        elif "[" in pat:
            # 区间: ...layers[0:18]
            base, rng = pat.split("["); lo, hi = map(int, rng.rstrip("]").split(":"))
            for i in range(lo, hi):
                ids |= {id(p) for p in _get_by_path(model, f"{base}.{i}").parameters()}
        else:
            # 现有精确路径
            ids |= {id(p) for p in _get_by_path(model, pat).parameters()}
    return ids
```

freeze_backbones 改为对 id in ids 的参数置 requires_grad=False; build_param_lr_groups:L117-125 改为调用同一函数。这样也顺带修掉“两处解析”耦合。另可在 VLATrainer.prepare_training 增加 trainer.freeze_llm_layers_below: 18 语法糖, 展开为上述区间。

9.2 (b) caption 数据共训, $L_{total} = L_{action} + 0.5 \cdot L_{VLM-CE}$

现状: train_starvla_cotrain.py:_train_step (L358-426) 每步做 action_loss.backward() 与 (vlm_loss * loss_scale.vlm).backward(), 等价于对和求梯度; loss_scale.vla 不被读取 (恒 1)。VLM 数据由 vlm_datasets.py 提供, qwen_data_config.py 已注册 sharegpt4v_coco (LLaVA-OneVision-COCO caption), data_preparation.sh 会下载它。labels 只监督 assistant 回答 (vlm_datasets.py:L232-246)。

配置:

```
datasets:
  vlm_data: {dataset_py: vlm_datasets, dataset_use: sharegpt4v_coco, model_type: qwen3vl, per_device_batch_size: 4, ...}
trainer:
  loss_scale: {vla: 1.0, vlm: 0.5}
```

启动 starVLA/training/train_starvla_cotrain.py 而非 train_starvla.py。注意: model_type 必须与骨干匹配 (qwen3vl 用 get_rope_index_3, LIBERO yaml 里写的是 qwen2.5vl); VLA 的 caption 源 (LLaVA-ReCap-CC3M、LLaVA-OneVision) 需按 qwen_data_config.data_dict 格式追加注册; 混合比例只能通过两个 per_device_batch_size 控制, 无按样本权重的混批。

9.3 (c) 多头共监督: OFT + PI + GR00T 共享潜表征

现状: 28 个框架全部持有单个 self.action_model。baseframework.compute_loss (base_framework.py:L194-230) 设计了多 tag loss 字典但训练器不调用; _train_step 只读 output_dict["action_loss"], 其余键被丢弃 (train_starvla.py:L420-423, L439-441)。LangForce (LangForce.py:L120-121, L192) 用 prior_loss_weight 加权同一 GR00T 头的两支流 loss, 是“多项 action loss 相加”的唯一先例。

需新增 starVLA/model/framework/VLM4A/QwenMultiHead.py (注册名 QwenMultiHead), 骨干前向一次、三头各自算 loss:

```
@FRAMEWORK_REGISTRY.register("QwenMultiHead")
class QwenMultiHead(baseframework):
    def __init__(self, config):
        super().__init__()
        self.config = merge_framework_config(QwenMultiHeadDefaultConfig, config)
        self.qwen_vl_interface = get_vlm_model(config=self.config)
        H = self.qwen_vl_interface.model.config.hidden_size
        # 1) OFT 头: 复用 MLP_ActionHeader; 需要 query token
        self.config.framework.action_model.action_hidden_dim = H
        self.offt_head = mlp_get_action_model(self.config)
        # 2) GR00T 头: cross_attention_dim = H
        self.config.framework.action_model.diffusion_model_cfg.cross_attention_dim = H
        self.groot_head = groot_get_action_model(self.config)
        # 3) PI 头: LayerwiseFM, 需 populate_layerwise_dit_cfg + project_layers
        populate_layerwise_dit_cfg(self.config, dit_hidden_dim=1024, num_dit_layers=num_vl_layers)
        self.pi_head = layerwise_get_action_model(self.config)
```

```

self.project_layers = nn.ModuleList([nn.Sequential(nn.LayerNorm(H), nn.Linear(H, 1024)) for _ in range(num_vl_layers)])
self.action_token_id = ... # 同 QwenOFT

def forward(self, examples):
    images, instrs, actions, masks = unpack(examples) # masks 见 9.4
    instrs = [s + oft_prompt_suffix(self.chunk_len) for s in instrs] # OFT 需要 query token 在序列里
    inputs = self.qwen_vl_interface.build_qwen_vl_inputs(images=images, instructions=instrs)
    out = self.qwen_vl_interface(**inputs, output_hidden_states=True, return_dict=True)
    hs, last = out.hidden_states, out.hidden_states[-1]
    target = torch.tensor(np.array(actions))[:, -self.action_horizon:]
    with torch.autocast("cuda", dtype=torch.float32):
        q = gather_action_tokens(last, inputs["input_ids"], self.action_token_id) # 复用 QwenOFT.gather_action_token_embeddings

    l_oft = masked_ll_loss(self.oft_head.predict_action(q), target, masks)
    l_groot = self.groot_head(last.repeat(r,1,1), target.repeat(r,1,1), None, encoder_attention_mask=..., action_mask=...)
    vl_list = [p(h) for p, h in zip(self.project_layers, hs[-len(self.project_layers):])]
    l_pi = self.pi_head([h.repeat(r,1,1) for h in vl_list], target.repeat(r,1,1), None, encoder_attention_mask=..., action_mask=...)
    return {"action_loss": l_oft + l_pi + l_groot, "oft_loss": l_oft, "pi_loss": l_pi, "groot_loss": l_groot}

```

配套改动: (i) `train_starvla*.py:train_step` 把 `output_dict` 中所有标量 Tensor 加入 `log_dict` (目前只记 `action_loss`), 便于监控三头; (ii) `predict_action(examples, head="oft")` 按参数选头; (iii) 三个头都会在序列中看到 `占位 token`, 若不希望 PI/GR00T 关注它们, 可用 `inputs["attention_mask"]` 复制一份把这些位置置 0 传入 `encoder_attention_mask`。`repeated_diffusion_steps` 的 `r` 只对两个 FM 头生效, OFT 头保持 `r=1`。

9.4 (d) 部分统一的 20 维跨本体动作布局 + 非激活维 mask

现状。三个层面:

- 数据层: `DataConfig.action_keys` 决定拼接顺序 (`datasets.py:L1408-1411`), `transform/concat.py` 只做拼接, 没有“槽位映射/填充”`transform`; `umi_datasets.fit_matrix` (L86-114) 只做右下角填充并产 `action_mask[T,D]`; `PI0._pad_array_2d` 零填充到 32 维且无 mask (等于 VLAct 图 4 中的“naive unified”)。
- 损失层: `QwenOFT.masked_ll_loss` 支持 `[B,T,D]` mask (L40-51); `AML_ActionHeader.forward(action_mask=[B,D])` (L343-353) 把 mask 扩到 `[B,T,D]` 后做掩码 MSE; `GR00T_ActionHeader.forward` (L312-363) 与 `LayerwiseFM_ActionHeader.forward` (L288-347) **不接受 mask**, `loss = ((pred - velocity)**2).mean()`。
- 统计/部署层: `save_dataset_statistics` 按 embodiment tag 分别保存原生 key 空间的统计 (`datasets.py:L2737-2846`); `PolicyNormProcessor.unapply_actions` 按 `unnorm_key` 反归一化原生维度。MiniCPMRobotManip 演示了框架内槽位映射 (`_to_80d`, L141-150) 和逐通道掩码归约 (`_reduce_qwenvla`, L156-165)。

建议改动:

- 新增 `starVLA/dataloader/gr00t_lerobot/transform/unified_layout.py`:

```

class UnifiedActionLayoutTransform(ModalityTransform):
    """把归一化后的原生 action.* 键写入固定 D_u=20 维槽位, 并输出 action_mask."""
    slot_map: dict[str, tuple[int, int]] # 例如 Franka: {"action.delta_eef_position": (12,15), "action.delta_eef_rotation": (15,18), "action.gripper_close": (18,19)}
    # AgileX: {"action.left_joints": (0,6), "action.right_joints": (6,12), "action.left_gripper": (18,19), "action.right_gripper": (19,20)}
    unified_dim: int = 20
    def apply(self, data):
        T = next(iter(data[k] for k in self.slot_map)).shape[0]
        out = torch.zeros(T, self.unified_dim); mask = torch.zeros(T, self.unified_dim, dtype=torch.bool)
        for key, (lo, hi) in self.slot_map.items():
            out[:, lo:hi] = data[key]; mask[:, lo:hi] = True
        data["action.unified"] = out; data["action_mask"] = mask
        return data

```

`DataConfig` 的 `action_keys` 改为 `["action.unified"]` (或让 `_pack_sample` 在 `data` 含 `action_mask` 时一并放进样本 dict——`datasets.py:L1413-1418` 加两行)。这样 `LeRobotMixtureDataset` 混合 Franka 与 AgileX 时 batch 内形状一致, 解决 `np.array(actions)` 报错。

- `GR00T_ActionHeader.forward` / `LayerwiseFM_ActionHeader.forward` 增加 `action_mask=None` 参数, `loss` 改为与 `AML_ActionHeader.py:L343-353` 相同的掩码归约; `QwenGR00T.forward` / `QwenPI_v3.forward` 像 `ABot_M0.py:L150-152, L197-208` 那样读取并 `repeat` mask。
- 统计: 为每个本体单独建 tag, 统计仍在原生 key 上计算 (q99 归一化在槽位映射之前完成), `dataset_statistics.json` 无需改; 部署侧 `PolicyNormProcessor` 需在 `unapply_actions` 前按 `slot_map` 取回该本体的激活槽位 (新增 `slot_map` 元数据到 `metadata`)。
- 夹具共享维 19: 两个本体的 `gripper` 键都映射到同一槽位; VLAct 要求“open/close 语义一致”, 需在 `DataConfig` 层统一 `binary` 归一化方向。

9.5 (e) 周期关节角的 wrap-aware L1

现状: 仓库无任何角度 wrap 逻辑 (`torch.remainder/fmod/2π` 检索仅命中 `OpenPI` 位置编码)。相关但不同的机制: `RotationTransform` (`state_action.py:L29-95`) 把 EE 旋转在 `axis_angle/euler/quaternion/rotation_6d/matrix` 之间转换; `StateActionSinCosTransform` (L596-610) 对 state 做 `sin/cos` 展开 (不可逆, 仅 state); `_DEFAULT_MIN_MAX_STATISTICS` 对 `euler/axis_angle` 用 `[-π, π]` 固定界归一化。

建议改动：

- 数据侧 wrap (VLAct 式 (1))：在 `Normalizer.forward` (`state_action.py:L108-192`) 前加一个可选步骤，或新增 `normalization mode`
"`wrap_pi`" : `x = torch remainder(x +  $\pi$ , 2 $\pi$ ) -  $\pi$` ，再按 `min_max` 用固定界 `[- $\pi$ ,  $\pi$ ]` 归一化到 `[-1, 1]` (此时归一化空间中 `2 $\pi$` 对应 2.0)。
- 损失侧 (式 (2)(3))：新增 `starVLA/model/modules/action_model/losses.py`：

```
def wrap_aware_l1(pred, target, periodic_mask, period=2.0):
    """periodic_mask: [D] bool: period 为归一化空间中的周期 ([- $\pi$ ,  $\pi$ ] $\rightarrow$ [-1,1] 时为 2.0)。"""
    delta = pred - target
    delta_p = torch remainder(delta + period / 2, period) - period / 2
    delta = torch.where(periodic_mask.view(1, 1, -1), delta_p, delta)
    return delta.abs()          # 交给调用方做 action_mask 掩码归约
```

接入点：OFT 在 `masked_l1_loss` 内替换 `error = torch.abs(prediction - target)` (`QwenOFT.py:L42`)；PI/GR00T 按 VLAct 附录 F 应作用于"最终采样动作"——训练时要么额外跑一次 `predict_action` 采样 (4 步，成本约 +4 次 DiT 前向)，要么用单步估计 `x1_hat = noisy + (1 - t)·v_pred` (`LayerwiseFM_ActionHeader.py:L308-346` 处可直接得到) 作近似，再加 `$\lambda$ ·wrap_aware_l1(x1_hat, actions, periodic_mask)`。`periodic_mask` 由 `DataConfig` 声明 (如 `periodic_action_keys = ["action.left_joints", "action.right_joints"]`) 并随 `slot_map` 写入 20 维索引。

9.6 (f) 下游微调：丢弃预训练头、重初始化任务头、全参数解冻

现状 (已有)：

```
trainer:
  pretrained_checkpoint: <pretrain_run>/checkpoints/steps_N_pytorch_model.pt
  reload_modules: "qwen_vl_interface"      # 只加载骨干; action_model 保持随机初始化
  freeze_modules: ""                      # 全参数可训
```

`load_pretrained_backbones` (`trainer_tools.py:L278-296`) 按前缀切出 `qwen_vl_interface.*` 子 `state_dict` 并 `strict=True` 加载到子模块；`action_model` 未列入即为重新初始化。多头预训练框架的 checkpoint 中骨干前缀同为 `qwen_vl_interface.`，因此可跨框架 (`QwenMultiHead` $\rightarrow$ `QwenOFT`) 加载。注意：`QwenPI_v3` 的 `project_layers` 属于框架而非 `action_model`，若想复用需写 `reload_modules: "qwen_vl_interface,project_layers"`；分模块学习率 `learning_rate.action_model` 让新头用更大 LR (LIBERO 默认 `1e-4` vs 骨干 `1e-5`)。

9.7 实施优先级

- yaml 级 (当天)：(a) 19 路径冻结、(b) `loss_scale.vlm=0.5` + caption 数据、(f) `reload_modules`。
- 小改 (1-2 天)：`freeze_backbones` 正则/区间；两个 FM 头加 `action_mask`；`_train_step` 记录额外 loss 键。
- 新模块 (3-5 天)：`UnifiedActionLayoutTransform` + `slot_map` 元数据贯通到 `PolicyNormProcessor`；`QwenMultiHead` 框架；`wrap_aware_l1` 与 `wrap_pi` 归一化。

10. 代码质量与可改进点

10.1 耦合处

位置	问题
<code>trainer_tools.py:L108-125</code> vs <code>L192-234</code>	冻结字符串在 <code>build_param_lr_groups</code> 与 <code>freeze_backbones</code> 各解析一次，语法变更需同步两处
<code>base_framework.py:L191, L246-252</code> 、 <code>train_starvla_cotrain.py:L376, L405</code>	属性名 <code>qwen_vl_interface</code> 被基类与训练器硬编码，非 Qwen 骨干 (WM4A、EgoVLA) 也被迫沿用
<code>VLM4A/*.py:L30</code> (如 <code>QwenOFT.py</code>)	模型层 import <code>deployment.model_server.tools.image_tools.to_pil_preserve</code> ，模型包依赖部署包 (依赖倒置)
<code>QwenGR00T.py:L14-17</code> 、 <code>WM4A/*.py:L24-26</code>	框架文件在 import 时把仓库根目录插入 <code>sys.path</code>
<code>vlm_datasets.py:L238-246</code>	VLM 数据 labels 依赖硬编码 Qwen token id (77091、151645)，非 Qwen 骨干无法共训
<code>Qwen3.py:L126-130</code>	VLM 接口读取 <code>config.datasets.vla_data.CoT_prompt</code> ——骨干包装依赖数据配置节点

10.2 重复代码

- 三个训练脚本 (`train_starvla.py`、`train_starvla_cotrain.py`、`train_starvlm.py`) 约 80% 相同，应抽成 `BaseTrainer` + 策略子类 (文件头注释本身也承诺"每种策略一个 `trainer_*.py`"，但没有共享基类)。
- `GR00T_ActionHeader.py`、`LayerwiseFM_ActionHeader.py`、`AML_ActionHeader.py` 各自复制了 `CategorySpecificLinear/CategorySpecificMLP/MLP/ActionEncoder/MultiEmbodimentActionEncoder/FlowmatchingActionHeadConfig` (前 190 行几乎逐行相同)。
- `QwenPI_v3.py:L403-423` 重新定义了 `state2str_transform/add_discretized_state_to_instruction`，而 `share_tools.py:L534-559` 已有同名函数 (`QwenOFT` 用的是后者)。

- `share_tools.read_model_config` (读 `config.json`) 与 `read_model_config` (读 `config.yaml`) 并存, `tools.py:L185-189` 再包一层 `re-export`。
- `mixture.py:L15-21` 与 `L45-51` 重复定义 `custom_dataset`、`custom_dataset_2` 键。
- 每个框架文件重复 60 行 `__main__` smoke test 样板。

10.3 测试缺口 (tests/)

41 个测试覆盖: 配置覆盖/推理期 override (21 个)、GR00T ZMQ 协议 (4)、MiniCPMRobotManip 掩码 loss 与数据打包 (4)、`preprocess_images` (2)、OpenPI Gemma 兼容 (2)、Robocasa 客户端请求 (4)、单进程无 `dist` 安全 (4)。**未覆盖**: 四个核心框架的 `forward/predict_action` 形状契约、`LeRobotMixtureDataset` 采样与统计合并、`apply_config_compat`、`freeze_backbones/build_param_lr_groups`、训练循环、`PolicyNormProcessor.unapply`。无 `.github/workflows`, `Makefile` 只有 `black/ruff`; 测试需 `torch`, 无 CPU 级 fake-VLM fixture。

10.4 配置散乱

- `yaml` 中存在但从未被读取的键: `trainer.gradient_accumulation_steps` (实际由 `ds_config.yaml` 决定, 值为 1)、`trainer.gradient_checkpointing` (仅 `train_starvln.py` 与 `MiniCPMRobotManip` 读)、`trainer.max_grad_norm`、顶层 `trainer.weight_decay` (生效的是 `trainer.optimizer.weight_decay`)、`trainer.loss_scale.vla`、`datasets.vla_data.action_type/sequential_step_sampling/load_all_data_for_training`。
- `ds_config.yaml`、`zero3.yaml` 是 JSON 内容却用 `.yaml` 后缀; `deepspeed_zero3.yaml` 缺 `num_processes`。
- 同一含义两套键: `action_horizon` vs `future_action_window_size` (兼容层修补); `framework.qwenvl.base_vlm` 同时用于 VLM、世界模型 (`CosmoPredict20FT` 默认配置里 `qwenvl.base_vlm` 指向 `Cosmos` 权重)。
- `repeated_diffusion_steps` 读取位置不一致: `QwenGR00T` 从 `framework.action_model` (默认 8), `QwenPI_v3` 从 `trainer` (默认 16, `QwenPI_v3.py:L315-317`), 其 `DefaultConfig` 中 `action_model.repeated_diffusion_steps: 2` 永不生效。
- `starVLA/config/training/*.yaml` 与 `examples/*/train_files/*.yaml` 两套并存, 前者指向 `Qwen2.5-VL-3B` 旧默认。

10.5 潜在 bug (均已核对代码)

文件:行	问题
<code>starVLA/model/modules/vlm/Qwen3.py:L52-53</code>	<code>attn_implementation = "sdpa"</code> 无条件覆盖配置, <code>Qwen3-VL</code> 永远不用 <code>flash_attention_2</code> (文档与 <code>yaml</code> 都要求安装 <code>flash-attn</code>)
<code>starVLA/model/modules/action_model/fast_ActionHeader.py:L85-89</code>	<code>decoder_action</code> 引用不存在的 <code>self._ACTION_TOKEN_MIN</code> (调用即 <code>AttributeError</code>); <code>_load_fast_processor</code> (L23-65) 定义后从未使用; 默认路径 <code>playground/Pretrained_models/fast</code> 为本地路径
<code>starVLA/model/framework/VLM4A/ABot_M0.py:L134-135</code>	<code>VGGT</code> import 被注释, 构造时 <code>NameError</code>
<code>starVLA/dataloader/gr00t_lerobot/data_config.py:L24, L126</code>	<code>GR00TTransform</code> import 被注释但 <code>OxeDroidDataConfig.transform()</code> 仍调用, 选 <code>oxe_droid</code> 即 <code>NameError</code>
<code>starVLA/training/trainer_utils/trainer_tools.py:L141</code>	<code>ReferenceError(...)</code> 只构造不 <code>raise</code> , 学习率组路径写错时静默
<code>Qwen3.py:L164</code> 、 <code>Qwen2_5.py:L295</code>	<code>RuntimeWarning(...)</code> 只构造不 <code>warn</code>
<code>starVLA/model/framework/VLM4A/QwenPI_v3.py:L315-317</code>	<code>repeated_diffusion_steps</code> 读错节点, 见 10.4
<code>examples/simBenchmarks/LIBERO/train_files/run_libero_train.sh:L45-46</code>	<code>--trainer.vla_data.video_backend</code> 路径写错 (应为 <code>datasets.vla_data</code>); <code>--trainer.freeze_modules</code> <code>\${freeze_module_list}</code> 变量为空时被 <code>shell</code> 吞掉, <code>normalize_dotlist_args</code> 解析成布尔 <code>true</code> , <code>freeze_backbones</code> 因非字符串而跳过——碰巧等价于"不冻结"
<code>starVLA/training/train_starvln_cotrain.py:L196</code>	<code>is_resume</code> 时访问不存在的 <code>self.config.resume_from_checkpoint</code>
<code>starVLA/training/trainer_utils/trainer_tools.py:L299</code>	整模型加载 <code>strict=False</code> , 键名漂移时静默丢权重; 与 <code>from_pretrained</code> 的 <code>strict=True</code> 行为不一致
<code>starVLA/model/framework/base_framework.py:L186-252</code>	<code>supports_training_tag/compute_loss/forward_vlm</code> 无调用者, <code>docstring</code> 提到的 <code>DataLoaderManager</code> 不存在
<code>docs/model_zoo.md:L19</code>	" <code>Qwen2.5-GR00T-Bridge-RT-1</code> " 链接指向 <code>Qwen-PI-Bridge-RT-1</code>

10.6 文档与代码不一致

- `README.md:L288`、`docs/faq.md:L33` 引用 `starVLA/training/train_internvla.py` (不存在); `faq.md:L95` 用 `--framework.framework_py`, 而 `build_framework` 只接受 `framework.name` (否则 `ValueError`, `base_framework.py:L93-94`)。
- `docs/starVLA_guideline.md:L171` 与 `run_libero_train.sh` 注释写 `QwenFAST`, 注册键是 `QwenFast` (`QwenFast.py:L84`), 注册表区分大小写会抛 `NotImplementedError`。
- `README/FAQ` 说冻结支持 "regex", 实现为精确路径。

- modules/vlm/README.md:L38-42 说 cosmos-reason2 委托给 get_world_model，代码直接 import vlm.CosmosReason2 (vlm/__init__.py:L33-38)；其"现有实现"表缺 CosmosReason2.py、VILA.py、OpenPIPaliGemma.py。
- docs/W4A.md:L56, L134 声称 7 种组合含 WM4A-OFT.py，目录只有 6 个文件；表格写 Cosmos hidden 2048、world_model/README.md 写 4096。
- examples/eval_protocol.md:L50 仍用 result["normalized_actions"]，服务器已改为返回反归一化的 data["actions"]；L65 链接 model2libero_client.py 不存在。
- starVLA_guideline.md:L7-9 链接 docs/integrate_your_dataset.md 与 agent_skills/.../README.md，本快照均不存在。
- run_libero_train.sh 默认 Framework_name=QwenPI、base_vlm=Qwen3.5-0.8B，与指南描述的 QwenOFT + Qwen3-VL-4B 不一致。
- 技术报告 §7 的"统一 32 维 padding 多基准共训"在代码中未找到通用实现。

10.7 设计层面的改进建议

1. 抽出 BaseTrainer，把三份训练脚本收敛为"数据源策略"差异。
2. 把 freeze_modules 解析、reload_modules 解析、learning_rate 分组统一到一个 ParamPolicy 对象，并支持正则。
3. 让 _train_step 消费 compute_loss 的多键 loss 字典（基类已设计），顺带记录所有标量键。
4. 把 to_pil_preserve 移到 starVLA/model/tools.py，解除模型层对 deployment 的依赖。
5. 为四个核心框架增加 CPU 级契约测试（用 2 层随机初始化的小 Qwen3-VL 配置）。
6. 引入 dataloader 层的 action_mask 一等字段（_pack_sample 输出），使掩码从 UMI 特例变成通用机制。

11. 新人上手路径：安装 → LIBERO 训练 → 评测

以下命令来自 docs/starVLA_guideline.md 与对应脚本，按最短路径整理。

```
# 0. 安装 (guideline §0)
git clone https://github.com/starVLA/starVLA && cd starVLA
conda create -n starVLA python=3.10 -y && conda activate starVLA
pip install -r requirements.txt          # transformers==4.57.0, accelerate==1.5.2, deepspeed==0.16.9 ...
pip install flash-attn --no-build-isolation # 验证过 2.7.4.post1 + nvcc 12.0/12.4
pip install -e .

# 1. 基座模型 + smoke test (guideline §1, §3)
huggingface-cli download Qwen/Qwen3-VL-4B-Instruct --local-dir playground/Pretrained_models/Qwen3-VL-4B-Instruct
python starVLA/model/framework/VLM4A/QwenGR00T.py # 打印结构并对假数据跑 forward/predict_action

# 2. 数据 (guideline §2): 4 个 LIBERO 子集 + LLaVA-OneVision-COCO, 复制 modality.json 到各 meta/
export DEST=/path/to/data && bash examples/simBenchmarks/LIBERO/data_preparation.sh
python starVLA/dataloader/lerobot_datasets.py --config_yaml examples/simBenchmarks/LIBERO/train_files/starvla_cotrain_libero.yaml

# 3. 训练 (guideline §5-6): 编辑 run_libero_train.sh 顶部变量
# Framework_name=QwenOFT | QwenFast | QwenPI_v3 | QwenGR00T ; base_vlm=playground/Pretrained_models/Qwen3-VL-4B-Instruct
# data_mix=libero_all ; run_id=<name> ; 按 GPU 数改 --num_processes 与 deepspeed_zero2.yaml 的 num_processes
bash examples/simBenchmarks/LIBERO/train_files/run_libero_train.sh
# 等价展开:
# accelerate launch --config_file starVLA/config/deepseeds/deepspeed_zero2.yaml --num_processes 8 \
#   starVLA/training/train_starvla.py --config_yaml examples/simBenchmarks/LIBERO/train_files/starvla_cotrain_libero.yaml \
#   --framework.name QwenOFT --framework.qwenvl.base_vlm playground/Pretrained_models/Qwen3-VL-4B-Instruct \
#   --datasets.vla_data.data_mix libero_all --trainer.max_train_steps 80000 --run_root_dir results/Checkpoints --run_id my_run
# 产物: results/Checkpoints/my_run/{config.yaml, config.full.yaml, dataset_statistics.json, checkpoints/steps_*_pytorch_model.pt}

# 4. 评测环境 (guideline §7 step 0)
conda create -n libero python=3.10 -y && conda activate libero
bash examples/simBenchmarks/LIBERO/eval_files/install_libero.sh # mujoco==3.2.3 + LIBERO + tyro/mediapy/websockets/msgpack/numpy==1.24.4

# 5. 策略服务器 (终端 1, starVLA 环境): 编辑 run_policy_server.sh 的 CKPT, 或下载官方权重
huggingface-cli download StarVLA/Qwen3-VL-OFT-LIBERO-4in1 --local-dir playground/Pretrained_models/StarVLA/Qwen3-VL-OFT-LIBERO-4in1
conda activate starVLA && bash examples/simBenchmarks/LIBERO/eval_files/run_policy_server.sh # 等待 "server listening on 0.0.0.0:6694"

# 6. 评测客户端 (终端 2, libero 环境)
conda activate libero && export MUJOCO_GL=egl PYOPENGL_PLATFORM=egl
bash examples/simBenchmarks/LIBERO/eval_files/eval_libero.sh # 每任务 50 回合, 视频与成功率写到 results/{suite}/{ckpt}/
```

注意事项: (1) 训练 yaml 默认 freeze_modules: 'qwen_vl_interface'，脚本用 CLI 覆盖，改脚本时留意；(2) --framework.name 大小写必须与注册键一致（QwenFast）；(3) Qwen3-VL 接口当前强制 sdpa，flash-attn 装不上也能跑；(4) 8xA100/H800 上 libero_all 约 30K 步 ≈ 10 epoch；(5) 单卡调试可设 STARVLA_DISABLE_DEEPSPEED=1 直接 python starVLA/training/train_starvla.py ...。

03 · StarVLA-α 解读：把 VLA 系统的复杂度降下来

项目	内容
论文	StarVLA-α: Reducing Complexity in Vision-Language-Action Systems
arXiv	2604.11757 , 2026-04; ECCV 2026
作者	Jinhui Ye†, Ning Gao†, Senqiao Yang, Jinliang Zheng, Zixuan Wang, Yuxin Chen, Pengguang Chen, Yilun Chen‡, Shu Liu, Jiaya Jia‡ (HKUST / XJTU / CUHK / THU / 通义实验室 / SmartMore)
本仓库文件	英文 PDF · 中文 PDF

0. 一句话

一个刻意做到最简的 VLA 基线——Qwen3-VL-4B + 读取专用 action token 隐状态的 MLP 头、原始 RGB + 指令输入、训练集 z-score 归一化、不用状态/历史帧/机器人预训练——在 LIBERO、SimplerEnv、RoboTwin 2.0、RoboCasa-GR1 四个基准上打平或超过 π 0.5、GR00T-N1.6、OpenVLA-OFT；把四个基准的数据合在一起训一个 generalist，在 GR1 上反而更高（57.3 vs 53.8）；真机 RoboChallenge ARX5 11 任务成功率 33.6 vs π 0.5 的 12.7。它的价值不是数字，而是提供了一块去掉混杂因素的对照板，用来重新审视 VLA 社区习以为常的设计。

1. 动机：VLA 领域的"巴别塔"

论文开篇的诊断（图 1）：现有 VLA 系统在架构（视觉塔、语言骨干、动作专家）、预训练数据、本体配置、基准专用工程四个维度上同时变化，报告的提升无法归因到具体的建模创新。VLM 领域已经收敛出标准配方（LLaVA 系列），VLA 还没有。StarVLA-α 的目标是方法论上的清晰，而不是架构新颖性。

2. 设计：最小充分性假设

假设：一个强 VLM 配一个轻量动作头，就能拿到通常归功于复杂设计的大部分收益。

- **最小数据处理**：所有环境共用一条数据管线；输入原始 RGB + 语言指令；动作只用训练集统计做零均值单位方差归一化；评测严格按各基准官方协议。
- **干净架构**：Qwen3-VL 原生处理视觉与语言，省掉挑选和拼接 CLIP/SigLIP/DINO 的步骤；顶层接一个 MLP，读取指定 action token 的隐状态，回归一个动作块。
- **统一基准集成**：异构性只允许存在于"薄适配器"里（观测格式、动作接口、评测入口），同一模型和配方跑遍所有基准。

训练细节（附录 C）：骨干 lr 1e-5、动作头 lr 1e-4、cosine 调度、每 GPU batch 16、最多 100k 步；LIBERO 8x A100，SimplerEnv / GR1 / RoboTwin-Clean 16x A100，RoboTwin Clean+Rand 48x A100，全基准联训 64x A100。

3. 主结果（表 1）

方法	LIBERO avg	SimplerEnv WidowX / Google VA / VM	RoboTwin clean / clean* / random*	RoboCasa-GR1
OpenVLA-OFT	97.1	31.3 / 54.3 / 63.0	—	—
π 0	94.1	27.1 / 54.8 / 58.8	46.4 / 65.9 / 58.4	—
π 0.5	96.9	46.9 / 68.4 / 72.7	60.2 / 82.7 / 76.8	37.0
GR00T-N1.6	97.0	62.0 / 65.3 / 67.7	—	47.6
StarVLA-α (specialist)	98.8	64.6 / 70.2 / 76.0	50.3 / 88.2 / 88.3	53.8
StarVLA-α (generalist)	97.8	65.2 / 69.8 / 74.3	— / 88.7 / 87.8	57.3

* 表示 clean + random 数据都用于训练。RoboTwin Base（仅 clean 50x50）上 π 0.5 的 60.2 高于 StarVLA-α 的 50.3——这是全表中最简基线输给对手的唯一一格，低数据 + 双臂关节角是它的弱点。

4. 三个"常识"的重新检验（第 3 节）

4.1 动作头设计重要吗？（表 2）

同一骨干换四种头：

头	LIBERO	WidowX	RoboTwin clean*	GR1
MLP（OFT 式）	98.8	64.6	88.2	53.8
FAST（离散 token）	97.8	35.6	72.5	45.0
GR00T（双系统流匹配）	98.7	65.3	88.0	52.8
π （流匹配专家）	98.1	65.9	88.1	48.9

结论：连续 > 离散（FAST 在 WidowX 上低 29 个点）；三种连续头之间差距很小。骨干够强时，动作头的额外复杂度是不必要的。这一发现是 VLAct pilot study 的直接前身——VLAct 进一步追问“同头表现好是否等于骨干可复用”，答案是否。

4.2 现有的动作预训练重要吗？（表 3）

中间预训练	轨迹数	RoboTwin Clean 50×50	+Random×500	GR1 24×10	GR1 24×1000
无	—	50.3	88.2	9.8	53.8
+ OXE	232.6k	30.2	83.6	1.2	27.8
+ InternData–A1	630k	63.6	88.6	2.8	35.4
+ RoboTwin–Rand	25k	79.7	88.8	2.2	33.3

这是全文最重要的一张表。跨域数据（OXE）全面拖后腿；同域数据（InternData–A1、RoboTwin–Rand）只在目标域的低数据段有帮助，换到 GR1 一律掉点，最惨的把 9.8 打到 1.2。作者的措辞是“双刃剑，应谨慎使用”。

这张表和四个月后 VLAct 的结果构成了一组对照：VLAct 用同样的 InternData–A1（加 DROID、RoboCoin、MolmoAct），换成表示中心配方后，GR1 从 48.8 涨到 54.0。朴素动作拟合会伤害跨本体迁移，问题在配方不在数据。详见 01 · VLAct 精读 第 6 节。

4.3 数据工程必要吗？（表 4）

本体感受、历史帧、delta 动作、relative 动作四项：低数据段（RoboTwin Clean 50×50、GR1 24×10）有 1–10 个点的小幅提升，本体感受在 RoboTwin 低数据段 +10.5 最明显；数据充足后全部回到基线水平；历史帧在多数设定下反而略降。

5. All-in-one Generalist 评测（第 4 节）

作者提出一个评测范式主张：像 LLM 领域一样，用一个模型跑所有基准来检验泛化，而不是每个基准单独微调。实现方式极简——把所有本体的动作零填充到 32 维，联合训练，不做任务特定设计。

分析部分的四个发现：

- 本体专用动作设计不必要（表 6）：零填充 vs RDT 动作空间 vs 多动作头，GR1 上 57.3 / 52.3 / 53.5，Google VM 上 74.3 / 71.4 / 67.8。
- 模型规模（表 10）：2B→4B 在 WidowX +18.1、GR1 +6.6；4B→8B 增益 <1%。4B 是当前训练规模下的甜点。
- 初始化质量（表 9）：随机初始化 GR1 28.8，Florence–2 39.2，Qwen2.5–VL 53.6，Qwen3–VL 57.3，Qwen3.5 56.1。骨干先验直接决定跨基准泛化上限。
- batch size（表 11）：64→1024，GR1 从 40.0 涨到 59.2，RoboTwin clean 从 80.4 到 88.8，几乎单调。作者认为 batch 多样性是 generalist 训练里最重要的优化因素，影响比模型规模更大、更一致。

6. 真机（第 5 节 + 附录 E/F）

- RoboChallenge Table–30（ARX5，11 任务）：SR 33.6 / 进度分 54.5， π 0.5 为 12.7 / 27.6， π 0 为 3.6 / 14.7。put cup on coaster 100% SR，fold dishcloth 0%。
- Franka OOD（附录 F）：三个任务（垃圾分类、拾取彩蛋、蛋托格子放置）ID 均值 85.3，OOD 均值 76.3；蛋托放置在未见行列组合上从 91.3 掉到 68.8，是最敏感的一项。

7. 评价

贡献

1. 提供了一块可复现、去混杂的对照板。后续 StarVLA 生态的多篇工作（VLAct、ST4VLA）都以它为基线。
2. 表 3 的“预训练双刃剑”是对领域默认假设的一次有数据支撑的质疑，直接催生了 VLAct 的问题定义。
3. “generalist 评测范式”和 batch size 结论对工程实践有直接指导意义。

局限

1. 所有结论建立在 Qwen3–VL–4B 上；“动作头不重要”在弱骨干（Florence–2）或更大骨干上是否成立没有验证。
2. 表 3 只比较了三种预训练数据源，没有区分“数据分布不匹配”与“训练方式不当”两种解释——VLAct 后来证明是后者，但 StarVLA– α 本身的表述容易被读成“预训练没用”。
3. RoboTwin Base 设定 50.3 低于 π 0.5 的 60.2，低数据 + 关节角控制是最简基线的明显短板，论文没有深入分析。
4. 附录 E 引用了一张不存在的表（“Table ??”），RoboChallenge 其他三个平台（UR5、Franka、ALOHA）的数字实际未给出。
5. 32 维零填充在只有四类本体时够用；本体数量增加后，填充维度和不同本体间坐标语义冲突的问题会放大，VLAct 表 12 已经显示“部分统一”比“朴素统一”多 1 个点。

8. 与本仓库其他材料的关系

- 代码实现：`starVLA/model/framework/VLM4A/Qwen0FT.py` 即 StarVLA– α 默认配置，见 02 · 代码库解析。

- 后续演进: [01 · VLAct](#) 在同一骨干上回答"如何让预训练不再是双刃剑"。
- 基准细节: [06 · 基准生态](#)。

04 · StarVLA 技术报告解读：一个乐高式 VLA 代码库的设计契约

项目	内容
论文	StarVLA: A Lego-like Codebase for Vision-Language-Action Model Developing
arXiv	2604.05014 , 2026-04 (持续更新的技术报告)
作者	StarVLA Community & Von Neumann Institute, HKUST
代码	https://github.com/starVLA/starVLA (MIT, ~3.6k stars, 源自 InternVLA-M1 fork)
本仓库文件	英文 PDF · 中文 PDF

0. 一句话

StarVLA 把 VLA 系统拆成"骨干 — 动作头"两个独立可换的部件，用两条契约（外层：原始观测 → 动作；内层：多模态输入 → hidden state → 动作）把 FAST / OFT / π / GR00T 四种解码范式、VLM 与视频世界模型两类骨干、SFT / 多模态共训 / 跨本体共训三种训练模式，以及 LIBERO、SimplerEnv、RoboTwin 2.0、RoboCasa-GR1、BEHAVIOR-1K 等基准接进同一套训练与评测管线。它的目标不是提出新模型，而是让"只改一个变量"的受控实验成为默认工作方式。

1. 问题：三个层面的碎片化

- **架构层**：自回归 token、并行回归、diffusion、flow matching 各成一派，跨范式比较困难。
- **系统层**：方法发布时把模型、数据处理、训练管线耦死，部件无法复用。
- **评测层**：各自报告不同基准子集、协议不一致，公平比较不可行。

作者把根因归为"缺少统一抽象"，现有代码库（OpenPI、Isaac-GR00T、OpenVLA-OFT、Dexbotic、X-VLA）都不同时支持可换动作头、可换 VLM、世界模型骨干、混合数据加载、开源多模态/跨本体共训、多基准联训（表 1）。StarVLA 是唯一全勾选的一行，集成基准数 7。

2. 两条契约（第 2.2 节）

统一 I/O 接口。所有 framework 继承同一基类，暴露两个方法：

- `forward({raw images, str, ...}) → loss dict`：训练入口，吃原始多视角 RGB + 指令 + 动作块。
- `predict_action({raw images, str, ...}) → {normalized_actions, ...}`：推理入口，吃同样格式（去掉 GT 动作）。

训练输入与部署观测同构，消除了 VLA 系统里常见的"dataloader 预处理张量 vs 真机原始流"的静默分布错配。作者把这条约定称为"部署时不变量"：无论骨干怎么预训练、怎么切图、用什么 tokenizer，推理时都必须吃机器人给的原始传感流。

组合式框架。内部再拆成 VL 骨干（吃原始观测、按统一输出规格给 hidden state）和动作头（按统一输入规格读 hidden state、产生动作），两步组装（先加载骨干，再挂头），全部由 YAML 声明式配置。两条边界都受契约约束，骨干和头可以互不影响地独立替换。

3. 四种实例（第 2.3 节）

变体	做法	对应文献
StarVLA-FAST	接 FAST tokenizer，在 LLM 自己的词表空间里自回归生成离散动作 token	π 0-FAST
StarVLA-OFT	轻量 MLP 读预定义 action token 的 hidden state，并行回归连续动作，L1 loss	OpenVLA-OFT
StarVLA- π	逐层 cross-DiT 流匹配动作专家，通过 cross-attention 条件于多层 VL hidden state，迭代去噪	π 0
StarVLA-GR00T	双系统：VL 骨干为 System 2（慢推理），DiT 流匹配模块为 System 1（快生成）	GR00T N1.5

四者共享骨干、基类和 forward/predict_action 契约，只在"如何从骨干表征里取动作"上不同。新范式只需实现并注册一个动作头。

4. 训练管线（第 3 节）

三种训练模式对应三个入口脚本，全部是显式 PyTorch 循环 + Accelerate + DeepSpeed：

模式	脚本	要点
行为克隆 SFT	<code>train_starvla.py</code>	目标是 forward 返回的 <code>action_loss</code> ；支持全参微调或 <code>trainer.freeze_modules</code> 按模块路径冻结； <code>trainer.learning_rate</code> 按模块分组 lr；bf16、梯度累积、梯度裁剪、带下限的 cosine
多目标共训	<code>train_starvla_cotrain.py</code>	双 dataloader（VLA + VLM），每步两次前向/反向：一次 <code>framework.forward</code> 得 <code>action_loss</code> ，一次 <code>qwen_vl_interface</code> 得 LM loss，后者乘 <code>trainer.loss_scale.vlm</code>
跨本体共训	配置项 <code>datasets.vla_data.data_mix</code>	一个命名 mixture 映射到（数据集，采样权重，机器人类型）列表，运行时物化为 <code>LeRobotMixtureDataset</code> ，按权重采样并按机器人类型打本体标签。跨本体预训练是"配置选择"而不是专用脚本
RL 微调	与 RLInf 合作	报告撰写时仍在集成中

5. 评测与部署（第 3.2 节）

薄 server—client 抽象：checkpoint 由 `baseframework.from_pretrained()` 加载，作为轻量 WebSocket 策略服务器运行在 StarVLA 环境；基准评测器在自己的 conda 环境（各模拟器依赖不同）里通过小客户端封装访问，payload 用 msgpack 序列化，包含 `image`、`lang`、可选 `state` 等字段，返回 `normalized_actions`。

基准差异被隔离在 `model2libero_interface.py`、`model2simpler_interface.py`、`model2robotwin_interface.py` 这类适配器里：缩放图像到训练分辨率、读 checkpoint 目录下的 `dataset_statistics.json` 反归一化、把归一化的 chunk 转成可执行动作、动作 ensembling、sticky gripper、delta/relative → absolute 转换。

真机部署复用同一契约：机器人控制器扮演客户端，采图、组同样的字典、查询远端策略服务器、执行返回动作。控制回路、安全逻辑、厂商 SDK / ROS 全在模型运行时之外。同一 checkpoint 可以不改代码从仿真搬到真机（RoboChallenge 就是这样跑的）。

6. 基准集成与基线结果（第 4—5 节）

每个基准的接入由三件对齐的东西组成：checkpoint 包（`config.yaml` + `dataset_statistics.json`）、可运行训练入口（`examples/<BENCH>/train_files/`）、可运行评测流程（`examples/<BENCH>/eval_files/`）。

LIBERO（表 2）——一个策略跑四个套件，8x A100，只训 30K 步（约 9.5 epoch）：

模型	步数	Epochs	Spatial	Object	Goal	Long	Avg
OpenVLA-OFT	175K	223	97.6	98.4	97.9	94.5	97.1
GR00T-N1.5	20K	203	92.0	92.0	86.0	76.0	86.5
StarVLA-FAST (Qwen3-VL-4B)	30K	9.54	97.3	97.4	96.3	90.6	95.4
StarVLA-OFT (Qwen3-VL-4B)	30K	9.54	97.8	98.6	96.2	93.8	96.6
StarVLA- π (Qwen3-VL-4B)	30K	9.54	98.8	99.6	95.8	88.4	95.7
StarVLA-GR00T (Qwen3-VL-4B)	30K	9.54	97.8	98.8	97.4	92.0	96.5
StarVLA-OFT (Cosmos-Predict2-2B)	30K	9.54	98.6	97.6	95.0	91.8	95.8

两个信息：OpenVLA-OFT 用 6 倍步数、23 倍 epoch 才多 0.5 个点；把骨干从 Qwen3-VL-4B 换成视频世界模型 Cosmos-Predict2-2B，三种头平均仍 ≥95.2，“骨干可换”不是口号。

SimplerEnv：16x A100，Bridge + Fractal 混合训练，每个设定跑 5 次完整官方评测取均值。WidowX VM 最高 65.3%（Qwen3-VL-4B）、61.6%（Cosmos-Predict2-2B）。

7. 多模态共训案例（第 6 节，引用 ST4VLA）

动作-only 微调会在数千步内让 VLM“忘掉”预训练能力：RefCOCO-g IoU@0.5 在 20K 步内掉到接近随机。共训的机制是维持感知相关通路上的梯度流。ST4VLA（基于 StarVLA 的空间引导共训研究）表 8：

策略	MME	RefCOCO-g IoU@0.5	RoboRefit Acc@0.5	Google VM
Vanilla VLA	—	—	—	66.1
+ 共训	1106	47.1	66.7	70.2
+ 空间引导	1374	68.1	72.5	78.8
+ 空间预训练	1411	71.2	74.3	84.6

共训不只是“保住多模态能力”，它同时把操作成功率从 66.1 拉到 84.6。VLAct 的 caption 混训与这条线一脉相承，只是选择了 caption 作为最强单源锚点。

8. 计算效率（第 8 节，数据来自 issue #158）

测量对象：StarVLA-GR00T + Qwen3-VL-4B，RoboCasa-GR1 数据，A100 80GB。

单节点（8x A100）：每 GPU batch 2→24，步延迟 0.703→2.404 s，样本吞吐 22.7→79.9 samples/s，GPU 利用率最高 96%。每 GPU batch 8 是延迟与利用率的平衡点。

多节点（每 GPU batch 8）：

GPU 数	全局 batch	秒/步	samples/s	扩展效率
8	64	0.735	87.0	100%
32	256	0.899	284.7	81.9%
64	512	0.925	553.8	79.6%
256	2048	0.931	2200.0	79.1%

跨节点通信带来一次性 ~0.2 s/步的开销，之后到 256 GPU 都平台化，扩展效率稳在 79—80%。实践指南：固定步数的训练不会因为加 GPU 变快；数据量驱动的训练可以放心扩到数百卡。结合 StarVLA- α 表 11 "batch size 是 generalist 训练最重要的优化因素"，大全局 batch 的价值在两篇论文里被同时确认。

9. "广义 VLA 视角"

作者从工程统一中提炼出一个观点：VLM 基方法与世界模型基方法不是两种范式，而是同一结构框架下的变体，差别主要在**辅助学习信号**的形式——语言对齐的推理，还是未来观测预测。这个观点的价值在于它给出了一个可操作的研究纲领：把"辅助信号"当作第三个可换部件，与骨干、动作头并列。VLAct 的 caption 共训、WM4A 的未来帧预测、ST4VLA 的空间 grounding，都是这个第三轴上的点。

10. 评价

优点

1. 两条契约的设计在报告里讲得很清楚，且代码确实这么做了（见 [02 · 代码库解析](#)）。
2. 表 2 的 LIBERO 基线把"步数 / epoch"列出来，是少数把训练成本和成功率放在同一张表里的报告。
3. 效率章节的数据来自公开 issue，可追溯。

局限

1. 报告以"持续更新"为前提，第 4.2 节声明的五个基准与 README 里的 13 个不一致；读者应以代码库为准。
2. 除 LIBERO 与 SimplerEnv 外，RoboTwin、GR1、BEHAVIOR 的详细结果在报告正文中较薄，多引用 StarVLA- α 。
3. 世界模型骨干（Cosmos-Predict2）只在 LIBERO 上展示，WM4A 在其他基准上的表现未给出。
4. 效率测量只覆盖 GR00T 头；FAST 的自回归解码和 π 的迭代去噪在推理侧的延迟差异没有量化，而这正是部署时选头的关键依据。

11. 与本仓库其他材料的关系

- 代码级细节（每个契约对应哪个类、哪一行）：[02 · 代码库解析](#)
- 用这套基础设施做的两项研究：[03 · StarVLA- \$\alpha\$](#) 、[01 · VLAct](#)
- 支持的基准逐个拆解：[06 · 基准生态](#)

05 · 动作头与动作表示：原理、证据与选型

StarVLA 生态的三篇论文共享同一个骨干（Qwen3-VL-4B）和同一套代码，这让“动作头”这个变量第一次可以被单独隔离出来比较。本文把四种头的数学形式、三篇论文给出的全部对照数字，以及“动作空间怎么设计”的证据汇总在一处，最后给一个选型指南。

1. 四种头的形式化

记骨干在时刻 t 的多模态 hidden state 为 H_t ，待预测的动作块为 $A_t = (a_t, \dots, a_{t+K-1})$ ， K 为 chunk 长度。

FAST：自回归离散 token

把连续动作块经 FAST tokenizer（DCT 压缩 + BPE）编成 token 序列 $z_{1:M} = \text{Tok}(A_t)$ ，在 LLM 自己的词表空间里做 next-token 预测：

$$\mathcal{L}_{\text{FAST}} = - \sum_{m=1}^M \log p_{\theta}(z_m \mid H_t, z_{<m})$$

推理时逐 token 生成再逆 tokenizer。优点：不改 VLM 架构；代价：连续控制经过离散瓶颈，且解码是串行的。

OFT：并行连续回归

在输入序列末尾拼 K 个 action query token，取其 hidden state $h_{t,k}^{\text{act}}$ 用小 MLP g_{ϕ} 逐个回归：

$$\hat{a}_{t+k} = g_{\phi}(h_{t,k}^{\text{act}}), \quad \mathcal{L}_{\text{OFT}} = \frac{1}{Kd_a} \sum_k \|g_{\phi}(h_{t,k}^{\text{act}}) - a_{t+k}\|_1$$

一次前向输出整个 chunk。它同时是一个诊断工具：如果 OFT 表现好，说明骨干表征里线性可读地暴露了连续控制所需的信息。代价是点估计，不能表达多模态动作分布。

PI：flow-matching 动作专家

学一个把高斯噪声搬运到示教动作块的向量场。取 $\epsilon \sim \mathcal{N}(0, I)$ 、 $\tau \in [0, 1]$ ，线性概率路径 $A_t^{\tau} = (1 - \tau)\epsilon + \tau A_t$ ，目标速度 $u = A_t - \epsilon$ ：

$$\mathcal{L}_{\text{PI}} = \mathbb{E} \left[\|v_{\phi}(A_t^{\tau}, \tau; H_t) - (A_t - \epsilon)\|_2^2 \right]$$

推理从噪声出发做 N 步欧拉积分。StarVLA 实现为逐层 cross-attention 到多层 VL hidden state 的 DiT。

GR00T：双系统 flow matching

同样的 flow-matching 目标，但 DiT 运动模块被显式分离为 System 1，额外条件于本体感受状态 s_t 和本体标识 e ：

$$\mathcal{L}_{\text{GR00T}} = \mathbb{E} \left[\|v_{\phi}(A_t^{\tau}, \tau; H_t, s_t, e) - (A_t - \epsilon)\|_2^2 \right]$$

VLM 作为 System 2 只提供场景与指令的 token；状态与噪声动作在独立运动模块里嵌入、处理、解码回本体原生动作空间。

2. 三篇论文的对照数字

2.1 数据充足时：连续头相当，离散头落后

来源	设定	FAST	OFT	PI	GR00T
StarVLA 报告表 2	LIBERO avg, 30K 步	95.4	96.6	95.7	96.5
StarVLA- α 表 2	LIBERO avg	97.8	98.8	98.1	98.7
StarVLA- α 表 2	SimplerEnv WidowX	35.6	64.6	65.9	65.3
StarVLA- α 表 2	RoboTwin clean*	72.5	88.2	88.1	88.0
StarVLA- α 表 2	RoboCasa-GR1	45.0	53.8	48.9	52.8
VLAct 表 2	RoboTwin Data Scaling Clean (VLAct 骨干)	—	92.5	93.0	89.6

FAST 在 WidowX 上落后 29 个点、RoboTwin 落后 16 个点；三种连续头之间差距在 LIBERO 与 RoboTwin 上不超过 1 个点，GR1 上 PI 低 5 个点。

2.2 数据稀少或扰动强时：回归头领先

来源	设定	OFT	PI	GR00T
VLAct 表 2	RoboTwin Base Clean (VLAct 骨干)	80.5	77.0	76.0
VLAct 表 2	RoboTwin Base Random (VLAct 骨干)	41.5	23.7	22.9
VLAct 表 9	RoboTwin, 从零微调	61.7	60.5	51.2

只用 50 条 clean 轨迹/任务时，两种生成式头在 Random 测试集上比回归头低 18 个点。这是“头无关”结论的明确边界，也是路线图 D3 的出发点。

2.3 预训练头如何影响下游换头（VLAct 表 8/9)

预训练头	下游 PI 微调	相对从零
无	60.5	—
OFT	55.1	−5.4
OFT + GR00T	63.1	+2.6
OFT + PI + GR00T	77.0	+16.5

下游头	从零	同头单头预训练	三头预训练	三头 − 单头
OFT	61.7	78.8	80.5	+1.7
PI	60.5	75.4	77.0	+1.6
GR00T	51.2	71.7	76.0	+4.3

单头预训练会让未见过的头低于从零起点（decoder lock-in）；加第二个头就翻正；三头对同头也有净增益，GR00T 受益最大。

3. 动作空间设计的证据

问题	方案	证据	结论
多本体动作维度不同怎么办	每本体独立头 vs 零填充到 32 维 vs RDT 物理统一空间	StarVLA-α 表 6: GR1 53.5 / 57.3 / 52.3	零填充 + 让 VLM 自己分辨本体，优于两种"专门设计"
零填充之上还能做什么	独立头 vs 统一头 vs 部分统一（夹爪共享、手臂分开、非激活维 mask）	VLAct 表 12: RoboTwin 78.5 / 79.5 / 80.5 ；LIBERO-Plus 81.1 / 81.4 / 82.6	物理同义维度显式共享再拿 1 个点
周期关节角	原始回归 vs 数据侧 wrap 到 $[-\pi, \pi]$ vs + 残差 wrap loss	VLAct 表 10: 75.5 / 78.6 / 80.5	+5 个点，是 VLAct 单项消融里最大的一项
关节 vs 末端、绝对 vs delta	delta / relative 动作	StarVLA-α 表 4: 低数据段 +1~6 个点，数据充足后归零	参数化的收益随数据量衰减
本体感受与历史帧	+proprio / +2 帧历史	StarVLA-α 表 4: proprio 在 RoboTwin 50×50 上 +10.5；历史帧多数设定略降	状态有用，朴素堆帧无用

4. 选型指南

场景	建议	理由
单本体、数据充足（每任务 ≥ 数百条）、追求简单	OFT	与生成式头持平，一次前向，实现最小
低数据（每任务 ≤ 50 条）或强视觉扰动	OFT	RoboTwin Base/Random 领先 18 个点
需要表达多模态动作分布（多解任务、高频接触）	PI 或 GR00T	回归头是点估计
需要状态条件、多本体、独立运动模块的部署形态	GR00T	状态与本体标识进 System 1, VLM 可独立更新
只能改 LLM 输出层、不能加模块	FAST	唯一不改架构的方案，接受 15—30 个点的代价
做持续预训练（塑造骨干供他人复用）	多头共监督 （至少两个连续头）	单头预训练会造成 decoder lock-in
多本体训练	零填充 + 把物理同义维（夹爪）对齐 + 非激活维 mask	StarVLA-α 表 6 与 VLAct 表 12 叠加
关节角控制	数据侧 wrap + 残差 wrap loss，必做	+5 个点，零成本

5. 悬而未决

- 三头预训练的计算开销没有公开数字（路线图 E2）。
- 生成式头在低数据段落后的原因未被诊断：是收敛慢、目标过拟合，还是去噪步数不足（路线图 D3）。
- "头多样性"是否可以推到非动作头（未来帧预测、空间 QA），是路线图 B1/B4 的问题。
- 部分统一布局在人形与灵巧手上如何扩展（路线图 C1）。

代码层面每种头的类、forward 路径与 loss 实现见 [02 · 代码库解析](#)。

06 · StarVLA 支持的评测基准全景与 VLAct 评测协议

撰写日期：2026-09-04。来源标注约定：

- 【P】VLAct 论文 arXiv 2608.27550 (awesome_starvla/papers/en/2608.27550_VLAct.pdf)，页码为 PDF 页码，Tab./Fig. 为论文编号。
- 【R】StarVLA 代码库 starVLA_code/ 下的相对路径 (README、脚本、yaml)。
- 【W】官方论文 / 官网 / 榜单，给出 URL。
- 无出处的数字不写；"—"表示该项在可查来源中没有给出。

0. 范围

StarVLA 在 examples/simBenchmarks/ 下接入 13 个仿真基准 (LIBERO、LIBERO-plus、RoboTwin 2.0、DOMINO、VLA-Arena、RoboDojo、RoboCasa-GR1 tabletop、RoboCasa365、SimplerEnv、BEHAVIOR-1K、CALVIN、MetaWorld MT50、VLN-CE)，在 examples/realRobots/ 下提供 5 套真机/真机数据流程 (Franka、RoboChallenge table30v2、EgoVLA、Realman RM-75、Unitree G1 WholeBody)。主 README 的 "Broad Benchmark Integration" 勾选了 SimplerEnv、LIBERO、LIBERO-plus、Robocasa-GR1、Robocasa365、RoboTwin 2.0、DOMINO、BEHAVIOR、Calvin、RoboDojo 十项，SO101 与 RLBench 未勾选【R: README.md L135-146】。

VLAct 论文在 LIBERO-Plus、VLA-Arena、RoboTwin 2.0、DOMINO、RoboCasa-GR1、RoboDojo 六个仿真基准和 Franka Research 3 真机上报告结果【P §4 p.8-11, App. B p.20-21】。SimplerEnv、BEHAVIOR、CALVIN、MetaWorld、VLN-CE、RoboCasa365 没有 VLAct 数字。

1. 总览表

基准	本体 / 仿真器	任务数	评测集大小 (StarVLA 默认 → 官方)	指标	扰动 / 泛化维度	训练数据规模
LIBERO	Franka Panda / robosuite-MuJoCo 【W】	130 (4 suites; 评测常用 Spatial/Object/Goal/10 各 10 任务) 【W】	10 任务 × 50 episodes / suite 【R: LIBERO/README】	success rate	空间 / 物体 / 目标 / 长程 四类分布偏移 【W】	50 demos/任务 【W】
LIBERO-plus	同 LIBERO	10,030 个测试任务实例，仅测试 【W】	每实例 1 次 rollout (num_trials_per_task=1) 【R: LIBERO-plus/eval_files/eval_libero.sh】	success rate (7 维 + Total)	Camera / Robot / Language / Light / Background / Noise / Layout 【W】	训练只用标准 LIBERO: 官方另有 20K+ 增广轨迹 【W】
RoboTwin 2.0	Aloha-AgileX 双臂 (另支持 5 本体) / SAPIEN 【W】	50 双臂任务 【W】	100 episodes/任务 × clean 与 randomized 两套 【W】	success rate (Easy/Hard; 榜单按 (c2c+c2r)/2) 【W】	clutter / lighting / background / tabletop height / language 五轴域随机化 【W】	Base 50 clean/任务 (2,500); Data Scaling 再加 500 randomized/任务 (25,000) 【P Tab.2 p.9】
DOMINO	5 本体 (franka-panda, ur5-wsg, aloha-agilex, ARX-X5, piper) / SAPIEN, 基于 RoboTwin 2.0 【W】	35 动态任务 【W】	100 episodes/任务 【W】	SR + Manipulation Score (MS) 【W】	运动 Level 1/2/3、动力学系数 α、clean/randomized 【W】	117K 轨迹 【W】
VLA-Arena	Franka / robosuite-MuJoCo 【W】	170 任务, 11 suites, 4 域 【W】	StarVLA 每 level 5 任务 × 10 episodes 【R: VLA-Arena/README】; 官方 30 episodes (10 × 3 seeds) 【W】	SR + Cumulative Cost (Safety 域) 【W】	L0→L1→L2 结构难度; W0-W4 语言扰动; V0-V4 视觉扰动 【W】	只在 L0 微调; VLA-Arena-L0 S/M/L = 10/30/50 轨迹/任务 【W】
RoboDojo	ARX X5 双臂 / Isaac Sim (真机另有 ARX X5, Piper, Piper X) 【W】	42 仿真任务 (Gen 12 / Memory 6 / Long-H 8 / Precision 8 / Open 8) + 18 真机 【W】	50 episodes/任务 → 2,100; 官方榜单要求 3 seeds 【W】	score (部分进度) / success rate 【W】	Gen 拆 25 standard + 25 random; Memory / Precision / Long-Horizon / Open 各成维度 【W】	3,500 轨迹 / 35 任务 (Open 不给训练数据) 【W】
RoboCasa-GR1 tabletop	Fourier GR-1 人形 (ArmsAndWaist + Fourier hands) / robocasa 分支 【W】	24 任务 【W】	50 rollouts/任务 【R: Robocasa_tabletop/README】	success rate	6 个 PnP-to-容器-关闭 + 18 个 "PnP Novel From X To Y" 新物体任务 【R】	1000 demos/任务 (GR00T Teleop Sim) 【W】

基准	本体 / 仿真器	任务数	评测集大小 (StarVLA 默认 → 官方)	指标	扰动 / 泛化维度	训练数据规模
RoboCasa365	Franka Panda + Omron 移动底盘 / robocasa (MuJoCo) 【W】	365 (65 atomic + 300 composite) 【W】	官方 50 rollouts/任务; StarVLA 示例默认 5 【R: Robocasa_365/README】	success rate	Atomic–Seen / Composite–Seen / Composite–Unseen 三 split 【W】	300 任务 × 100 人类示教 = 30K 【W】
SimplerEnv	WidowX (BridgeV2) 与 Google Robot / SAPIEN–ManiSkill 【W】	WidowX 4 任务 + Google Robot 若干 【W】	WidowX 常用 24 trials/任务 【W】	success rate	Visual Matching vs Variant Aggregation 【W】	Bridge + Fractal (OXE) 【R: docs/model_zoo.md】
BEHAVIOR–1K	Galaxea R1 Pro 轮式人形 / OmniGibson (Isaac Sim) 【W】	2025 Challenge 50 项 全流程家务任务 【W】	官方 10 episodes/任务 【W】	Q–score (BDDL 目标谓词部分完成) + 效率指标 【W】	长程、导航+双臂、房屋级场景 【W】	10,000 示教 / 1,200+ h 【W】
CALVIN	Franka Panda / PyBullet 【W】	34 子任务, 4 环境 A–D 【W】	LH–MTLC 1000 条 5 步指令链 【W】	每步 SR + Avg. Len (0–5) 【W】	ABCD→D / ABC→D / D→D 环境泛化 【W】	~24 h 无结构 play 数据 【W】
MetaWorld MT50	Sawyer 单臂 / MuJoCo (Meta–World) 【W】	50 任务, 4 难度桶 (28/11/6/5) 【R: MetaWorld/README】	10 episodes/任务, max 400 步 【R】	success rate (桶均值再平均) 【R】	无显式扰动维度	MT50 LeRobot 数据集 【R】
VLN–CE	Habitat 连续环境 (导航) 【W】	R2R–CE / RxR–CE 【W】	val–unseen split 【W】	NE / OS / SR / SPL / nDTW 【W】	未见场景 【W】	NaVILA–Dataset R2R/RxR 【R: VLN–CE/README】
真机 Franka	Franka (Research 3 / Panda), 7D 或 14D 动作 【R: realRobots/Franka/README】	用户自定	VLAct: 10 rollouts/任务 【P App. J.2 p.32】	success rate; 长程按步计分 【P J.3 p.33】	ID / OOD 物体替换、序列扩展 【P §4.3 p.10】	VLAct: 单臂 50、双臂 100 demos/任务 【P J.2】

2. 逐基准小节

2.1 LIBERO

- 定位：单臂 Franka 桌面操作的"标准答案"基准，最初为终身学习设计，现在是 VLA 微调的默认试金石 【W: papers.neurips.cc LIBERO 2023】。
- 任务构成：130 个语言条件任务，四个 suite：LIBERO–Spatial / Object / Goal 各 10 任务，LIBERO–100 拆成 LIBERO–90（预训练）和 LIBERO–10（长程评测）；每任务 50 条人类遥操作示教 【W: github.com/Lifelong–Robot–Learning/LIBERO】。
- 评测协议（StarVLA）：一个策略同时训 4 个 suite，每个 suite 10 任务 × 50 episodes = 500 trials 【R: examples/simBenchmarks/LIBERO/README.md】；NUM_TRIALS_PER_TASK 默认 50 【R: LIBERO/eval_files/eval_libero.sh】；每 suite 最大步数 spatial 220 / object 280 / goal 300 / libero_10 520，图像 224×224 【R: LIBERO/eval_files/eval_libero.py L79–87；LIBERO–plus 版同值】。成功判定用 LIBERO 自带的 done 信号；初始状态取 task_suite.get_task_init_states。
- 代表性数字 【R: LIBERO/README 表】：StarVLA–OFT(Qwen3–VL) Spatial 97.8 / Object 98.6 / Goal 96.2 / Long 93.8 / Avg 96.6, 30K steps、9.54 epochs; StarVLA–π(Qwen3–VL) 95.7; OpenVLA–OFT 97.1 (175K steps、223 epochs); π0 94.1; π0–FAST 85.5; GR00T–N1.5 86.5。
- 入口：服务端 LIBERO/eval_files/run_policy_server.sh；仿真端 LIBERO/eval_files/eval_libero.sh（需 LIBERO_HOME）；训练 LIBERO/train_files/run_libero_train.sh（80K steps、16/GPU）与 starvla_cotrain_libero.yaml（action_dim 7、action_horizon 8、max_train_steps 100000）；数据 data_preparation.sh 拉取 IPEC–COMMUNITY 的 *no_noops_1.0.0_lerobot 四个子集。已发布的 Qwen3–VL–PI–LIBERO–4in1 需 USE_CANONICAL_FORWARD=false 兼容 PR #373 之前的 LayerwiseFM 语义 【R: LIBERO/README】。
- 已知问题：VLAct 论文直接称其为 "saturated standard LIBERO setting" 【P §4.1 p.8】；LIBERO–plus 论文发现模型对语言扰动几乎不敏感，"tend to ignore language instructions completely" 【W: arxiv.org/abs/2510.13626 Abstract】；README 表中 StarVLA 30K steps 与 OpenVLA–OFT 175K steps 并排比较，训练预算不对齐。

2.2 LIBERO–plus

- 定位：LIBERO 的鲁棒性放大版，回答"高 LIBERO 分数是否等于真能力"。
- 任务构成：从 LIBERO 40 个评测任务出发，7 个扰动维度各生成 500 实例 → 14,000 候选，剔除所有模型都能解的实例后得到 10,030 个测试任务；维度分布 Camera 1599 / Robot 1550 / Language 1537 / Light 1142 / Background 1076 / Noise 1601 / Layout 1525，21 个子维度，并按 4 个代表模型的表现分成 L1–L5 难度 【W: arxiv.org/html/2510.13626 App. C】。
- 评测协议：只在标准 LIBERO 上训练，零样本迁移 【R: LIBERO–plus/README; P Tab.1 caption p.8】。StarVLA 脚本对四个 suite 各起一个进程，num_trials_per_task=1（每个任务实例即一次 rollout），共用 9883 端口 【R: LIBERO–plus/eval_files/eval_libero.sh】；README 提醒 10,030 任务全跑"extremely long"，提供 parallel_eval/ 集群脚本 【R: LIBERO–plus/README】。

- 代表性数字：StarVLA Qwen3-VL-PI Total 77.0 (Camera 64.3 / Robot 57.2)、Qwen3-VL-OFT 75.0、Qwen2.5-VL-OFT 67.2、Qwen2.5-VL-GR00T 66.4、Qwen2.5-VL-FAST 48.9；对照 OpenVLA-OFT 69.6、 $\pi 0$ 53.6、 $\pi 0$ -FAST 61.6、ABot-M0 80.5 【R: LIBERO-plus/README 表】。VLAct (OFT 头) Total 82.6, Camera 73.9 / Robot 68.4 / Lang 81.5 / Light 96.7 / Bg 96.7 / Noise 86.0 / Layout 83.3, 比同骨干 Qwen3VL-OFT 高 7.6, 比 ABot-M0 高 2.1 【P Tab.1 p.8】。VLAct 加 20K RealOmin UMI 轨迹预训练后 83.7 【P Tab.11 p.28】。
- 入口：LIBERO-plus/eval_files/run_policy_server.sh + eval_libero.sh (MUJOCO_GL=osmesa), aggregate_results.py 汇总。训练无独立 train_files, 直接复用 LIBERO 的 checkpoint 【R: LIBERO-plus/README】。
- 已知问题：Camera 与 Robot 初始状态是所有模型最弱的两维 (OpenVLA-OFT 从 97.1 掉到 59.7 / 37.2) 【W: 2510.13626 Tab.1】；Language 维度分数高并不代表语言理解好，而是模型忽略语言；VLAct 在 Language 维 (81.5) 反而低于 Qwen3VL-OFT (87.0) 【P Tab.1】。

2.3 RoboTwin 2.0

- 定位：双臂 AgileX 操作 + 强域随机化的大规模仿真基准与数据生成器；VLAct 以它作为双臂主基准 【P §4.2 p.9】。
- 任务构成：50 个双臂协作任务，5 种本体，100K+ 预采轨迹，RoboTwin-OD 资产库 731 物体 / 147 类；域随机化五轴：clutter、lighting、background、tabletop height、language 【W: robotwin-platform.github.io; arxiv.org/abs/2506.18088】。动作 14 维关节 【W: huggingface.co/docs/lerobot/main/en/robotwin】。
- 评测协议：官方榜单固定训练数据为 50 demo_clean x 50 任务 (2,500 条, Aloha-AgileX)，每任务 100 trials，分别在 demo_clean (Easy/c2c) 与 demo_randomized (Hard/c2r) 下评测，默认按 (c2c+c2r)/2 排名，上榜要求公开代码 + 权重 + 技术报告 【W: robotwin-platform.github.io/leaderboard】。StarVLA 侧 deploy_policy.yml 设 instruction_type: unseen、action_mode: abs、normalization_mode: min_max 【R: Robotwin/eval_files/deploy_policy.yml】；需给第三方 RoboTwin 的 script/eval_policy.py 打 --policy_ckpt_path 补丁 【R: Robotwin/README】。
- 代表性数字 (Base, 50 clean/任务)：StarVLA-OFT Easy 50.38 (表中 open_microwave、put_bottles_dustbin 两项为 "--", 实为 48 任务均值)；论文基线 RDT 34.50/13.72、Pi0 46.42/16.34、DP3 55.24/4.96 【R: Robotwin/README 第二张表】。VLAct Base: OFT 80.5 clean / 41.5 random, GR00T 76.0/22.9, PI 77.0/23.7, 同库 Qwen3VL-OFT 61.7/10.5; $\pi 0$ 46.4/16.4, X-VLA 70.0/39.0 【P Tab.2 p.9】。
- 代表性数字 (Data Scaling, 50 clean + 500 randomized/任务)：StarVLA-OFT 88.18/88.32 (Qwen3-VL-OFT-RoboTwin2-All); Motus 88.66/87.02、lingbot-vla w/ depth 88.56/86.68、 $\pi 0.5$ 82.74/76.76、X-VLA 72.80/72.84 【R: Robotwin/README 第一张表】。VLAct-OFT 92.5/90.8, VLAct-PI 93.0/88.8; HoloBrain-0-QW 91.9/92.3, Fast-WAM 91.9/91.8, Being-H0.7 90.2/89.6, InternVLA-A1 89.4/89.6, Lingbot-VLA 88.6/86.7 【P Tab.2】。
- 入口：Robotwin/eval_files/start_eval.sh -m demo_clean|demo_randomized -n <name> -c <ckpt> all, 自动多 GPU 调度、流式打印 [RESULT]；底层 eval.sh 与 run_policy_server.sh；训练 run_robotwin_train.sh (Qwen3-VL-4B、robotwin_all_50、150K steps、4/GPU) 与 starvla_cotrain_robotwin_abs.yaml (action_dim 14、action_horizon 50) 【R】。
- 已知问题：(1) Base 基线不一致——StarVLA README 的 OFT Easy 50.38 与 VLAct 论文同名基线 61.7 相差 11 点，说明 checkpoint/步数不同；(2) Data Scaling 的 500 条 randomized 示教就是按 demo_randomized 分布采的，“Random” 评测不再是分布外，StarVLA-OFT Easy/Hard 差距只有 0.14 【R: Robotwin/README】；(3) 论文明确写 “Training compute, optimization schedules, and checkpoint-selection procedures may nevertheless differ across methods” 【P §4.2 p.9】；(4) VLAct 预训练用了 InternData-A1 与 RoboCoin 的 AgileX 数据 【P App. H p.28】，评测本体不是未见本体。

2.4 DOMINO

- 定位：动态操作基准——目标在动、场景随时间变，专门考察单帧 VLA 缺失的时空推理 【W: arxiv.org/abs/2603.15620】。
- 任务构成：35 个动态任务，两类 (dynamic interception / dynamic tracking)，5 种本体，117K 专家轨迹，clean 与 randomized 两套；运动复杂度 Level 1 (匀速) / Level 2 (多项式曲线) / Level 3 (分段随机突变)；动力学系数 α 表示目标最大速度 (m/s)， $\alpha=0$ 即静态，记作 DOMINO@ α 【W: 2603.15620 §2.3】。基于 RoboTwin 2.0 + SAPIEN，静态对照任务直接取 RoboTwin 2.0 同名任务 【W: 2603.15620 App. A.1】。
- 评测协议：每任务 100 episodes；目标移出相机视野即判失败；抓取后目标停止自主运动；lift 类任务要求高度超阈值 【W: 2603.15620 附录 Evaluation Details】。指标 SR 与 MS = Route Completion x 惩罚系数 (目标出界/出视野 x0.5, 碰撞杂物 x0.8)，成功 episode RC=100 【W; R: DOMINO/README "Metrics"]。
- 代表性数字 (35 任务单策略，clean dynamic, $\alpha=0.1$)：StarVLA-OFT 10.86 SR / 30.49 MS, StarVLA-GR00T 6.10/28.60, StarVLA-FAST 5.74/20.66, StarVLA-Adapter 4.40/24.31; PUMA 17.20/34.97, $\pi 0.5$ 9.63/26.17, OpenVLA-OFT 9.06/24.06, $\pi 0$ 8.17/23.96 【R: DOMINO/README 表】。VLAct-OFT 18.50 SR / 34.20 MS, 比 Qwen3VL-OFT 高 7.64 SR / 3.71 MS 【P Tab.5 p.21】。
- 入口：DOMINO/eval_files/start_eval.sh -m demo_clean_dynamic|demo_random_dynamic -n <run> -c <ckpt> all; deploy_policy.yml 中 history_k (默认 0)、history_mode: flow|frames|none 控制历史帧输入，history_flow_utils.py 算光流 【R】；训练 run_domino_train.sh (data_mix 可选 domino_clean / domino_random / domino / domino_cotrain, 150K steps) 【R: DOMINO/README】。
- 已知问题：所有 VLA 的 SR 都在 20% 以下；StarVLA 桥接暴露历史接口但默认关闭，报告的 StarVLA/VLAct 数字都是单帧策略；PUMA 的优势来自历史光流 + world queries 【W】，与骨干无关，属于架构变量。

2.5 VLA-Arena

- 定位：PKU-Alignment 的能力边界基准，用 L0→L2 分级 + 安全代价把“过拟合 L0”暴露出来 【W: arxiv.org/abs/2512.22539】。
- 任务构成：170 任务、11 suites、4 域：Safety 5 suites (static_obstacles、cautious_grasp、hazard_avoidance、state_preservation、dynamic_obstacles)、Distractor 2 suites 30 任务、Extrapolation 3 suites 45 任务、Long Horizon 1 suite 20 任务 【W: github.com/PKU-

Alignment/VLA-Arena; R: VLA-Arena/eval_files/eval_vla_arena.sh TASK_SUITES】。每 suite 三个难度 L0/L1/L2; 正交的 W0-W4 语言扰动与 V0-V4 视觉扰动; 任务用 CBDDL 定义【W】。

- 评测协议: 只在 L0 微调, 评 L1/L2 泛化; 官方数据集 VLA-Arena-L0 S/M/L = 10/30/50 轨迹/任务, 官方结果为 30 episodes (10 × 3 seeds)【W: 2512.22539 §Evaluation】。StarVLA 每 level 5 任务 × 10 episodes = 50 trials【R: VLA-Arena/README】; 脚本默认 NUM_TRIALS=10、SEED=7、init_state_offset=true, 视觉扰动与 apply_safety_constraint 默认关闭【R: eval_vla_arena.sh L28-47】。Safety 域同时输出 SR 与 Cumulative Cost (CC)。
- 代表性数字: StarVLA 的 6 个模型按 suite × level 列在 eval_results.png, 例如 Qwen3-VL-OFT StaticObstacles SR L0 0.84 / L1 0.14 / L2 0.08, LongHorizon L0 0.98 / L1 0.03 / L2 0.0; Qwen2.5-VL-GR00T CautiousGrasp L1 的 CC 高达 116.1【R: VLA-Arena/eval_results.png】。VLAct (L0/L1/L2 平均、按 11 suite 官方加权) Safety 63.2 / Distractor 64.0 / Extrap. 36.9 / Long-H 50.0 / Avg 54.8; 同骨干 Qwen3-VL-OFT 33.4、Qwen3-VL- π 34.1; π 0.5 44.3、 π 0 42.3、OpenVLA-OFT 39.9、UniVLA 38.7、GR00T-N1.6 27.8、SmoVLA 16.3【P Tab.4 p.21】。
- 入口: VLA-Arena/eval_files/run_parallel_eval.sh -c <ckpt> --vla-arena-env <uv env> (11 suite 自动分 GPU); 单 suite 用 uv run --project ... eval_vla_arena.sh --suites ... --levels "0 1 2"; 训练 run_vla_arena_train.sh (默认 Qwen2.5-VL-3B、vla_arena_L0_L、80K steps); data_preparation.sh 拉 VLA-Arena/VLA_Arena_L0_L_1erobot_openpi【R】。
- 已知问题: L1/L2 普遍崩塌 (论文称 "memorization over generalization")【W】; VLAct 的 54.8 是 L0/L1/L2 三级平均, L0 分数拉高总分; StarVLA 默认 10 episodes/任务、单 seed, 与官方 30 episodes 不一致; README 的结果只有图片, 无可解析表格。

2.6 RoboDojo

- 定位: sim + real 统一基准, 官方云端评测、隐藏布局验证, 是目前唯一有"第三方跑分"性质的 VLA 榜单【W: arxiv.org/abs/2607.04434】。
- 任务构成: 42 个 ARX X5 双臂仿真任务 (两臂基座相距 0.6 m, Isaac Sim / Isaac Lab, 基于 MagicSim), 分 Generalization 12 / Memory 6 / Long-Horizon 8 / Precision 8 / Open 8; 18 个真机任务覆盖 ARX X5、Piper、Piper X【W: 2607.04434 §3, §4】。训练集 3,500 条 / 35 任务 (1,859,602 帧, 25 Hz, 20.66 h), Open 维度无训练数据, 另有 100 条域随机化 DLC 轨迹【W: 2607.04434 Tab.10】。
- 评测协议: 每任务 50 episodes → 2,100; Generalization 拆 25 standard + 25 random; 总分是 5 个维度的均值而非 42 任务均值; 报 score (部分进度) 和 success rate; 官方 verified 榜单要求 3 个随机种子、通过隐藏布局验证、通过 XPolicyLab 公开 checkpoint 与训练/部署代码【W: 2607.04434 §5, App.】。真机 18 任务 × 10 trials, 三名评审双盲打分【W】。
- 代表性数字: StarVLA 发布的三个 Qwen3-VL-4B recipe (SR / score): QwenPL_v3 6.19 / 9.60 (Gen 4.17/7.28、Precision 14.00/19.06、Long-H 10.00/17.84、Memory 2.00/2.32、Open 0.75/0.88), QwenOFT 4.86/8.01, QwenGR00T 3.81/7.35【R: RoboDojo/README】。官方榜单 2026-08-24 快照 (score / SR): DM0.5 24.90/19.34, GalaxeaVLA G0.5 20.23/14.88, Xiaomi-Robotics-1 20.07/13.93, Hy-Embodied-0.5-VLA 13.07/8.80, Spatial Forcing 12.38/8.04, π 0.5 11.41/6.91, InternVLA-A1.5 11.15/7.14, VLAct 10.66/7.60 (score 第 8、SR 第 6, 共 35 策略), X-VLA 10.13/6.52, X-WAM 7.69/3.83, StarVLA- α 6.40/3.24, LingBot-VLA 5.50/2.96, ABot-M0 3.67/1.73【P Tab.3 p.11】。VLAct 各维 (score/SR): Gen-Std 16.33/12.00, Gen-Rand 2.74/1.00, Precision 20.62/15.25, Long-H 20.12/13.67, Memory 0.66/0.56, Open 2.37/2.25【P Tab.3】。
- 入口: RoboDojo/eval_files/start_eval.sh <oft|groot|pi_v3> <task> <seed> <policy_gpu> <sim_gpu> <starvla_env> <robodojo_env> [episodes|native], 全部委托给 \$ROBODOJO_PATH/XPolicyLab/policy/starVLA/scripts/eval_hf_robodojo.sh (分支 fix/starvla-hf-robodojo-eval)【R: RoboDojo/eval_files/start_eval.sh】; 训练 run_robodojo_train.sh + 三个 yaml (h50、q99; OFT 100K / GR00T 130K / PI 100K steps 发布); 观测 head + 双臂 224×224, 14D 绝对关节, 预测 50 步执行 16 步【R: RoboDojo/README】。
- 已知问题: Memory 维极低——VLAct 0.66 分 / 0.56%, StarVLA 三个 recipe 1.67-3.33%, 榜首 DM0.5 为 47.74/47.44, 其余多数低于 15%【P Tab.3; W: 2607.04434 Finding 5】; Open 维几乎全零; 官方明确"leaderboard does not normalize for training compute"【P p.11】; StarVLA README 指出高分对手都从机器人策略 checkpoint 初始化、部分加历史帧 (Hy-Embodied 6 帧 × 20 步间隔) 或 VGGT 对齐 (Spatial Forcing), 这些变量未被隔离【R: RoboDojo/README "Observed gaps"】。

2.7 RoboCasa-GR1 Tabletop

- 定位: NVIDIA 为 GR00T N1 系列发布的人形 (Fourier GR-1) 桌面操作基准, 是 VLAct 的"未见本体迁移"主实验【P §4.4 p.10】。
- 任务构成: 24 个任务, 全部为 pick-and-place: 6 个 "PnP X To 容器 Close" (Bottle→Cabinet、Can→Drawer、Cup→Drawer、Milk→Microwave、Potato→Microwave、Wine→Cabinet) + 18 个 "PnP Novel From {Cuttingboard, Placemat, Plate, Tray} To {Basket, Bowl, Pan, Pot, Plate, ...}" 新物体任务; 本体 GR1ArmsAndWaistFourierHands; GR00T Teleop Sim 数据集每任务 1000 条示教【W: github.com/robocasa/robocasa-gr1-tabletop-tasks; R: Robocasa_tabletop/README 表】。
- 评测协议: 单模型训 24 任务, 每任务 50 rollouts; StarVLA 脚本 max_episode_steps 720、n_action_steps 12、n_envs 1; OFT checkpoint 需 --args.no_send_state (否则 state 被离散化拼进 prompt, 见 issue #355), GR00T checkpoint 保留 state【R: Robocasa_tabletop/README】。
- 代表性数字: StarVLA-OFT-Qwen3 Avg 48.8, StarVLA-GR00T-Qwen3 47.8, StarVLA- π -Qwen3 43.9, StarVLA-FAST-Qwen3 39.0; GR00T-N1.6 47.6【R: Robocasa_tabletop/README 表】。VLAct-OFT 全量数据 54.0, 10% 数据 41.42, 20% 数据 49.5, 50% 数据 51.0; 对照 Qwen3VL-OFT 48.8、GR00T-N1.6 47.6、 π 0.5 37.0【P Fig.5(d) p.12; §4.4 p.10】。GR-1 在 VLAct 预训练中未出现【P §4.4】。
- 入口: 服务端 deployment/model_server/server_policy.py --ckpt_path ... --port 5678 --use_bf16; 仿真端 Robocasa_tabletop/eval_files/simulation_env.py; 批量 batch_eval_args.sh; 训练 run_robocasa.sh (fourier_gr1_unified_1000、100K steps、8/GPU、lr 3e-5), 发布 checkpoint 为 steps_90000【R】。
- 已知问题: 并行环境数会显著改变 SR (Isaac-GR00T issue #260 报告 n_envs=5 时 0.52、n_envs=50 时 0.00)【W: github.com/NVIDIA/Isaac-GR00T/issues/260】; 24 任务全是 PnP, 能力覆盖窄; "数据比例曲线"只有 VLAct 一条, 基线没有同样的 10%/20%/50% 点。

2.8 RoboCasa365

- 定位: RoboCasa 的 365 任务扩展版 (ICLR 2026), 面向移动操作与终身学习 【W: robocasa.ai; arxiv.org/abs/2603.04356】。
- 任务构成: 365 任务 = 65 atomic + 300 composite, 220 个需要移动操作; 2,500 个厨房场景; 本体 Franka Panda + Omron 移动底盘; 预训练数据 300 任务 × 100 人类示教 = 30K, 另有 1,600+ h 合成数据 【W】。
- 评测协议: 官方在 50 个 target 任务 (Atomic-Seen / Composite-Seen / Composite-Unseen) 上每任务 50 rollouts, atomic `max_episode_steps=500`、composite 1000+ 【W: robocasa.ai/docs benchmarking; R: Robocasa_365/README §5】。
- StarVLA 现状: 只有一个 100-step walk-through——下载 OpenDrawer (target/human) 单任务 LeRobot v2.1 数据, Qwen3VL-OFT 训 100 步, 2 个 episode 成功 0/2 【R: Robocasa_365/README §3-5】。状态 16 维 (base_position 3 + base_rotation 4 + eef_pos_rel 3 + eef_rot_rel 4 + gripper 2), 动作 12 维 (eef_pos 3 + eef_rot 3 + gripper 1 + base_motion 4 + control_mode 1), 单相机 `robot0_agentview_left` 256→224 【R】。
- 入口: `Robocasa_365/eval_files/run_eval.sh server|client` (默认 5 rollouts, 需 `N_EPISODES=50` 对齐榜单); 训练 `run_robocasa365.sh + starvla_qwenoft_robocasa365.yaml` (`action_dim 12`、`action_horizon 16`); 数据注册 `train_files/data_registry/data_config.py` 【R】。
- 已知问题: README 写 "atomic ~24 / composite ~341", 与官方 65/300 不一致 【R vs W】; 无任何有效基线数字; VLA 未评。

2.9 SimplerEnv

- 定位: real-to-sim 评测——用仿真复刻 Google Robot 与 WidowX BridgeV2 的真机场景, 验证与真机排名相关 【W: arxiv.org/abs/2405.05941】。
- 任务构成: WidowX 4 任务 (spoon on towel、carrot on plate、stack green on yellow cube、eggplant in basket) + Google Robot 的 pick coke can / move near / open-close drawer / place in drawer 等; 两种协议 Visual Matching (贴真图背景) 与 Variant Aggregation (随机化场景取均值), 论文推荐 Visual Matching 【W: github.com/simpler-env/SimplerEnv】。WidowX 通常 24 trials/任务 【W: arxiv.org/html/2507.05116v5】。
- 评测协议 (StarVLA): `start_simpler_env.sh <ckpt>` 自动跑 WidowX 全部任务; 服务端 `run_policy_server.sh`; `model2simpler_interface.py` 含 `adaptive_ensemble.py` 动作集成 【R: SimplerEnv/README、eval_files】。
- 代表性数字 (WidowX 均值): Qwen3VL-PI_v3-Bridge-RT-1 69.8, Qwen3VL-GR00T-Bridge-RT-1 65.3, Qwen3VL-OFT-Bridge-RT-1 42.7; Qwen2.5 系列 GR00T 63.6 / PI 62.5 / FAST 58.6 / OFT 41.8; 仅用 Bridge 训练的 QWen-GR00T-Bridge 71.4 【R: docs/model_zoo.md】。
- 训练: Bridge (`bridge_orig_learner`) + Fractal (`fractal20220817_data_learner`), `data_mix: bridge_rt_1`, 配置 `starvla_cotrain_oxe.yaml` 【R: SimplerEnv/README】。
- 已知问题: 4 个任务 × 24 trials 方差大; A100 上 Vulkan 缺失报错 【R: SimplerEnv/README】; VLA 未评 (其预训练数据 DROID/MolmoAct 为 Franka, 与 WidowX 无关)。

2.10 BEHAVIOR-1K (2025 Challenge 子集)

- 定位: 房屋级长程家务任务, 导航 + 双臂 + 高层规划, OmniGibson (Isaac Sim) 【W: behavior.stanford.edu/challenge】。
- 任务构成: 2025 Challenge 选 50 个全流程任务, 本体 Galaxea R1 Pro 轮式人形, 10,000 条遥操作示教 (1,200+ h, 每任务 200 条); 单条任务平均 6.6 分钟 【W: behavior.stanford.edu; arxiv.org/abs/2512.06951】。动作 23 维 (base 3 + torso 4 + 左臂 7 + 左夹爪 1 + 右臂 7 + 右夹爪 1) 【R: Behavior/README】。
- 评测协议: 主指标 Q-score = 满足的 BDDL 目标谓词比例 (部分成功计分), 每任务 10 episodes, 另有仿真时间 / 导航距离 / 手部位移等效率指标 【W】。
- 代表性数字: 2025 榜单第一 Robot Learning Collective Q-score 0.2605 (公开验证) / 0.2599 (隐藏测试), full success 0.112/0.124; NVIDIA Comet 0.1830/0.2514; "StarVLA" 条目排第 14, 榜单对该行只显示两列数值 0.0000 / 0.0019 (页面未标注这两列对应公开验证还是隐藏测试) 【W: behavior.stanford.edu/challenge/leaderboard.html】。StarVLA README 无自报数字。
- 入口: `Behavior/start_parallel_eval.sh` (需 `star_vla_python`、`sim_python`、`TASKS_JSONL_PATH`、`BEHAVIOR_ASSET_PATH`); 调试用 `start_server.sh + start_client.sh`; `start_parallel_eval_per_task.sh` 逐任务分配实例防 OOM; 观测 wrapper 用 RGBLowResWrapper (224×224 RGB) 【R: Behavior/README】。
- 已知问题: README 首行 "Under construction"; 不能在无 RT core 的 A100/H100 上跑 (Segfault / 低分辨率, 见 BEHAVIOR-1K issue #1872、#1875) 【R】; 评测极慢; StarVLA 在该榜单接近零分。

2.11 CALVIN

- 定位: 语言条件长程操作的经典基准, 考察连续 5 条指令的技能拼接 【W: calvin.cs.uni-freiburg.de】。
- 任务构成: 34 个子任务, 4 个环境 A/B/C/D (同桌面几何、不同纹理与摆放), Franka Panda (PyBullet), ~24 h 无结构 play 数据, 20K 语言标注 【W: arxiv.org/abs/2112.03227】。
- 评测协议: LH-MTLC——1000 条唯一的 5 步指令链, 每步成功才进入下一步, 每链前机器人复位; 指标为第 1-5 步的累计成功率与 Avg. Len (0-5) 【W】。StarVLA 评的是 D→D split (`task_D_D`), 用 `eval_sequences.json` 固定链 【R: calvin/README】。
- 代表性数字 (D→D, Avg. Len): qwenr00t(qwen2.5-vl-3B-instruct-action) 3.786 (步 1-5: 92.5 / 83.9 / 74.4 / 67.9 / 59.9%), qwenr00t(qwen3-vl-4B-instruct-action) 3.757, qwenpi(qwen2.5-vl-3B) 3.576; StarVLA 自训的 PI0.5* 3.885、PI0* 2.954 【R: calvin/README 表】。
- 入口: `calvin/eval_files/run_policy_server.sh + eval_calvin.sh` (需改 `eval_calvin.py` 中 `dataset_path`、`calvin_config_path`、`eval_sequences_path`); 训练 `run_calvin_train.sh`, 数据需先按 RoboTron-Mani 转 LeRobot, mixture 名 `calvin_task_D_D` 【R】。

calvin/README】。发布 checkpoint `StarVLA-QwenGR00T_..._calvin_D_D` Avg. Len 3.786 【R: docs/model_zoo.md】。

- 已知问题：训练用 LeRobot 格式、评测用原始 CALVIN 格式，两套数据【R】；只报 D→D，没有 ABC→D 的零样本环境泛化；由 UNT 团队贡献，"Other experimental results will be released soon"【R】；官方榜 ABCD→D 上 MoDE 已到 4.39、FLOWER 4.5 左右【W: calvin.cs.uni-freiburg.de】，接近饱和。

2.12 MetaWorld MT50

- 定位：Sawyer 单臂 50 任务的多任务基准（Meta-World, CoRL 2020），在 StarVLA 中主要作为 LA4VLA（Language-Action 预训练）的验证场【R: MetaWorld/README §4】。
- 任务构成：50 任务，按难度分 easy 28 / medium 11 / hard 6 / very_hard 5；动作 4 维（xyz + gripper），单相机 corner2，预处理 ROT180 + center_crop(2/3) + resize 224 【R: MetaWorld/README §2】。
- 评测协议：每任务 10 episodes，max 400 步，seed 4042，共 500 episodes；Overall SR = 四个难度桶 SR 的均值（不是 50 任务均值）【R: MetaWorld/README §3-4】。
- 代表性数字（QwenPL_v3 on Qwen2.5-VL-3B, steps_60000）：`la_finetune`（LA4VLA 预训练骨干）easy 0.846 / medium 0.673 / hard 0.517 / very_hard 0.800 / Overall 0.709；`baseline_finetune` 0.857 / 0.600 / 0.467 / 0.440 / 0.591 【R: MetaWorld/README 表】。
- 入口：`MetaWorld/eval_files/run_policy_server.sh`（CKPT、PORT、GPU_ID）与 `eval_metaworld.sh`（EPISODES_PER_TASK），或直接 `eval_metaworld.py --args.levels easy,medium,hard,very_hard`；训练配置 `starvla_qwenpiv3_metaworld_mt50.yaml` 【R】。
- 已知问题：桶均值让 5 个 very_hard 任务权重等于 28 个 easy 任务，`la_finetune` 的总分优势主要来自 very_hard（0.800 vs 0.440）；10 episodes/任务分辨率为 10%；Sawyer 4D 动作与 VLA 常用的 7D/14D 空间差异大，迁移意义有限。

2.13 VLN-CE

- 定位：视觉语言导航（非操作），Habitat 连续环境；StarVLA 在这里只提供 VLM 文本动作服务与训练脚本【R: VLN-CE/README】。
- 任务构成：R2R-CE 与 RxR-CE，val-unseen split；指标 NE、OS、SR、SPL、nDTW 【W: arxiv.org/abs/2412.04453 NaVILA §III-A】。
- 评测协议：`run_qwenvl_vlm_server.sh <ckpt>` 起 QwenVL 服务（默认端口 6694、`MAX_NEW_TOKENS=128`、`GPU_IDS=all` 每卡一服务），仿真则由独立仓库 StarVLA-VLN-CE-Evaluation 负责【R: VLN-CE/README】。
- 训练：`train_starvln.py`，数据为 NaVILA-Dataset 的 R2R/RxR 转 QwenVL 对话 JSON（`annotation_processing.py`），注册在 `starvla/dataloader/qwenvl_llavajson/qwen_data_config.py` 【R】。示例模型 `Ricky06662/StarVLA-VLNCE-Qwen3VL-4B`。
- 已知问题：README 没有任何评测数字；与操作基准的 WebSocket `{image, lang} → normalized_actions` 协议不同，这里返回文本。

2.14 真机与真机数据流程（examples/realRobots/）

- Franka：单臂 7D `[x,y,z,roll,pitch,yaw,gripper]`（位姿增量 + 夹爪 ±1）或双臂 14D；服务端 `server_policy.py --port 5694 --use_bf16`；客户端 `WebsocketClientPolicy` 发 `{image: List[np.ndarray], lang}`，收 `normalized_actions [B,T,7]`；反归一化 `0.5*(a+1)*(max-min)+min`，夹爪维先按 0.5 阈值二值化；统计文件 `dataset_statistics.json` 的 key 为 `franka`（单臂）或 `new_embodiment`（双臂）；示例 `inference_single_example.py` / `inference_dual_example.py` 【R: realRobots/Franka/README】。
- RoboChallenge table30v2：UR5，取 `leftjoint` 6+1 维状态、发 `leftpos` 8 维末端位姿动作，chunk (8,8)；三步流程 self-test → mock server → 正式提交（第三步标 TODO）【R: realRobots/RoboChallenge_table30v2/eval_files/README】。
- EgoVLA：VILA 骨干（SigLIP-384 + Qwen2-1.5B）双手 48 维动作，GR00T ZMQ 协议端口 5555 【R: realRobots/EgoVLA/README】。
- Realman RM-75：VM4A 的 ACT / Diffusion Policy，8 维（7 关节增量 + 1 夹爪），数据不公开【R: realRobots/Realman/README】。
- Unitree G1 WholeBody：流程脚手架，推荐走 GR00T-WholeBodyControl / SONIC 路线，78D 潜动作（64 运动 token + 双手 7+7）或分组直接控制【R: realRobots/UnitreeG1_WholeBody/README】。

3. 评测协议的公平性

3.1 examples/eval_protocol.md 实际规定了什么

这份文件是接口协议，不是统计协议。它规定的内容只有三点【R: examples/eval_protocol.md】：

- 架构：Sim/Real Controller（灰）↔ `PolicyClient.py` + WebSocket ↔ `PolicyServer` ↔ `Framework.predict_action`（橙），模型端与环境端解耦，各自独立 conda 环境。
- 数据契约：客户端发 `{"image": List[np.ndarray], "lang": str}`（可加 `state`、`episode id` 等辅助 key），服务端返回 `normalized_actions`，客户端取 `[0]` 执行；图像必须以 `np.ndarray` 传输，PIL 在服务端还原。
- `model2{bench}_client.py`（如 `model2libero_interface.py`、`model2robotwin_interface.py`）负责基准特定的对齐：动作集成（`adaptive_ensemble.py`）、delta→绝对关节转换、仿真器怪癖。

它没有规定：每任务 episode 数、seed、checkpoint 选择规则、训练步数、是否多 seed 汇报、如何报告方差。这些散落在各基准 README 与脚本默认值里，并且互不一致（见 3.4）。

3.2 VLA 的“只换骨干”受控对比是怎么做的

- 论文声明：在所有 VLAct-vs-基线对比中，"the VLM backbone weights are the only thing that changes: the action head and its fresh initialization, the downstream data, the optimizer, and the fine-tuning budget are all identical", 7.6–21.4 点的提升归因于骨干【P §1 p.4】。具体做法是：持续预训练阶段用 OFT + PI + GR00T 三头共同监督、冻结视觉编码器与 LLM 下半层、混入 caption 数据、部分统一的 20 维跨本体动作布局；微调阶段丢弃预训练头与 caption 流，重新初始化任务头，全参数解冻【P §3.1 p.5, App. I.2 p.31】。
- 被隔离的成对比较（同 Qwen3-VL-4B、同 OFT 头、同数据、同预算）：LIBERO-Plus 82.6 vs 75.0【P Tab.1】；VLA-Arena 54.8 vs 33.4【P Tab.4】；RoboTwin Base 80.5/41.5 vs 61.7/10.5、Data Scaling 92.5/90.8 vs 88.2/88.3【P Tab.2】；DOMINO 18.50/34.20 vs 10.86/30.49【P Tab.5】；RoboCasa-GR1 54.0 vs 48.8【P Fig.5(d)】；真机 Qwen3VL-4B-OFT 无预训练基线【P §4.3】。这一组内部对比是可信的。
- 未被隔离的比较：与 ABot-M0、LingBot-VLA、 π 0.5、HoloBrain-0、Fast-WAM 等外部系统的比较，数字取自各自论文或"same-protocol re-evaluations"，论文自己写明 "Training compute, optimization schedules, and checkpoint-selection procedures may nevertheless differ across methods"【P Tab.2 caption & §4.2 p.9】；RoboDojo 榜单 "does not normalize for training compute"【P p.11】；LIBERO-Plus 基线数字直接来自 LIBERO-Plus 论文【P §4.1 p.8】。
- 预训练数据与评测本体的重叠：VLAct 预训练用 DROID + MolmoAct (Franka)、InternData-A1 + RoboCoin (AgileX)【P App. H p.28】。因此 LIBERO-Plus/VLA-Arena (Franka) 与 RoboTwin/DOMINO (AgileX) 都是"见过的本体"，只有 GR-1 (RoboCasa) 与 ARX X5 (RoboDojo) 是 held-out【P §4.4 p.10】。论文把两类实验分开陈述，但总觉性口径 ("consistently improves") 没有区分。

3.3 Base vs Data Scaling, clean vs random

- Base = 50 clean demos/任务 (2,500 条)，是 RoboTwin 官方榜单的唯一协议【W: robotwin-platform.github.io/leaderboard】。Data Scaling = 再加 500 条 demo_randomized 专家示教/任务 (25,000 条)【P §4.2 p.9】。这 500 条示教就是在评测用的随机化分布 (背景、杂物、桌高、光照) 下采的，所以 Data Scaling 的 "Random" 列不再衡量分布外泛化：StarVLA-OFT 88.18 vs 88.32【R: Robotwin/README】，VLAct 92.5 vs 90.8【P Tab.2】，差距接近零。
- Base 设定下 clean→random 才是真正的分布偏移：VLAct 80.5→41.5 (掉 39 点)，Qwen3VL-OFT 61.7→10.5， π 0 46.4→16.4，X-VLA 70.0→39.0【P Tab.2】。RoboDojo 的 Gen-Std→Gen-Rand 同理：VLAct 16.33→2.74，Hy-Embodied 21.98→1.57 (相对掉 92.9%)【P Tab.3；W: 2607.04434 Finding 2】。
- 结论：比较两篇论文的 RoboTwin 数字前必须先对齐 Base/Data Scaling；VLAct 的 "92.5% 超过 LingBot-VLA/ABot-M0" 是 Data Scaling 口径，"80.5/41.5 最强" 是 Base 口径，两者不可互换。

3.4 训练预算、步数与 checkpoint 选择不统一 (StarVLA 内部)

基准	StarVLA 训练步数 / 批量	发布 checkpoint	评测 episode 数	seeds	出处
LIBERO	30K steps (README 表)；脚本 80K, yaml 100K	Qwen3-VL-PI-LIBERO-4in1 steps_100000	50/任务	1	【R: LIBERO/README、run_libero_train.sh、starvla_cotrain_libero.yaml】
LIBERO-plus	复用 LIBERO checkpoint	同上	1/任务实例 (10,030)	1	【R: LIBERO-plus/eval_libero.sh】
RoboTwin	150K steps, 4/GPU	Qwen3-VL-OFT-RoboTwin2 / -All	100/任务	-s 0 默认	【R: run_robotwin_train.sh、start_eval.sh】
DOMINO	150K steps, 4/GPU	—	100/任务	1	【R: run_domino_train.sh；W】
VLA-Arena	80K steps, 16/GPU, Qwen2.5-VL-3B 默认	—	10/任务 (官方 30)	seed 7	【R: run_vla_arena_train.sh、eval_vla_arena.sh】
RoboDojo	OFT 100K / GR00T 130K / PI 100K, 16/GPU	三个 HF checkpoint	50/任务	1 (官方要求 3)	【R: RoboDojo/README】
RoboCasa-GR1	100K steps, 8/GPU, lr 3e-5	steps_90000	50/任务	1	【R: run_robocasa.sh、README】
MetaWorld	—	steps_60000	10/任务	4042	【R: MetaWorld/README】
VLAct 真机	50k steps, 8xH800	—	10/任务	固定 10 个初始配置	【P §4.3 p.10、App. J.2 p.32】

三点观察：(1) 同一个 RoboDojo 表里三个头的发布步数不同 (100K / 130K / 100K)，checkpoint 显然是按验证表现挑的；(2) LIBERO README 把 30K steps 的 StarVLA 与 175K steps 的 OpenVLA-OFT 并排；(3) 除 VLAct 真机固定初始状态外，所有仿真结果都是单 seed、无方差，而 RoboDojo 官方与 VLA-Arena 官方都要求 3 seeds。

4. 真机评测

4.1 VLAct 的 Franka Research 3 设置【P §4.3 p.10；App. J p.32–34】

- 硬件：桌面固定 Franka Research 3 7-DoF (单臂任务 1 台，双臂任务 2 台)；外部 Intel RealSense D435 一台做第三人称视角，每臂一台腕部 D405；所有图像 resize 到 224x224【P J.1 p.32】。
- 数据：GELLO 遥操作采集，采集时随机化物体初始位置与机械臂初始构型；11 个任务套件；单臂任务 50 条示教/任务，双臂 100 条/任务【P J.2 p.32；Fig.10 p.32】。

- 训练：单臂任务一个模型、双臂任务一个模型，各 50k steps、8×H800；VLAct 与 Qwen3VL-4B-OFT 基线用完全相同的示教、动作头、优化器与预算【P §4.3 p.10；J.2】。
- 评测：每任务 10 次 rollout，两个模型共用同一组 10 个固定的机器人/物体初始配置；单臂短程与双臂任务按 0/1 计成功；单臂长程按完成步数计分，例如 table cleaning 三个独立步骤各 0.33【P J.2~J.3 p.32~33】。
- 任务与结果（VLAct vs 基线）【P §4.3 p.10、J.3 p.33、Fig.5 p.12】：
 - 单臂短程 ID：carrot-from-pot 100 vs 100、button pressing 100 vs 90、cube stacking 90 vs 60、pen-in-cup 80 vs 60，平均 92.5% vs 77.5%。
 - 短程 OOD：novel object from pot (egg/pepper/garlic) 90.0 vs 73.3；novel object in cup (cube/egg) 90.0 vs 65.0。
 - 单臂长程：table cleaning 加权 86.6 vs 73.3；scoop beans 80.0 vs 33.3（基线常跳过舀豆直接倒空勺）。
 - 长程 OOD：extended（加玩具鸡）82.5 vs 47.5；full substitution（cube、toy chicken、pepper 全换）83.3 vs 46.6。
 - 双臂协调：unplugging 80 vs 60、breakfast preparation 90 vs 70、banana handover-place 70 vs 30（论文正文未单列，由 72.0/44.0 的双臂均值与其余四项反推）、fold pants 70 vs 40、fold towel 50 vs 20，平均 72.0% vs 44.0%。VLAct 预训练只有单臂 Franka 与 AgileX 数据，双臂 Franka 是新组合【P §4.3】。
- 局限：10 rollouts/任务的分辨率是 10 个百分点；只与一个自家基线比；没有第三方复现；"weighted score" 的单臂长程结果与 0/1 成功率混在同一张图里【P Fig.5】。

4.2 StarVLA examples/realRobots 的部署流程【R: realRobots/Franka/README】

1. 数据：真机数据转 LeRobot 2.1 (franka2lerobot/README.md)，写 meta/modality.json，在 train_files/data_registry/ 或 starVLA/dataloader/gr00t_lerobot/data_config.py 注册，mixture 在 mixtures.py；python starVLA/dataloader/lerobot_datasets.py --config_yaml ... 验证 dataloader。
2. 模型：yaml 里设 action_dim 7（单臂）/ 14（双臂）、state_dim；python starVLA/model/framework/VLM4A/QwenOFT.py --config_yaml ... 做前向冒烟测试；训练脚本 run_franka_train_single.sh / run_franka_train_dual.sh。
3. 服务：bash examples/realRobots/Franka/eval_files/run_policy_server.sh (server_policy.py --ckpt_path ... --port 5694 --use_bf16)。
4. 客户端：WebsocketClientPolicy(host, port)；采多视角 (H,W,3) uint8 RGB（视角数须与训练一致，例如 wrist + base 两张）；predict_action({"examples":[{"image": images, "lang": prompt}]}); 取 result["data"]["normalized_actions"][0] 得 [T, action_dim] chunk；用 dataset_statistics.json 反归一化 (clip 到 [-1,1]、夹爪按 0.5 二值化为 ±1、线性映射 min-max)；逐步 env.step(action)，env.step 内部同时处理位姿增量与夹爪 (阈值 ±0.9)。
5. 换机器人只需实现：图像采集、env.step、env.reset、匹配的 dataset_statistics.json；Policy Server、WebSocket、反归一化、请求格式不动。
6. 与 VLAct 真机的对应：VLAct 的相机（外部 + 腕部）、224×224、GELLO 示教、7D/14D 动作与这份 README 的约定一致；VLAct 论文没有给出评测脚本，examples/realRobots/Franka 里也没有 rollout 计分或固定初始配置的工具，这一步仍是人工流程。

5. 基准选择建议：面向"VLA 持续预训练 / 表示学习"的研究

5.1 推荐组合（按优先级）

1. **LIBERO-plus（零样本鲁棒性，主指标）**。理由：训练集固定为标准 LIBERO（4 suites × 10 任务 × 50 demos），任何提升只能来自骨干带来的迁移；10,030 个实例使单次评测的统计噪声远小于 50 episodes/任务 的 LIBERO；7 个维度能区分"视觉-空间表示"（Camera、Robot、Layout）与"外观不变性"（Light、Background、Noise）。VLAct 最大提升正好落在 Camera / Robot / Noise / Layout【P §4.1 p.8】。成本：单臂 MuJoCo，可并行；StarVLA 有现成脚本【R: LIBERO-plus/eval_files】。
2. **RoboTwin 2.0 Base 设定（少样本 + clean→random）**。理由：50 clean/任务 的低数据量让预训练表示的价值放大（VLAct 61.7→80.5），random 列直接量化分布外泛化（10.5→41.5）【P Tab.2】；有官方榜单与 100 episodes/任务 的标准协议【W】。必须同时报 clean 与 random，并注明 Base；Data Scaling 只作为"更多下游数据是否仍受益"的补充曲线。
3. **RoboCasa-GR1 数据比例曲线（未见本体的样本效率）**。理由：GR-1 人形与 Franka/AgileX 形态、动作空间都不同，是检验"表示是否跨本体可迁移"的最干净设定；10%/20%/50%/100% 曲线比单点更能说明样本效率【P Fig.5(d)】。注意基线也要跑同样的比例点（VLAct 论文没有），并固定 n_envs【W: Isaac-GR00T issue #260】。
4. **VLA-Arena (L0→L1/L2 外推 + 安全代价)**。理由：只在 L0 微调、评 L1/L2 的设计与"预训练表示能否支撑结构外推"的问题一致；Long-Horizon 与 Safety 是 VLAct 相对 π0.5 提升最大的两维（+21.0、+11.7）【P App. B.1 p.20】；CC 提供 SR 之外的第二个轴。缺点：StarVLA 默认 10 episodes/任务、单 seed，需改为官方 30【W】。
5. **RoboDojo（第三方裁判 + 能力维度诊断）**。理由：唯一带隐藏布局验证、官方云端评测、五维拆分的榜单【W: 2607.04434】；Memory / Precision / Open 三维给出其他基准没有的诊断信号。缺点：绝对分数低（榜首 SR 19.34%，VLAct 7.60%【P Tab.3】），单 seed 差异可能淹没在噪声里；官方要求 3 seeds；Isaac Sim 评测慢。适合作为最终报告而非日常迭代。

5.2 不建议作为主基准的

- LIBERO 标准版：StarVLA 各头 95~97【R: LIBERO/README】，头部差距在 1 点以内，已无区分度；且 LIBERO-plus 证明高分不等于语言理解【W: 2510.13626】。可保留为 LIBERO-plus 的训练集与 sanity check。

- SimplerEnv WidowX: 4 任务 × 24 trials, 单次评测方差大; StarVLA 不同头 41.8–71.4 的跨度【R: docs/model_zoo.md】更多反映动作头与集成策略而非骨干。
- MetaWorld MT50: Sawyer 4D 动作、10 episodes/任务、桶均值指标; 对 VLA 骨干研究的信号弱。
- CALVIN D→D: Avg. Len 3.5–3.9【R: calvin/README】, 官方 ABCD→D 已到 4.4 左右【W】; 若用它, 应改评 ABC→D 零样本环境泛化。
- BEHAVIOR-1K: 评测成本极高、需要 RT core GPU、StarVLA 目前接近零分【W: 榜单第 14, 显示值最高 0.0019】; 不适合作迭代基准。

5.3 报告协议建议 (针对持续预训练论文)

- 固定下游预算: 同步数、同 batch、同 lr、同 checkpoint 选择规则 (例如固定最后一个 checkpoint, 或在独立验证任务上选), 并写进表格 caption; VLA 做到了"同预算", 但没有说明 checkpoint 规则。
- 至少 3 seeds 报均值 ± 标准差 (RoboDojo、VLA-Arena 官方均如此); StarVLA 现有脚本全部单 seed。
- 区分"见过的本体"与"held-out 本体"两组结论: 预训练混合数据里出现过的本体 (VLA 的 Franka、AgileX) 不能用于宣称跨本体迁移。
- RoboTwin 同时给 Base 与 Data Scaling, clean 与 random 四个数, 不用单一 "92.5%" 概括。
- 补充表示层诊断: VLA 的 Fig.2 / Tab.8 / Tab.9 (头间迁移、decoder lock-in)【P p.4, p.26】是比 SR 更贴近"表示质量"的指标, 值得作为标准附表。

5.4 目前没有好基准的维度

- **Memory (非马尔可夫决策)**: 只有 RoboDojo 的 6 个 Memory 任务, 除 DM0.5 (47.74) 外几乎全部低于 15%, VLA 0.56% SR【P Tab.3】; 没有基准能控制"需要记住多久之前的信息"这一变量, 也没有为 history-length 消融设计的协议。RoboDojo 论文引用的 RMBench 专注该维度, 但 StarVLA 未接入【W: 2607.04434 §3】。**补注 (2026-09)**: StarVLA 生态内现已有 [RoboTwin-MeM](#) (EventVLA 提出, 建在 RoboTwin 2.0 上的 8 个双臂任务, 每个任务以"必须记住的关键帧数 n=1–5"参数化, 每回合 430–1544 步) 与其复现的 RMBench; 在 RoboTwin-MeM 上无记忆的 QwenOFT 为 3.8%, EventVLA 为 75.2%, 是目前唯一能把"要记多少"当自变量的评测。
- **Long-horizon**: CALVIN 链已接近饱和; VLA-Arena long_horizon 在 L1 即崩到 0.03【R: eval_results.png】; RoboDojo Long-Horizon 榜首 SR 也只有 32.25%【P Tab.3】; BEHAVIOR 的 Q-score 上限 0.26 且评测极慢【W】。缺一个"中等长度、可快速评测、有分步计分"的基准; VLA 真机的按步计分【P J.3】是这种协议的雏形, 但只有 2 个任务。
- **跨本体零样本**: 所有现有基准都要求在目标本体上微调 (RoboCasa-GR1、RoboDojo 均如此); RoboTwin 2.0 与 DOMINO 各有 5 种本体的数据【W】, 但榜单固定 AgileX, 没有"训 A 本体、测 B 本体"的官方 split。
- **动态场景**: 只有 DOMINO; 且 StarVLA 与 VLA 报告的都是不带历史的单帧策略【R: DOMINO/deploy_policy.yml history_k: 0】。
- **真机可复现性**: RoboDojo-RealEval (18 任务、三评审双盲、10 trials) 与 RoboChallenge 是仅有的第三方真机评测; StarVLA 的 RoboChallenge 接入尚在 mock 阶段【R: RoboChallenge_table30v2/eval_files/README】。

6. 附：基准 → 考察能力 映射

能力	主要基准 (维度)	次要基准	备注
视觉扰动不变性 (光照 / 背景 / 噪声)	LIBERO-plus (Light / Background / Noise); RoboTwin 2.0 random (clutter、lighting、background、桌高)	RoboDojo Gen-Rand; VLA-Arena V0-V4	LIBERO-plus 上模型对 Light/Background 相对稳健, 对 Noise 差异大【W: 2510.13626 Finding 2】
视角 / 本体初始状态鲁棒性	LIBERO-plus (Camera / Robot)	—	所有模型最弱的两维【W】
布局 / 干扰物	LIBERO-plus Layout; VLA-Arena Distractor; RoboDojo Gen (最多 25 个杂物)【W: 2607.04434 §3】	RoboTwin random clutter	—
语言接地	VLA-Arena (W0-W4; 去指令后掉 52–64%)【W: 2512.22539 §Comparison with LIBERO】; RoboDojo Open	LIBERO-plus Language	LIBERO 去指令仅掉 28%【W】, 不能测语言
少样本适应	RoboTwin Base (50 demos/任务); RoboCasa-GR1 数据比例	VLA-Arena S/M/L 数据集	—
未见本体迁移 (需微调)	RoboCasa-GR1 (GR-1); RoboDojo (ARX X5)	—	无零样本本体迁移基准
双臂协调	RoboTwin 2.0; RoboDojo; DOMINO 双臂任务	VLA 真机双臂 5 任务【P §4.3】	—
精细操作	RoboDojo Precision (8 任务)	RoboTwin hanging_mug、click_bell 等低分任务【R: Robotwin/README】	—
长程 / 技能拼接	CALVIN 5 步链; RoboDojo Long-Horizon; VLA-Arena long_horizon; BEHAVIOR 50 任务	VLA 真机 table cleaning / scoop beans (按步计分)	见 5.4
记忆 / 部分可观测	RoboDojo Memory (6 任务)	—	见 5.4
动态目标 / 时序推理	DOMINO (SR + MS, Level 1–3, α)	VLA-Arena dynamic_obstacles / dynamic_distractors	—

能力	主要基准（维度）	次要基准	备注
安全约束	VLA-Arena Safety（SR + CC）	DOMINO MS 的碰撞 / 出界惩罚	—
移动操作 / 房屋级	RoboCasa365；BEHAVIOR-1K	VLN-CE（纯导航）	StarVLA 均无有效基线
真机部署	VLAct Franka（10 rollouts）；RoboDojo-RealEval；RoboChallenge	—	—

```
flowchart LR
    subgraph 单臂Franka
        LIB[LIBERO] --> SAT[已饱和 · sanity check]
        LIBP[LIBERO-plus] --> VIS[视觉扰动不变性]
        LIBP --> CAM[视角/初始状态鲁棒性]
        ARENA[VLA-Arena] --> LANG[语言接地/外推]
        ARENA --> SAFE[安全约束 CC]
        ARENA --> LH[长程拼接]
    end
    end
    subgraph 双臂AgileX
        RT[RoboTwin 2.0 Base] --> FEW[少样本适应]
        RT --> C2R[clean→random 泛化]
        DOM[DOMINO] --> DYN[动态目标/时序推理]
    end
    end
    subgraph 未见本体
        GR1[RoboCasa-GR1] --> XEMB[跨本体迁移-需微调]
        DOJ0[RoboDojo ARX X5] --> XEMB
        DOJ0 --> MEM[记忆]
        DOJ0 --> PREC[精细操作]
        DOJ0 --> OPEN[开放指令]
    end
    end
    subgraph 其他
        SIM[SimplerEnv] --> R2S[real-to-sim 相关性]
        CAL[CALVIN] --> LH
        BEH[BEHAVIOR-1K] --> HOUSE[房屋级长程]
        RC365[RoboCasa365] --> HOUSE
        MW[MetaWorld] --> MT[多任务·弱信号]
        VLN[VLN-CE] --> NAV[导航]
    end
    end
    REAL[VLAct 真机 Franka] --> OOD[新物体/序列扩展/双臂]
```

07 · 研究路线图：在 StarVLA / VLAct 之上做什么

这份文档回答一个问题：读完 StarVLA 技术报告、StarVLA- α 、VLAct 三篇论文并看过代码之后，下一步的研究该往哪里走。每个方向给出问题、已有证据、具体做法、在 StarVLA 代码中的落点、评测方案和风险。方向按"表征诊断 → 预训练目标 → 动作空间 → 能力短板 → scaling → 评测方法"六类组织，最后给一个可以直接执行的六个月计划。

0. 三篇论文留下的地图

已被证实	证据	留下的空白
强 VLM + MLP 头 + 最少数据工程就是一个强基线	StarVLA- α 表 1: LIBERO 98.8、GR1 53.8	只在 4B 验证；低数据 + 关节角控制是短板 (RoboTwin Base 50.3 < π 0.5 60.2)
连续头 $\gg$ 离散头；三种连续头在数据充足时相当	StarVLA- α 表 2；VLAct pilot	低数据 + 强扰动时头差距仍大 (VLAct RoboTwin Base/Random: OFT 41.5 vs GR00T 22.9)
朴素动作预训练伤害跨本体迁移	StarVLA- α 表 3: OXE 把 GR1 24x10 从 9.8 打到 1.2	为什么伤害？哪一层的表征被破坏？
保护先验 + 多头 + 部分统一动作空间能把预训练变成净收益	VLAct: GR1 48.8 $\rightarrow$ 54.0, 20% 数据超全量 GR00T-N1.6	每个组件的作用机制仍是经验性的；规模、本体数量、头的种类都没有扫过
单一动作头会让骨干表征坍塌到该头的解码几何 (decoder lock-in)	VLAct 表 8: PI 微调 60.5 $\rightarrow$ 55.1 (OFT-only 预训练)	没有直接的表征度量，只有下游成功率这一间接证据
动作-only 微调让 VLM 在 20K 步内忘掉 grounding；共训能同时提升操作	StarVLA 报告第 6 节 (ST4VLA): Google VM 66.1 $\rightarrow$ 84.6	辅助数据的配比、调度、与动作损失的梯度冲突尚未系统研究
generalist 联训 + 大 batch 优于 specialist	StarVLA- α 表 5、表 11: GR1 53.8 $\rightarrow$ 57.3, batch 64 $\rightarrow$ 1024 使 GR1 40.0 $\rightarrow$ 59.2	generalist $\times$ 持续预训练的组合没有做
记忆是所有 StarVLA 系模型的死角	RoboDojo: VLAct Memory 0.66/0.56, DM0.5 47.74/47.44	单帧输入的架构假设没有被挑战

1. 方向清单

每个方向的编号在后文的优先级矩阵和六个月计划中复用。

A. 表征诊断与度量

A1 · 骨干可复用性诊断套件。 问题：decoder lock-in 目前只能通过"换头微调看成功率"间接观察，一次实验要几十 GPU 小时。做法：在冻结骨干上训练轻量线性/MLP 探针，从 action query 隐状态预测动作，报告不同头几何下的探针误差；用 CKA / 线性回归可解释方差度量预训练前后各层表征变化；把 RefCOCO-g IoU、POPE、MME 做成训练过程中的周期性探针 (ST4VLA 已展示可行)。落点：新增 `starVLA/training/trainer_utils/probes.py`，在 `train_starvla*.py` 的评估钩子里调用；VLM 能力探针可复用 `train_starvlm.py` 的评测数据接口。评测：诊断指标与下游跨头迁移成功率的相关性 (目标：Spearman $\rho > 0.7$)。风险：探针本身可能对头几何敏感，需要多种探针取平均。

A2 · "哪层该冻"的自动化。 问题：VLAct 冻结"视觉编码器 + LLM 下半层"是一个粗粒度的手工选择 (表 6: 三档消融)。做法：用 A1 的逐层漂移度量在训练早期决定每层的学习率衰减系数 (layer-wise LR decay) 或 LoRA rank；对比三种方案：硬冻结、LLRD、上半层 LoRA。落点：`TrainerUtils.freeze_backbones` 已支持正则/名单冻结，`build_param_lr_groups` 已支持按模块分组 lr，把"按层深度生成 lr 组"加进去即可。评测：LIBERO-Plus 的 Camera / Robot / Noise 维度 (VLAct 增益最大处)，以及 A1 的漂移曲线。

B. 预训练目标设计

B1 · 头多样性作为正则化的推广。 问题：VLAct 用 OFT + PI + GR00T 三个连续头。头的"多样性"是否可以更极端、更便宜？做法：(i) 加入 FAST 离散头作为第四个辅助头，检验离散监督注入的"粗粒度结构"是否与连续头互补 (VLAct pilot 显示 FAST 预训练 $\rightarrow$ GR00T 微调略优于从零)；(ii) 加入未来帧预测头 (与 WM4A 汇合，见 B3)；(iii) 加入空间 QA 头 (RoboPoint 式的点预测)。做 2^k 因子实验找出哪些头组合真正有效。落点：`starVLA/model/framework/VLM4A/` 下新建 `QwenMultiHead.py`，在 forward 里把多个 `action_model` 的 loss 相加；每个头的开关和权重走 YAML。评测：VLAct 表 8/9 协议——每个预训练变体接四种下游头微调。风险：头越多，预训练显存越大；需要先测量三头方案的实际开销 (E2)。

B2 · 辅助数据的自动配比与调度。 问题：VLAct 固定 0.5 的 VLM loss 权重、固定 caption 采样比；全混合 (82.5) 略低于 caption-only (82.6)，作者归因为稀释。做法：把 A1 的漂移信号作为控制量：漂移超过阈值就提高 VLM 数据比例，反之降低 (类似 curriculum / bandit 调度)；对比固定比例、线性退火、漂移驱动三种策略。落点：`train_starvla_cotrain.py` 的双 dataloader 循环里加一个调度器，控制每步 VLM batch 的采样概率和 `loss_scale.vlm`。评测：LIBERO-Plus 总分 + RefCOCO-g 保持率，画 Pareto 前沿。

B3 · 统一潜动作 vs 显式多头。 问题：多头共监督是"让表征对多种解码器可读"；另一条路是学一个头无关的潜动作空间 (LAPA、UniVLA)，再让各头从潜空间解码。做法：在同一骨干上比较 (a) VLAct 三头显式监督，(b) 潜动作 VQ 目标 + 单解码头，(c) 两者叠加。落点：`examples/modelExtensions/DiscreteDiffusion` 已有离散动作建模的例子可以借用 codebook 部分。评测：跨头迁移 (VLAct 表 8 协议) + 未见本体迁移 (RoboCasa-GR1 数据效率曲线)。

B4 · 世界模型作为辅助信号。 问题：StarVLA 报告的"广义 VLA 视角"认为 VLM 路线和世界模型路线只差辅助信号。WM4A 已把 Cosmos-Predict2 / Wan2.2 当骨干，但"VLM 骨干 + 未来帧预测辅助头"这一组合没有做。做法：在 Qwen3-VL 骨干上加一个轻量的未来帧/未来特征预测头 (预测 DINO 或

SigLIP 特征而非像素，降低成本)，与动作头共训。落点：`docs/WM4A.md` 描述的接口 + `starVLA/model/framework/WM4A/`；特征预测头可放在 `starVLA/model/modules/`。评测：DOMINO（动态场景，需要预测）和 RoboTwin Random。

C. 动作空间与跨本体

C1 · 部分统一动作空间的可学习化。 问题：VLAct 的 20 维布局手工指定“夹爪共享、手臂分开”，只覆盖 6-DoF 双臂关节角和 6-DoF 单臂 delta EE。人形（GR1）、灵巧手、移动底盘进来后要重画。做法：用一个可学习的“维度对齐矩阵”决定各本体动作维度到共享坐标的映射，以稀疏性正则鼓励物理同义维度共享；或者用本体描述文本（DoF、末端类型、控制模式）作为条件让骨干自己路由。对照组：VLAct 手工布局、StarVLA- α 32 维零填充、每本体独立头。落点：`starVLA/dataloader/lerobot_datasets.py` 的动作拼装与 mask 逻辑；动作头输出维与 mask 在 `action_model` 配置中定义。评测：加入 GR1 数据做三本体预训练，看 RoboCasa-GR1 与 RoboDojo (ARX X5) 迁移。

C2 · 几何一致的动作参数化。 问题：wrap-aware loss 只处理了周期关节角（RoboTwin 75.5→80.5，+5 个点，是 VLAct 单项消融里最大的）。旋转表示（欧拉 / 四元数 / Rot6D）、关节空间与末端空间的混合仍按普通欧氏量回归。做法：系统比较旋转表示 $\times$ 对应的测地线 loss；对 delta EE 的旋转分量用 SO(3) 上的距离；检验 StarVLA- α 表 4 中 delta / relative 动作在低数据段的小幅收益是否被 loss 选择放大。落点：`starVLA/dataloader` 的动作转换 + 每个 head 的 loss 函数。评测：RoboTwin Base（关节角）、LIBERO-Plus Robot 维度（EE 控制）。

C3 · 零样本 / 极少样本本体迁移。 问题：VLAct 在 GR1 上 20% 数据 49.5，但零样本未测；头是重新初始化的。做法：保留预训练的统一头，用本体描述做条件（C1 的路由方案），测 0 / 1% / 5% / 20% 数据曲线；对比“重新初始化头”与“复用统一头”。评测：RoboCasa-GR1、RoboDojo 数据效率曲线。

D. 能力短板

D1 · 记忆与长程。 问题：RoboDojo Memory 维度 0.66 / 0.56，榜首 DM0.5 47.74 / 47.44。StarVLA- α 表 4 显示简单堆两帧历史反而降分。做法：四条路线并行小规模验证：(i) 压缩历史 token（把过去 k 帧的 VLM 特征池化成少量 memory token 拼进上下文）；(ii) 双系统里让 System 2 维护显式状态（语言化的任务进度），System 1 只看当前帧；(iii) 在 GR00T 头里加入过去动作块作为条件；(iv) **稀疏事件记忆**——**EventVLA** 已在 StarVLA-OFT 上验证：初始帧 + 最近 K 帧的规则锚点，加一个与动作头并联的关键帧预测头（KEM），命中就把原图写进有界 FIFO 缓冲；RoboTwin-MeM 上 QwenOFT 3.8 → 75.2。它的消融表明“存原图让 VLM 重看”优于存特征，这应作为 (i) 的对照。落点：`QwenGR00T.py` 的条件输入；`dataloader` 需支持返回历史帧（`umi_datasets.py` / `lerobot_datasets.py` 的 frame 索引逻辑）。评测：**RoboTwin-MeM**（8 个双臂任务，需记忆的关键帧数 $n=1-5$ 可控，见 09 §3）与 RMBench 作为主指标；RoboDojo Memory 与 Long-Horizon 子集作外部核验；VLAct 真机 scoop beans 这类多阶段任务。首个具体实验：用 VLAct 骨干替换 EventVLA 的 Qwen3-VL 初始化，看两者是否叠加。

D2 · 语言泛化。 问题：LIBERO-Plus Language 维度 VLAct 81.5 低于同骨干基线 87.0，是唯一掉分的维度。做法：预训练时对任务名做 LLM 改写增广（同义指令、不同粒度）；把纯文本指令数据（VLAct 已证明有正向作用）与机器人样本做指令级对齐共训。落点：`train_starvla_cotrain.py` 的 VLM 数据流；改写脚本可放 `scripts/`。评测：LIBERO-Plus Language 维度、VLA-Arena Extrapolation。

D3 · 低数据 + 强扰动下的头差距。 问题：RoboTwin Base/Random：OFT 41.5、GR00T 22.9、PI 23.7。生成式头在低数据段明显落后于回归头，与“三头相当”的一般结论矛盾。做法：检验是训练步数不足（去噪头收敛慢）、还是 flow-matching 目标在小数据上过拟合；尝试从 OFT 头蒸馏到 PI/GR00T 头（回归先验作为教师），或减少去噪步数。评测：RoboTwin Base 上按训练步数画曲线。

E. 系统与 scaling

E1 · 持续预训练的规模曲线。 问题：StarVLA- α 说 4B→8B 增益 <1%，但那是无预训练；VLAct 只做了 4B。做法：Qwen3-VL 2B / 4B / 8B（可能加 Qwen3.5 9B） $\times$ {无预训练, 朴素预训练, VLAct 配方}，固定下游协议。评测：LIBERO-Plus、RoboTwin Base、RoboCasa-GR1 20% 数据。资源：这是路线图里最贵的一项，8B 三头预训练需要 32+ GPU。

E2 · 三头共训的开销测量与降本。 问题：VLAct 声称“只增加头的计算”，但 PI 和 GR00T 各带一个 DiT，没有数字。做法：用 StarVLA 报告第 8 节的方法（issue #158 的测量脚本）测三头 vs 单头的 samples/s 和显存；尝试头间共享 DiT 主干、头 dropout（每步随机只激活一个头）。落点：`Makefile` / 测试脚本 + 效率 issue 的复现。

E3 · 固定 GPU 预算下的数据 vs 配方 Pareto。 问题：论文标题说 beyond data scaling，但没有画“数据量 $\times$ 配方”的二维图。做法：预训练数据 25% / 50% / 100% $\times$ {朴素, VLAct}，固定 16 GPU 步数。评测：GR1 20% 数据成功率。

F. 评测方法学

F1 · 骨干基准 (backbone benchmark)。 问题：VLAct 的“只换骨干权重”协议是目前最干净的归因方式，但没有被标准化。做法：在 StarVLA `examples/eval_protocol.md` 之上定义一个“骨干评测协议”：固定下游头、数据、优化器、步数，只替换 `qwen_vl_interface` 权重；提供脚本一键跑 LIBERO-Plus + RoboTwin Base + GR1-20%。落点：`examples/simBenchmarks/*/train_files/` 的 YAML 模板 + 一个新的 `examples/backboneBench/`。

F2 · generalist $\times$ 持续预训练。 问题：StarVLA- α 的 all-in-one generalist 和 VLAct 的持续预训练是两条独立的线。做法：VLAct 骨干 $\rightarrow$ all-in-one 联训四基准，看是否叠加。评测：StarVLA- α 表 5 协议。

F3 · 真机统计功效。 问题：每任务 10 次 rollout，10–20 个点以内的差距不可靠。做法：用已发布的真机数据做 bootstrap，给出在 $n=10 / 20 / 50$ 下能分辨的最小差距；在仓库里提供计算脚本。

2. 优先级矩阵

影响 = 对“持续预训练配方”这一核心问题的推进程度；成本 = GPU 小时与工程量。

方向	影响	成本	建议
A1 诊断套件	高（所有后续方向的度量基础）	低	先做

方向	影响	成本	建议
E2 三头开销	中	低	先做，一周内出数字
F1 骨干基准	高	低	先做，产出可复用脚本
A2 自动冻结	中	低	第二批
B1 头多样性推广	高	中	第二批
D1 记忆	高	中	第二批（RoboDojo 最大短板）
C1 可学习动作空间	高	中	第三批
B2 辅助数据调度	中	中	第三批
D3 低数据头差距	中	低	第三批
B4 世界模型辅助头	高	高	第四批
C2 几何一致参数化	中	低	第四批
E1 规模曲线	高	很高	有资源再做
B3 / C3 / D2 / E3 / F2 / F3	中	低-中	穿插

3. 六个月执行计划

前提：16 张 A100/H800 级 GPU，StarVLA 代码库，VLA_{ct} 数据配方全部开源可下载。

月份	目标	交付物
1	复现 VLA _{ct} 基线；实现 A1 探针与 F1 骨干基准脚本；测 E2 开销	复现报告（LIBERO-Plus 82.6 ± ?）、 <code>probes.py</code> 、 <code>examples/backboneBench/</code> 、开销表
2	A2 自动冻结 + B1 头组合因子实验（4 头 × 2^4 子集中挑 6 个）	漂移曲线 vs 冻结策略图；头组合 → 跨头迁移矩阵
3	D1 记忆三路线小规模验证；D3 低数据头差距诊断	RoboDojo Memory 子集结果；训练步数曲线
4	C1 可学习动作空间，加入 GR1 做三本体预训练	三本体 → GR1 / ARX X5 迁移曲线，对照 VLA _{ct} 手工布局
5	B2 调度器 + B4 特征预测辅助头	Pareto 前沿图；DOMINO / RoboTwin Random 结果
6	整合最优组合，跑完整评测（LIBERO-Plus、VLA-Arena、RoboTwin、DOMINO、GR1、RoboDojo 提交），写论文	一篇"VLA _{ct} 之后"的方法论文 + 向 StarVLA 提 PR

每个月的实验都遵守 F1 协议：只换骨干，下游一切固定。所有数字进 `assets/` 的 CSV，图由脚本生成。

4. 风险与对策

- **复现偏差**。VLA_{ct} 代码与 checkpoint 发布进度未知；若拿不到官方权重，先用 StarVLA- α 配置 + 论文附录 H 的数据清洗规则自行复现，把复现差距作为第一个报告。
- **算力不足**。三头预训练在 16 GPU 上的吞吐是瓶颈（E2 先测）；必要时用头 dropout 或 2B 骨干做方法验证，4B 只跑最终配置。
- **基准饱和与噪声**。LIBERO 已饱和（>98%），用 LIBERO-Plus / VLA-Arena 替代；SimplerEnv 方差大，按 StarVLA 报告的做法跑 5 次取均值；RoboDojo 榜单不归一化算力，只用于相对比较。
- **归因陷阱**。任何新组件都要同时报告"同头"和"跨头"两组数字（VLA_{ct} 表 8/9 协议），避免重蹈"单头成功率高估复用性"的覆辙。

5. 与本仓库其他材料的关系

- 证据来源：[01 · VLA_{ct} 精读](#)、[03 · StarVLA- \$\alpha\$](#) 、[04 · 技术报告](#)
- 每个方向在代码中的落点细节：[02 · 代码库解析](#)，尤其第 9 章"VLA_{ct} 配方对照"
- 评测选择依据：[06 · 基准生态](#)

08 · 讲稿：四种动作头（FAST / OFT / PI / GR00T）到底是什么

用途：一次 60 分钟的组会讲解，面向已经了解 VLM 但没有做过 VLA 的听众。每节开头给“讲述提示”（用什么直觉切入、板书什么），正文是可以直接照读的讲稿，方框内是公式速查。配套幻灯片：[report/action_heads_lecture_slides.pdf](#)。

0. 材料清单

头	原始论文	出处	英文 PDF	中文翻译
FAST	Pertsch et al., <i>FAST: Efficient Action Tokenization for Vision–Language–Action Models</i> , arXiv 2501.09747 , 2025–01	Physical Intelligence	papers/en	papers/zh
OFT	Kim, Finn, Liang, <i>Fine-Tuning Vision–Language–Action Models: Optimizing Speed and Success</i> (OpenVLA–OFT), arXiv 2502.19645 , 2025–02	Stanford	papers/en	papers/zh
PI	Black et al., $\pi 0$: <i>A Vision–Language–Action Flow Model for General Robot Control</i> , arXiv 2410.24164 , 2024–10	Physical Intelligence	arXiv 非独占许可，不随仓库分发	同左；中译版只保存在本机 <code>~/Desktop/research/papers/heads_local/</code>
GR00T	Bjorck et al., <i>GR00T N1: An Open Foundation Model for Generalist Humanoid Robots</i> , arXiv 2503.14734 , 2025–03	NVIDIA	papers/en	papers/zh

背景论文（不翻译，讲稿中会解释所需部分）：Lipman et al., *Flow Matching for Generative Modeling*, [arXiv 2210.02747](#)；OpenVLA, [arXiv 2406.09246](#)。StarVLA 中四个头的实现文件在 §7 列出。

四篇论文中 FAST、OpenVLA–OFT、GR00T N1 为 CC BY 4.0，原文与译文均已入库； $\pi 0$ 为 arXiv 非独占许可，仓库只放链接。

1. 开场（5 分钟）

讲述提示：先不讲任何一个头，先把“问题”钉死。板书一行：VLM 会输出 token，机器人要的是连续、高频、多维的动作序列。四个头就是四种把前者变成后者

的办法。
一个 VLA (Vision–Language–Action) 模型由两部分组成：一个在互联网图文上预训练过的视觉–语言模型 (VLM) 负责“看懂场景、听懂指令”，一个动作头负责把 VLM 的内部表征变成机器人能执行的控制量。VLM 那一半大家都熟，真正让 VLA 与聊天模型不同的是动作头。

动作头要解决的问题有三个特点：

- 1. **连续**。关节角、末端位姿、夹爪开合都是实数，VLM 的原生输出是离散 token。
- 2. **高频且成块**。现代策略一次预测一个“动作块” (action chunk)：未来 H 步的动作一起输出， H 从 8 到 50 不等，控制频率 10–50 Hz。一次前向要产出 $H \times D$ 个实数 (D 是动作维度，单臂 7、双臂 14、人形更多)。
- 3. **多模态分布**。同一场景下可能有多种同样合理的动作（从左边绕还是从右边绕），点估计会取平均，产生两边都不是的动作。

四条路线一句话：

- **FAST**：不改 VLM，把动作块压缩成一串离散 token，让 VLM 像生成文字一样生成动作。
- **OFT**：在 VLM 末尾拼一组占位 token，用一个小 MLP 把它们的隐状态直接回归成连续动作，一次前向出整块。
- **PI ($\pi 0$)**：给 VLM 配一个专门的“动作专家”网络，用 flow matching 从噪声逐步生成动作块，能表达多模态分布。
- **GR00T**：同样用 flow matching，但把它放进一个显式分离的“快系统” (DiT)，VLM 作为“慢系统”只提供条件；状态和本体标识进快系统。

接下来每个头讲四件事：它想解决什么、核心机制、训练与推理怎么做、在 StarVLA 代码里长什么样。

2. 共同记号

记号	含义
$\mathbf{o}_t$	时刻 t 的观测：多视角 RGB 图像 + 语言指令（部分头还含本体感受状态 $\mathbf{s}_t$ ）
$\mathbf{A}_t = (\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H-1})$	动作块， H 为块长，每个 $\mathbf{a} \in \mathbb{R}^D$
$\mathbf{H}_t$	VLM 处理 $\mathbf{o}_t$ 后的隐状态序列（最后一层或多层）
$\mathbf{z}$	骨干交给动作头的潜表征（StarVLA 报告里的写法）
$\tau \in [0, 1]$	flow matching 的“时间”，与机器人时刻 t 无关
$\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$	高斯噪声，形状与 $\mathbf{A}_t$ 相同

3. FAST：把动作当时间序列压缩（12 分钟）

讲述提示：从“为什么最直接的办法会失败”讲起。板书两条正弦曲线，一条采样 5 Hz，一条 50 Hz，问听众：如果模型每次只预测下一个采样点，哪条更好学？

3.1 它要解决的问题

最早的 VLA (RT-2、OpenVLA) 用最朴素的办法把动作变成 token：每个时刻、每个维度单独分成 256 个箱，一个箱一个 token。7 维动作一步就是 7 个 token，一个 50 步的块要 350 个 token。这在低频数据 (Bridge、RT-1，约 5 Hz) 上能用，到高频数据 (DROID 15 Hz 以上) 就学不动了。

FAST 论文用一个玩具实验把原因说清楚：自回归模型的学习信号来自"给定前面所有 token，下一个 token 带来的新信息"。对平滑信号，采样频率越高，相邻两步的差别越小，下一个 token 的边际信息趋近于零；模型最省力的解法是复制上一个 token，于是学出来的策略几乎不动。这不是数据太难，是 token 化方式把信息稀释了。

3.2 核心机制：DCT + BPE

直觉是 JPEG。图像像素在空间上平滑，所以离散余弦变换 (DCT) 后能量集中在少数低频系数，把高频小系数丢掉就是压缩。动作在时间上也平滑，同样的招数可以用在每个动作维度的时间序列上。

FAST 的五步 (论文图 4、算法 1)：

1. **归一化**：用训练集每个维度的 1% 和 99% 分位数把动作映射到 $[-1, 1]$ 。用分位数而不是最值，是为了不被大数据集里偶发的离群动作带偏，也让不同本体、不同尺度的数据集能共用一个 tokenizer。
2. **逐维 DCT**：对块内每个维度的 H 个值做 DCT，得到 $D \times H$ 的系数矩阵。
3. **缩放取整**：系数乘一个尺度因子 γ 后取整。 γ 是唯一的超参数，控制"有损程度 vs 压缩率"。取整后矩阵大部分是零。
4. **展平**：把矩阵拉成一维整数序列，**低频系数在前**、各维度交错。这样序列开头就是动作块的整体形状，后面才是细节。
5. **BPE**：在整数序列上训练一个 byte-pair encoding tokenizer (词表 1024)，把成串的零和常一起出现的系数组合成一个 token。这是整条流水线里唯一需要"训练"的部分，训练只要几分钟。

结果：一个 1 秒的动作块每条手臂大约 30 个 token (论文表 I)，比分箱少一个量级。这些 token 直接覆写 VLM 词表里最少使用的那部分，VLM 的架构和训练目标完全不变——还是 next-token 交叉熵。

$$\mathcal{L}_{\text{FAST}} = - \sum_{m=1}^M \log p_{\theta}(z_m | H_t, z_{<m}), \quad z_{1:M} = \text{Tok}_{\text{FAST}}(A_t)$$

推理时自回归地生成 token，再走逆变换：BPE 解码 $\rightarrow$ 还原系数矩阵 $\rightarrow$ 逆 DCT $\rightarrow$ 反归一化。

FAST+：作者在约 100 万条真实机器人轨迹上训练了一个通用 tokenizer，覆盖单臂、双臂、移动机器人和各种控制频率，可以拿来直接用，不必为每个数据集重训。

3.3 结果与代价

- 在 DROID (15 Hz) 上，用 FAST 训练的 π_0 骨干做到了首个"零样本"桌面操作评测；在 LIBERO、20 Hz 的餐桌清理、T 恤折叠上与 diffusion 版 π_0 相当。
- 训练效率：达到同等性能所需 GPU 小时比 diffusion 版 π_0 少约 5 倍——因为目标就是普通的交叉熵，不需要在每个样本上采样噪声和时间步。
- 代价：**推理慢**。动作要一个 token 一个 token 地生成，几十个 token 的自回归解码比一次前向的回归头或几步去噪的生成头都慢；这也是后面 OFT 要解决的问题。
- 另一个代价：解码可能产生不合法的 token 序列 (BPE 无法还原)，需要兜底处理。

3.4 在 StarVLA 中

starVLA/model/framework/VLM4A/QwenFast.py：动作经 physical-intelligence/fast tokenizer 变成 `<robot_action_k>` 字符串放进 assistant 回合；loss 就是 VLM 的 next-token 交叉熵；推理调用 `generate()`，然后按 Qwen3-VL-Action 词表中预留的 2048 个 action token 的 id 区间抽出 token 解码。它是四个头里唯一不给 VLM 加任何新模块的。

3.5 一句话记住

FAST 把"动作 token 化"从每步每维分箱换成时间序列压缩，用 DCT 去掉冗余、用 BPE 变成稠密 token；它保住了 VLM 的原生接口，付出的是推理速度和一层离散化的信息损失。StarVLA-a 表 2 里 FAST 在 WidowX 上比连续头低 29 个点，VLAct 的先导实验也显示 FAST 预训练能迁移但保留 FAST 头本身表现差——离散瓶颈是结构性的。

4. OFT：并行解码 + 连续回归 (10 分钟)

讲述提示：这一节是"工程上把三件显而易见的事一起做对"。先算一笔账：OpenVLA 一步 7 个 token 串行解码，3–5 Hz；如果要 25 Hz 的双臂控制，差了一个量级。

4.1 它要解决的问题

OpenVLA–OFT 的出发点不是新架构，是问：拿一个现成的自回归 VLA (OpenVLA, 7B)，微调时哪些设计决定了速度和成功率？作者系统比较了三组选择——生成方式 (自回归 vs 并行)、动作表示 (离散 vs 连续)、学习目标 (next-token 交叉熵 vs L1 回归 vs diffusion) ——然后把最优组合打包成 OFT 配方。

4.2 核心机制

并行解码。自回归模型每生成一个 token 要跑一次解码器前向，7 维动作就是 7 次，一个 K 步的块就是 $K \times D$ 次。OFT 的改法：往输入里放一组“空动作嵌入”（可以理解占位 token），把解码器的因果注意力掩码换成双向注意力，一次前向同时得到所有位置的隐状态。

动作块。有了并行解码，预测 K 步只需多放 K 组占位 token，吞吐提高 K 倍而延迟几乎不变。论文在 LIBERO 用 K=8，在 ALOHA 用 K=25（对应 25 Hz 控制下的一秒）。

连续输出 + L1。占位 token 的最终隐状态不再经过 softmax 选箱，而是送进一个小 MLP 直接回归归一化后的连续动作，用 L1 loss 训练：

$$\hat{a}_{t+k} = g_{\phi}(h_{t,k}^{\text{act}}), \quad \mathcal{L}_{\text{OFT}} = \frac{1}{Kd_a} \sum_{k=0}^{K-1} \|\hat{a}_{t+k} - a_{t+k}\|_1$$

作者也试了 diffusion 目标，发现他们的设定下 L1 回归成功率相当、而训练和推理都简单得多。

FiLM (OFT+)。在 ALOHA 上加了腕部相机后，策略容易只看图像、忽略语言（比如抓到勺子后不管指令让它做什么都倒进同一个碗）。解决办法是在视觉特征上做特征级线性调制（FiLM）：用语言嵌入生成缩放和偏置，强迫视觉通路携带指令信息。带 FiLM 的版本叫 OpenVLA-OFT+。

4.3 结果

- LIBERO 四套件平均从原版 OpenVLA 的 76.5% 提到 97.1%，当时的最好结果。
- 动作生成吞吐提升 26 倍；在 ALOHA 上用 25 步的块做到 43 倍，能跟上 25 Hz 控制。
- 真机 ALOHA 双臂任务上超过按默认配方微调的 π_0 与 RDT-1B，也超过从零训练的 ACT 与 Diffusion Policy。

4.4 在 StarVLA 中

`starVLA/model/framework/VLM4A/QwenOFT.py`：在指令末尾拼一句形如“预测接下来 N 个动作”的提示并附 N 个 占位字符，取最后一层隐状态中这 N 个位置，送入两层残差 MLP（MLPResNet， $H \rightarrow 2H \rightarrow D$ ），loss 是带 mask 的 L1： $(|\hat{a} - a| \cdot m) \cdot \text{sum}() / m \cdot \text{sum}()$ 。单次前向，无迭代。这是 StarVLA- α 的默认配置，也是 VLAct 的默认下游头。

4.5 一句话记住

OFT 证明：给一个足够强的 VLM，最简单的“占位 token + MLP + L1”就能拿到顶尖成功率和最快的推理。它的边界在表达能力——输出是点估计，多解任务上会取平均；但 StarVLA- α 和 VLAct 的数据表明，在数据充足或低数据 + 强扰动两种设定下，它都至少不输给生成式头（RoboTwin Base/Random 下 OFT 41.5 vs GR00T 22.9、PI 23.7）。

5. PI (π_0)：用 flow matching 生成动作块（12 分钟）

讲述提示：这一节的难点是 flow matching 本身。不要从概率论讲，用“把一团噪声搬到一段轨迹”的图像讲：起点是高斯噪声，终点是真实动作块，网络学的是每个中间位置该往哪个方向走多远。

5.1 它要解决的问题

π_0 的目标是通用的灵巧操作：叠衣服、收拾桌子、装袋，这些任务需要 50 Hz 的高频控制和长达几十步的动作块，而且同一状态下合理的动作不止一种。离散 token 有信息损失，点估计回归会把多解平均掉，所以作者选择了生成式建模——具体是 flow matching，它是 diffusion 的一个更简洁的变体。

5.2 flow matching 的最小版本

把真实动作块记为 A ，噪声记为 ϵ 。在两者之间画一条直线，用 $\tau \in [0, 1]$ 标记位置：

$$A^\tau = \tau A + (1 - \tau) \epsilon$$

$\tau = 0$ 是纯噪声， $\tau = 1$ 是真实动作。沿这条直线走，每单位 τ 的位移是常数：

$$u = A - \epsilon$$

训练一个网络 $v_\theta(A^\tau, \tau; o_t)$ 去预测这个位移（“速度场”），损失就是均方误差：

$$\mathcal{L}_{\text{PI}} = \mathbb{E}_{A, \epsilon, \tau} \left[\|v_\theta(A^\tau, \tau; o_t) - (A - \epsilon)\|_2^2 \right]$$

推理时从噪声出发，沿学到的速度场走 N 步（前向欧拉）：

$$A^{\tau+\delta} = A^\tau + \delta v_\theta(A^\tau, \tau; o_t)$$

π_0 用 N=10 步。两个实践细节：(i) 训练时 τ 不是均匀采样，而是从一个偏向小 τ （更接近噪声）的 Beta 分布采样，让网络多练“从很糟的起点纠正”这一段；(ii) 动作块整体一起去噪，块内各步之间用双向注意力，所以输出的 H 步动作是相互协调的。

和 diffusion 的关系：都是“从噪声迭代生成”，flow matching 用直线路径和速度目标，公式更简单、步数更少，StarVLA 里 PI 与 GR00T 两个头共用这一套目标。

5.3 动作专家：第二组权重

flow matching 需要一个网络吃噪声动作 A^τ 、时间 τ 和观测条件。 π_0 的做法是在 VLM 旁边加一个**动作专家**（action expert）：它和 VLM 走同一个 transformer 的注意力，但拥有**独立的一套权重**，只处理机器人状态和动作 token；图像和文本 token 仍由 VLM 的权重处理。这相当于一个两成员的混合专家

(MoE)：按 token 类型路由到不同的权重。作者的理由是动作 token 的统计性质与语言 token 差异太大，混用一套权重会拖累两边。

规模：VLM 骨干用 PaliGemma (3B, 互联网图文预训练)，动作专家从零初始化约 3 亿参数，总计 3.3B。动作块 H=50，控制频率最高 50 Hz。

5.4 训练配方

预训练数据约 1 万小时的自有灵巧操作数据 (7 种机器人配置、68 个任务) 加上开源 OXE。作者强调两阶段：预训练数据要"广而杂"，让模型见过各种状态并学会从错误里恢复；后训练数据要"精而顺"，教模型把一个任务做得流畅。两者缺一不可——只有精数据的模型不会纠错，只有杂数据的模型做不利索。

5.5 在 StarVLA 中

starVLA/model/framework/VLM4A/QwenPI_v3.py：不复用 PaliGemma，而是把 Qwen3-VL 最后 36 层的隐状态各经一个 LayerNorm + Linear 投影到 1024 维，逐层 cross-attention 到一个 36 层的 DiT (全 cross-attention，不交错 self-attention)；本体状态量化成 256 桶写进文本；flow matching 目标同上， τ 用 Beta(1.5, 1) 变换采样；推理 4 步欧拉。参数量 4.44B (骨干) + 5.39 亿 (DiT) + 0.95 亿 (投影)。

5.6 一句话记住

π_0 把"动作生成"从分类或回归换成了生成式建模，能表达多模态、高频、长块的动作分布，代价是每次推理要跑 N 步去噪、并且多出一套动作专家的权重。VLAAct 的表 8 显示：只用 OFT 单头预训练的骨干，换 PI 头微调反而低于从零 (60.5 $\rightarrow$ 55.1)，说明生成式头对骨干表征的"读法"和回归头不同——这是多头共监督的动机。

6. GR00T：双系统里的 flow matching (10 分钟)

讲述提示：GR00T 和 π_0 都是 flow matching，讲清楚"差别在哪"比重复公式更重要。板书两栏： π_0 的动作专家与 VLM 共享注意力、同频运行；GR00T 的 DiT 是独立模块、通过 cross-attention 读 VLM 特征、可以跑得比 VLM 快得多。

6.1 它要解决的问题

GR00T N1 是面向人形机器人的开源基础模型。人形本体维度高、需要高频闭环控制，而 VLM 的推理速度跟不上。作者借用"快思考 / 慢思考"的比喻做了一个双系统：**System 2** 是 VLM，负责理解场景和指令，约 10 Hz；**System 1** 是一个 diffusion transformer (DiT)，负责实时生成动作，约 120 Hz。两者联合端端训练，但推理时可以异步。

6.2 核心机制

System 2. 用 NVIDIA 的 Eagle-2 VLM (SmolLM2 语言模型 + SigLIP-2 图像编码器)。一个重要的工程发现：不用最后一层，而用**第 12 层**的隐状态作为条件——中间层保留了更多空间和视觉细节，末层过于语义化。公开的 GR00T-N1-2B 总参数 2.2B，其中 VLM 1.34B。

System 1. 一个自带适应层归一化 (adaLN，用它注入去噪时间步 τ) 的 DiT。块内交替两种注意力：self-attention 在"噪声动作 token + 本体状态 token"上做，cross-attention 去读 System 2 输出的视觉-语言 token。训练目标仍是 flow matching：

$$\mathcal{L}_{\text{GR00T}} = \mathbb{E} \left[\left\| v_{\theta}(A^{\tau}, \tau; H_t, s_t, e) - (A - \epsilon) \right\|_2^2 \right]$$

多出来的条件 s_t (本体感受状态) 和 e (本体标识) 是它与 π_0 的关键区别之一。

跨本体的编解码。 不同机器人的状态和动作维度不同，GR00T 给每个本体配一对小 MLP：状态编码器把 s_t 投到统一维度，动作解码器把 DiT 输出映射回该本体的原生动作空间。DiT 主体是共享的。动作块 16 步，推理 4 步去噪，在 L40 上一块动作 63.9 ms。

数据金字塔与潜动作。 底层是网页数据和人类第一视角视频，中层是合成的"神经轨迹" (用视频生成模型扩充) 与仿真数据，顶层是真机数据。没有动作标签的视频怎么用？训练一个 VQ-VAE，从相邻两帧 (x_t, x_{t+H}) 学出一个"潜动作" z_t ，把它当作一种额外的本体 (LAPA)，用同样的 flow matching 目标训练。这样人类视频也能进同一个训练循环。

6.3 在 StarVLA 中

starVLA/model/framework/VLM4A/QwenGR00T.py：用 Qwen3-VL **最后一层**隐状态作为 cross-attention 的 K/V (StarVLA 没有复刻"取第 12 层")；DiT-B (768 维、12 头) 16 层，交替 self/cross；状态经 MLP 编码后拼在序列最前；flow matching 目标同 PI，4 步欧拉，训练时每个样本重复 8 次不同噪声 (repeated_diffusion_steps=8)。VLAAct 的多头版本里 GR00T 头以 state_dim: 0 构建、状态改走文本，以便与其他头共用混合本体的 batch。

6.4 π_0 与 GR00T 的差别 (听众最常问)

	π_0 的动作专家	GR00T 的 System 1
与 VLM 的耦合	同一注意力、独立权重 (MoE 式)	独立 DiT，通过 cross-attention 读 VLM 特征
取哪层特征	与 VLM 逐层交互	单层 (论文第 12 层；StarVLA 用末层)
运行频率	与 VLM 同步	可异步，System 1 远快于 System 2
状态与本体	状态 token 进专家	状态 + 本体标识经本体特定 MLP 进 DiT
块长 / 步数	H=50, 10 步	H=16, 4 步
跨本体	统一填充动作空间	每本体一对编解码 MLP

6.5 一句话记住

GR00T 把 flow matching 放进一个可以独立高频运行的模块，并用本体特定的编解码 MLP 处理多本体，同时把没有动作标签的视频通过潜动作纳入训练。StarVLA- α 表 2 里它与 OFT、 π 在数据充足时相当；VLAct 表 9 里它是多头预训练受益最大的头 (+4.3)。

7. 横向对比 (8 分钟)

讲述提示：这张表是整场讲解的落点。让听众能在"接口 / 目标 / 推理 / 表达力 / 代码类"五个维度上一眼分清四个头。

维度	FAST	OFT	PI (π_0)	GR00T
与 VLM 的接口	复用词表，动作 = token	占位 token 的隐状态 $\rightarrow$ MLP	独立权重的动作专家	独立 DiT，cross-attention 读 VLM
动作表示	离散 (DCT + BPE)	连续，点估计	连续，分布	连续，分布
训练目标	next-token 交叉熵	L1	flow matching MSE	flow matching MSE (+ 状态、本体条件)
推理	自回归解码几十个 token	一次前向	N 步欧拉 (π_0 10 步；StarVLA 4 步)	N 步欧拉 (4 步)
新增参数	0	一个小 MLP	~ 3 亿 (π_0) / 5.4 亿 + 0.95 亿 (StarVLA PI_v3)	DiT-B 16 层 + 本体 MLP
多模态动作分布	可以 (分类分布)	不能	可以	可以
高频长块	依赖压缩比	天然支持	天然支持	天然支持
跨本体	统一 tokenizer (FAST+)	零填充 / mask	统一填充	本体特定 MLP
StarVLA 类	QwenFast	QwenOFT	QwenPI_v3	QwenGR00T

同一骨干、同一数据下的数字 (StarVLA- α 表 2, Qwen3-VL-4B)：

	LIBERO avg	WidowX	RoboTwin clean*	RoboCasa-GR1
FAST	97.8	35.6	72.5	45.0
OFT (MLP)	98.8	64.6	88.2	53.8
PI	98.1	65.9	88.1	48.9
GR00T	98.7	65.3	88.0	52.8

结论有两层：离散头明显落后；三种连续头在数据充足时相当。但 VLAct 补了两条边界：低数据 + 强扰动下回归头领生成式头 18 个点 (RoboTwin Base/Random：OFT 41.5 vs GR00T 22.9 / PI 23.7)；单头预训练会让骨干"锁进"该头的解码几何，换头微调反而低于从零 (decoder lock-in)，三头共监督才能得到对头不敏感的骨干。

8. 选型与常见问题 (5 分钟)

- Q：我只有一个机械臂、几百条示教，用哪个？ OFT。实现最小、推理最快，成功率不输生成式头。
- Q：任务有明显的多解 (比如从哪一侧抓)，OFT 会怎样？ 回归头输出条件均值，可能落在两个解之间。这时换 PI 或 GR00T，它们输出的是分布中的一个样本。
- Q：想复用别人的 VLM checkpoint、不想加任何模块？ FAST 是唯一不改架构的方案，接受推理慢和 15-30 个点的代价。
- Q：多本体、需要状态输入、要把运动模块单独部署？ GR00T。状态和本体标识进 System 1，DiT 可以独立于 VLM 更新和加速。
- Q：我在做持续预训练，要交付一个骨干给别人用？ 至少两个连续头一起监督 (VLAct 表 8：OFT 单头让 PI 微调 -5.4，加 GR00T 头后 +2.6)。
- Q：flow matching 和 diffusion 到底什么关系？ 同一族方法。diffusion 学"去噪" (预测噪声)，flow matching 学"位移" (预测 $\mathbf{A} - \epsilon$)，路径是直线，公式和采样都更简单。在动作生成里两者效果接近，flow matching 步数更少。
- Q：为什么 GR00T 取 VLM 第 12 层而不是最后一层？ 中间层保留更多空间与视觉细节。这与 VLAct "冻结下半层保护低层视觉表征"的发现是同一件事的两面。

9. 阅读顺序

- 先读 OpenVLA-OFT (中译) 第 IV 节：三组设计决策的对照实验最直观。
- 再读 FAST (中译) 第 IV-V 节：玩具实验 + 算法 1。
- π_0 第 IV 节 (模型) 与附录 B (flow matching 细节)：只看公式和图 3 即可；预训练配方部分留到做数据时再读。
- GR00T N1 (中译) 第 2 节 (模型) 与 3.1 (数据金字塔、潜动作)。
- 回到 StarVLA 的四个框架文件对照代码 (02 · 代码库解析 第 4 章)，再看 05 · 动作头与动作表示 的证据汇总。

附：公式速查卡

头	前向	损失
FAST	$\mathbf{z}_{1:M} = \text{BPE}(\text{round}(\gamma \text{DCT}(\text{norm}(\mathbf{A}))))$	$-\sum_m \log p_\theta(\mathbf{z}_m \mid \mathbf{H}_t, \mathbf{z}_{<m})$
OFT	$\hat{\mathbf{a}}_{t+k} = g_\phi(\mathbf{h}_{t,k}^{\text{act}})$	$\frac{1}{Kd_a} \sum_k \hat{\mathbf{a}}_{t+k} - \mathbf{a}_{t+k} _1$

头	前向	损失
PI	$\mathbf{A}^\tau = \tau \mathbf{A} + (1 - \tau) \epsilon$; 推理 $\mathbf{A} \leftarrow \mathbf{A} + \delta \mathbf{v}_\theta$	$\mathbb{E}[\mathbf{v}_\theta(\mathbf{A}^\tau, \tau; \mathbf{o}_t) - (\mathbf{A} - \epsilon)]_2^2$
GR00T	同 PI, 条件多了 $\mathbf{s}_t, \mathbf{e}$; 本体 MLP 编解码	$\mathbb{E}[\mathbf{v}_\theta(\mathbf{A}^\tau, \tau; \mathbf{H}_t, \mathbf{s}_t, \mathbf{e}) - (\mathbf{A} - \epsilon)]_2^2$

记号约定见 §2; π_0 论文原文用 $\tau \mathbf{A} + (1 - \tau) \epsilon$, VLAct 论文写成 $(1 - \tau) \epsilon + \tau \mathbf{A}$, 是同一条直线。

09 · EventVLA 解读：给 StarVLA-OFT 装上稀疏视觉证据记忆

项目	内容
论文	EventVLA: Event-Driven Visual Evidence Memory for Long-Horizon Vision-Language-Action Policies
arXiv	2606.20092, 2026-06 (CC BY 4.0)
代码	官方 InternRobotics/EventVLA ；本仓库以子模块收录分支 asimfish/EventVLA （ <code>code/EventVLA/</code> ，含 RoboTwin-MeM 基准）
模型 / 数据	HF ganlinyang/EventVLA 、 HF RoboTwin-MeM
基础模型	StarVLA 的 QwenOFT（Qwen3-VL + MLP 回归头）；checkpoint 布局即 StarVLA 格式
本仓库文件	英文 PDF · 中文 PDF

0. 一句话

标准 VLA 是马尔可夫策略：只看当前帧，一旦任务关键线索被遮挡或消失就无从决策。EventVLA 在 StarVLA-OFT 上加了一个**稀疏视觉证据记忆**：两条规则锚点（初始帧 + 最近 K 帧）负责场景布局与运动连续性，一个与动作头并联的轻量**关键帧证据记忆（KEM）**头从同一份隐状态预测"未来 H 步里哪一步会是关键事件"，命中就把那一帧原图写进一个有界 FIFO 缓冲。在作者新建的 RoboTwin-MeM 记忆基准上，基线 QwenOFT 3.8%，只加锚点 18.0%，加 KEM 后 75.2%；真机 ARX 双臂四个记忆任务 60–90%。它直接回应了本仓库路线图 D1 与基准报告 §5.4 指出的"Memory 是所有 StarVLA 系模型的死角"。

1. 问题：非马尔可夫的操作任务

作者把长程操作形式化为非马尔可夫决策过程： $\boldsymbol{a}_t = \pi(\boldsymbol{o}_t, \boldsymbol{M}_{t-1}, \boldsymbol{\ell})$ ，多出来的 $\boldsymbol{M}_{t-1}$ 是一个外部视觉记忆缓冲。为什么需要它？典型场景：掀开盖子看一眼里面是什么颜色的方块再盖回去，之后要按颜色顺序操作——关键证据只短暂出现过一次。RoboDojo 榜单上几乎所有策略在 Memory 维度接近零（VLAct 0.66 / 0.56，见 01 §4.2），就是这类任务。

作者把已有的记忆方案分成三类并各指出短板：双系统（高层 VLM 规划 + 低层控制）延迟高、误差会传播；循环 / 压缩记忆有信息瓶颈；无选择的历史缓冲把大量冗余帧塞进上下文。EventVLA 的立场是：**记忆应当稀疏、按事件写入，并且由策略自己决定写什么。**

2. 方法

2.1 两条规则锚点（Visual Anchors）

$\boldsymbol{A}_t = \{\boldsymbol{o}_0\} \cup \{\boldsymbol{o}_{t-K}, \dots, \boldsymbol{o}_{t-1}\}$ 。初始帧 $\boldsymbol{o}_0$ 是永久的空间锚，记住东西被挪动前的布局；最近 K 帧提供运动与任务进度线索。这部分不需要学习。消融显示两者都不可缺：在 RMBench 上去掉初始帧掉到 33.7%，去掉短期窗掉到 23.8%（全量 67.8%）。

2.2 关键帧证据记忆（KEM）头

- 输入**：与动作头完全相同的最后一层隐状态 $\boldsymbol{h}_t \in \mathbb{R}^{H \times d}$ （H 为动作块长度）。这些位置本来就编码了"接下来 H 步要做什么"，所以 KEM 天然带有对未来执行计划的预判。
- 输出**： $\hat{\boldsymbol{p}}_t = \sigma(\text{MLP}(\boldsymbol{h}_t)) \in [0, 1]^H$ ，第 i 个分量是"未来第 i 步是关键帧"的概率。按块预测而不是逐步分类，是因为关键事件可能在块中间一闪而过，逐步分类器会错过。
- 写入**：某个 $\hat{\boldsymbol{p}}_t^i \geq \tau_{\text{commit}}$ 时，把 $t + i$ 时刻的原图写进事件缓冲 $\boldsymbol{E}_t$ ；缓冲有上限 N_{max} ，FIFO 淘汰。推理时先用一维 NMS + 时间冷却把连续的高概率段压成离散的写入事件，避免同一事件写入多帧。
- 拼接**： $\boldsymbol{o}_t$ 、 $\boldsymbol{A}_t$ 、 $\boldsymbol{E}_{t-1}$ 按时间顺序排成一个图像序列送进 VLM。记忆就是"多几张图"，架构不变。

2.3 训练

- 关键帧的真值来自一条离线的 Qwen3-VL 自动标注流水线，不需要人工标时间戳。
- 用时间平滑的软标签和按序列平均的 BCE 监督 KEM： $\mathcal{L} = \mathcal{L}_{\text{action}} + \lambda \mathcal{L}_{\text{kem}}$ ，与动作损失端到端联合优化。
- 训练时的记忆内容从"真值关键帧"逐步过渡到"模型自己预测的关键帧"（teacher-to-student 课程），弥合训练与推理的分布差。

3. RoboTwin-MeM 基准

建在 RoboTwin 2.0 上的 8 个双臂任务，每个任务用参数 n （1–5）显式标出**成功必须记住几个转瞬即逝的关键帧**：Rearrange Blocks Hard（n=1）、Put Back Block Hard（2）、Pick Objects in Order（3）、Pick the Unhidden Block（3）、Cover Blocks Hard（4）、Reproduce Route（4）、Find Seal and Seal Stamp（1–4）、Press Button Keyframe（2–5）。每回合平均 430–1544 步。这是目前 StarVLA 生态里唯一一把"要记多少"作为可控变量的基准，正是 06 · 基准生态 §5.4 说缺的东西。

4. 结果

4.1 RMBench（记忆只依赖持久布局与固定动作风格）

方法	均值
QwenOFT（基线，无记忆）	5.6
$\pi 0.5$	10.4
Mem-0（双系统）	42.0
MemoryVLA（QwenOFT 版）	41.7
EventVLA，仅锚点	67.8

只靠两条规则锚点就是最好成绩，说明这类基准考的是"记住布局"，不是中间事件。

4.2 RoboTwin-MeM（考中间证据）

方法	8 任务均值
QwenOFT	3.8
$\pi 0.5$	7.8
MemER / Mem-0	10.5 / 0.0
MemoryVLA（QwenOFT 版）	10.8
EventVLA 仅锚点	18.0
EventVLA 锚点 + KEM	75.2

消融里最有信息量的几行：把原图缓冲换成隐式记忆库掉到 24.9（说明"存原图、让 VLM 重新看"比存特征有效）；硬标签替代软标签掉到 48.8；去掉 NMS 掉到 53.4；缓冲上限 $N_{\max} = 2$ 掉到 32.0；把动作块从默认缩到 15 步掉到 13.6——KEM 的"预见"距离依赖足够长的块。

4.3 不伤常规任务，真机可用

RoboTwin 2.0 常规（马尔可夫）任务上，EventVLA 相对 QwenOFT 基线 Easy 80.0 $\rightarrow$ 83.8、Hard 78.0 $\rightarrow$ 81.6。真机 ARX ACONE 双臂四个记忆任务（找被藏起的方块、按读到的次数抓取、按指示顺序抓取），每任务 20 次：EventVLA 90 / 60 / 90 / 75， $\pi 0.5$ 0–10，记忆增强基线 π MEM 30–50。

5. 评价

做得好的地方

- 记忆机制与 StarVLA 的接口完全兼容：多一个并联的 MLP 头、多几张输入图，训练循环和 OFT 头不变。这与 VLA_{ct} 的多头共监督在结构上同源——都是"在同一份隐状态上再挂一个头"，只是监督信号不同（关键帧概率 vs 另一种动作解码）。
- 把"记忆写什么"变成可学习、可消融的对象，并且用一个参数化的基准把"要记几件事"作为自变量。
- 存原图而非特征的选择被消融支持，也解释了为什么 RoboDojo 上单帧模型的 Memory 分接近零：不是骨干不够强，是根本没看到需要的帧。

局限与未回答的问题

- 缓冲有界（作者自述）：超过 10 分钟、事件密集的任务会饱和并淘汰早期证据；层次化或压缩记忆是下一步。
- 关键帧真值依赖 Qwen3-VL 离线标注，标注质量的上限决定 KEM 的上限；论文没有报告标注错误率。
- 只在 OFT 头上验证。KEM 与 flow-matching 头（PI / GR00T）共存时，隐状态 h_t 的语义是否仍然"包含未来计划"，需要实验。
- 与 VLA_{ct} 配方的叠加未做：VLA_{ct} 是持续预训练阶段的骨干配方，EventVLA 是下游架构改动，两者正交，可以直接组合（见 §6）。

6. 与本仓库其他材料的关系

- 路线图 D1（记忆与长程）**：EventVLA 就是 D1 路线 (i)"压缩历史"之外的第四条路——稀疏事件记忆；RoboTwin-MeM 是 D1 应当采用的评测。已在 [07 · 路线图 D1](#) 中更新。
- 基准 §5.4**：RoboTwin-MeM（8 任务、n 可控）与 RMBench 填上了"没有好记忆基准"的空白，已在 [06](#) 中补注。
- 可直接做的实验**：用 VLA_{ct} 骨干替换 EventVLA 的 Qwen3-VL 初始化，其余不变，看 RoboTwin-MeM 是否叠加提升；以及把 KEM 头加进 [code/vlact_ext](#) 的 QwenMultiHead，检验多头共监督下 KEM 的预测是否更准。
- 代码入口**：[code/EventVLA/EventVLA/](#)（模型与训练）、[code/EventVLA/RoboTwin-Mem/](#)（基准）；模型 checkpoint 与 StarVLA [from_pretrained](#) 兼容。